

Előadás_#01.

1. Bevezetés

Minden magára kicsit is adó tudomány vizsgálatát, vagy legalább a tudomány látszatát kelteni szándékozó okoskodást azzal szokás kezdeni, hogy „...már az ókori görögök tapasztalatai szerint is...” vagy „...már az arab matematika hajnalán is...” vagy „...már az egyiptomi fáraók idejében is...”. Az operációs rendszerek esetében ez a sablon megbukik, mert az általunk vizsgálandó témakör nem tekint ilyen dicső múltra vissza. Az ókorból talán csak a praktikus és logikus megoldások keresésének vágya maradt ránk.

A technika gyors fejlődésének következtében a 30-40 vagy akár csak az 5-10 évvel ezelőtti történetek is roppant távolinak, és (lássuk be) korszerűtlennek tűnnek. Persze az előző állítás sem abszolút igazság, hiszen a legalapvetőbb belső mechanizmusok, ütemezések, algoritmusok tekintetében sok – informatikai szempontból matuzsálemi kort megért, és azóta is vígan működő – régi, jól bejáratott megoldással találkozunk.

Az operációs rendszer – mint a működés/működtetés alapvető része – minden számítógépes rendszerben megtalálható. Operációs rendszer nélkül a hardver egy átlag felhasználó számára kb. annyit ér, mint az utas számára egy helikopter pilóta nélkül.

Az operációs rendszer az a program, amely közvetítőként működik a felhasználó, valamint a felhasználó által futtatott szoftverek és a számítógép hardvere között. Elsődleges cél az, hogy a felhasználó egyszerűen, zökkenőmentesen és a megfelelő biztonsági szinten legyen képes futtatni a saját programját. A másodlagos cél a számítógép hardverének minél hatékonyabb használata.

A számítógépek hardver felépítéséről, azon belül is a processzor, az alaplap, a különféle tárolóeszközök és perifériák működéséről a „Számítógépek Felépítése” című tantárgy keretein belül már foglalkoztunk. A lényeg, hogy ne felejtsük el: egy informatikai értelemben hardvernek nevezett eszköz (komplex berendezés) működtetéséhez mindenképpen valamilyen szoftverre van szükségünk.

Az operációs rendszer feladata tehát az, hogy a rendelkezésre álló szoftveres és hardveres erőforrásokat (legyen az fizikai, vagy logikai erőforrás), a felhasználó (vagy felhasználók) által futtatott programok részére optimálisan és hatékonyan rendelkezésére bocsájtja.

Az operációs rendszerek feladataikat és funkcióikat jellemzően réteges szerkezetben valósítják meg. Az egyes rétegek csak a közvetlenül alatta vagy felette elhelyezkedő réteggel kommunikálnak. (Hasonló logikával felépített modellek fognak visszaköszönni majd a „Számítógéphálózatok” tantárgyban is.)

SZOFTVER	Alkalmazások (Szövegszerkesztő, Adatbázis kezelő, Webböngésző, stb.)	
	Magas szintű nyelvek (C++, Pascal, Basic)	
	Alkalmazási Interfész (API) Alacsony szintű nyelvek (Assembly, Gépi kód)	
	OPERÁCIÓS RENDSZER	user mód ----- kernel mód
HARDVER	Buszrendszer, Memória, Háttértár, Perifériák	
	Processzor (ALU, regiszterek)	
	Logikai kapuáramkörök	

A táblázatban a BIOS illetve az UEFI nem szerepel, de nem szabad arról megfeledkezni, hogy a BIOS illetve az UEFI is az operációs rendszer és a hardver között teremt kapcsolatot. A BIOS-szal illetve az UEFI-val a „Számítógépek Felépítése” című tantárgy keretein belül már foglalkoztunk.

2. Az operációs rendszerek fejlődése

Az operációs rendszerek evolúciója nem kezelhető a hardver komponensek fejlődésétől, változásától függetlenül. A hardverek és szoftverek fejlődése általában jótékony hatással vannak egymásra, de persze akadnak kivételek is. Ilyen például a Windows Vista, amely operációs rendszer fejlesztői abból indultak ki, hogy felesleges a kódot egy szinten túl optimalizálni (azaz nem baj, ha lassú a rendszer), hiszen a hardver elemek gyorsan fejlődnek, valószínűleg az áruk is csökken, és várhatóan annyit gyorsulnak, hogy kompenzálni fogják a szoftver lassúságát.

Ok és okozati viszonyban, másképpen fogalmazva „tyúk és a tojás problémája” szintjén könnyen belátható, hogy csak egy hardver megjelenése után lehet a hozzá szánt szoftvereket elkezdeni fejleszteni, optimalizálni.

A hardverek fejlődése során az egyik legjelentősebb lépés a buszrendszerek megjelenés után a specializálódás volt. Eleinte minden aritmetikát a CPU számolt, majd megjelentek az egyedi vezérlők (Controller), melyek csak egy adott periféria működéséért voltak felelősek. Megjelentek továbbá a megszakítások (IRQ, NMI), és megjelent a perifériák közvetlen memória elérése is (DMA).

3. Az operációs rendszerek típusai

Az első számítógépek elnevezésükön túl nem sok hasonlóságot mutattak a mai számítógépekkel. Nem létezett sem a billentyűzet, sem az egér, sem az alfanumerikus vagy grafikus kijelző. A futtatni kívánt kódot egy manuális telefonközpont kapcsolótáblájához hasonló dugaszolós táblán lehetett közölni a géppel. A kód lefutási idejét legfeljebb tapasztalati állandók segítségével lehetett megsaccolni. Az eredmény – ami jellemzően egy számérték volt – helyiérték lámpák segítségével volt leolvasható. Komoly előrelépés volt a lyukkártya megjelenése, amely egy részt adatbevitelre, másrészt az eredmény megjelenítésére is használatos volt a nyomtatás megjelenéséig.

A köteget (batch) feldolgozás a programozó mellett még egy operátor jelenlétét is igényelte. Az operátor (mivel programoznia nem kellett) folyamatosan képes volt a számítógépet újabb és újabb lyukkártyákkal ellátni, ezzel csökkentve a holtidőt, viszont a programozó így már csak a kód írásával, azaz a program megalkotásával kellett hogy foglalkozzon. A háttértárak fejlődésével, a mágnesszalag megjelenésével lehetőség nyílt a programok lyukkártyáról szalagra másolására, tovább növelve ezzel a hatékonyságot. A „teljes kihasználtságot” az egyszerű monitor (Resident Monitor) kifejlesztése jelentette, amikor egy kód lefutása és az eredmények tárolása után a következő kód automatikusan képes volt elindulni. A „resident” szó arra utal, hogy ennek a programnak az aktuális programokkal egy időben kellett a memóriában lennie, azaz a memóriát két részre kellett osztani.

A perifériák sebessége ma is lényegesen lassabb a processzor sebességénél, így már az informatika hajnalán elkezdődött a megoldás keresése arra problémára, hogy a processzor lehetőleg ne a lassú perifériákra való várakozással töltse az idejét.

A spooling technika (Simultaneous Peripheral Operation On-Line / Spool → orsó) a mágneses tárat, mint nagyméretű puffert használja, így egyszerre nemcsak egy, hanem több kódot (azaz programot) is lemezre tölt és tárol. Ez a megoldás lényegében az off-line perifériás műveletek egy gépen való megvalósítása. Így lehetővé válik a perifériás és processzor műveletek szétválasztása oly módon, hogy több munka is átlapolódhat. Például az egyik munka végrehajtásával egy időben egy másik munka beolvasása is folyhat, valamint egy harmadik munka eredményeinek rögzítése is történhet. Ezáltal a perifériák és a processzor kihasználtsága egyaránt jelentősen javult.

Gyakorlatilag ezzel megnyílt az út (azaz modell már létezett) a folyamatok illetve a szálak megjelenéséhez.

4. Multiprogramozott rendszerek, Időosztásos rendszerek

A multiprogramozás megjelenése, azaz több programkód együttes futása megnövelte a processzor kihasználtságát. Így amíg az egyik program például egy lassú perifériával cserél adatot, addig a processzor egy másik programot képes futtatni. További előnyt jelentett a RAM (Random Access Memory) szervezésű táruk illetve memóriák megjelenése. A „random” szó jelentése: tetszőleges. Ez azt jelenti, hogy a korábbi kizárólag szekvenciális (azaz kötött sorrendű) elérést a tetszőleges sorrendű elérés váltotta fel. A multiprogramozás foka ebben a megközelítésben jellemezhető az adott pillanatban a memóriában jelenlévő és tetszőlegesen elérhető folyamatok számával.

A felhasználó szempontjából a hatékonyság így már képes volt megközelíteni a processzor teljesítőképességének körülbelül 90%-át. A felhasználó szempontjából a 100%-os kihasználtság elméletileg sem érhető el, hiszen a háttérfolyamatok, az egyes programok közötti váltás, a környezetváltás, a használatban lévő adatok mentése illetve visszaállítása is igényel erőforrást a processzortól.

Az időosztásos rendszerek első generációi arra voltak képesek, hogy több felhasználó igényeit (kódját) gyakorlatilag közel egyszerre befogadják, és mindenegyfelhasználó úgy érezhette, hogy a számítógép csak vele, vagyis csak az ő programjával foglalkozik. Megszületettek tehát a folyamatok és az ütemezés. Az ütemezés lehet idő alapú, vagy folyamat alapú. Első esetben egy időszel lejártaig fut a folyamat, utóbbi esetben a folyamat befejeződéséig.

A multiprogramozás lépései:

- A rendszer nyilvántartja és tárolja a futtatandó munkákat.
- A kiválasztott munka addig fut, amíg várakozni nem kényszerül, vagy esetleg be nem fejeződik.
- Az operációs rendszer feljegyzi a várakozás okát, majd kiválaszt egy másik futni képes munkát és azt elindítja.
- Ha a félbehagyott munka várakoztatása alatt a munka folytatásának feltételei teljesülnek, akkor amint lehetőség nyílik rá, a munka újra futtatásra kerül.

Az időosztásos (Time Sharing) rendszerek:

- Az időosztásos rendszerek közvetlen interaktív kommunikációt biztosítanak a felhasználó és a programja, valamint az operációs rendszer között.
- Adatok közvetlen elérésű (On-line) állományrendszerben tárolódnak.
- A felhasználó egy jellemzően lassú „periféria”, így például adatbevitel közben az operációs rendszer más tevékenységet végre tud hajtani.

- Gyors a reakció a parancsokra, a válaszidő (Response Time) kicsi, de cserébe a processzornak gyakran kell a programok között váltani.
- Több felhasználó egymástól függetlenül használja a gépet úgy, mintha mindenki egy saját gépen dolgozna.

5. Operációs rendszerek tipikus komponensei és jellemző felépítése

Az összetett funkcionalitás megköveteli a komplex feladatok részekre bontását.

A legjellemzőbb rendszer komponensek:

- Folyamatkezelő
- Központi tárkezelő (operatív tár)
- Állománykezelő (elsődleges háttértár)
- I/O rendszer kezelő
- Állománykezelő (másodlagos háttértár)
- Védelmi rendszer
- Hálózat kezelő
- Felhasználói felület kezelő

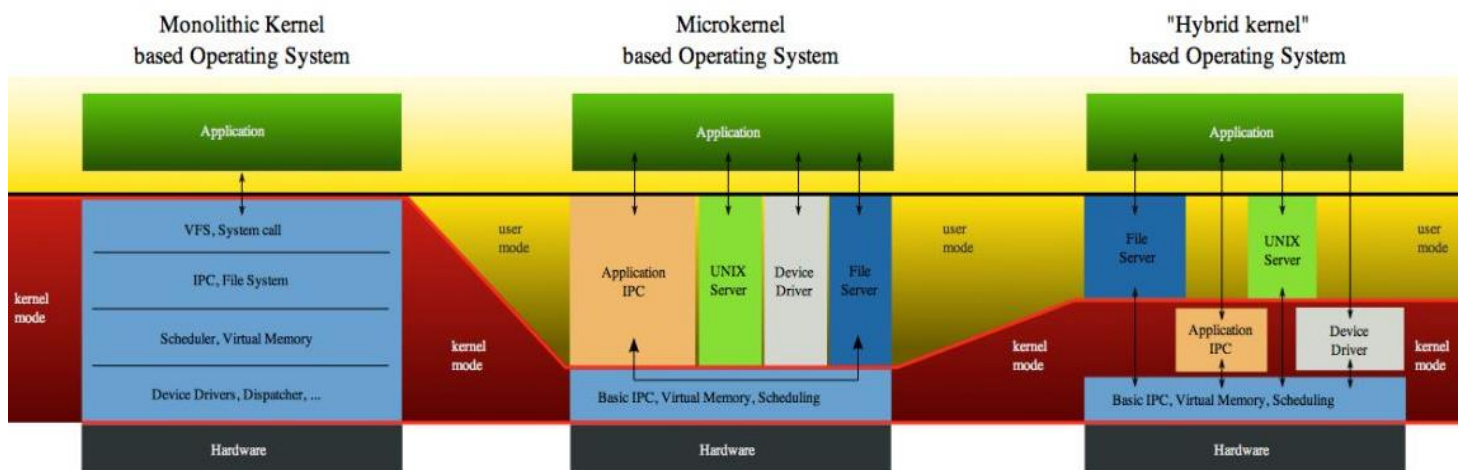
Az operációs rendszer alapja (vagy magja) a kernel. Maga a kernel a felhasználó számára láthatatlan program, hiszen a háttérben fut és a legalapvetőbb feladatok ellátása a feladata. Az operációs rendszerek különböző módon választják szét a kernel módban illetve a user módban futó komponenseket.

A kerneleknek négy fő kategóriáját különböztethetjük meg (illetve léteznek olyan programkörnyezetek is, melyek kernel nélkül futnak):

- A monolitikus kernelek gazdag és hatékony absztrakciókat biztosítanak az alattuk található hardver elemekhez. Jellemző az egyetlen nagy programból álló rendszermag, azaz nem a különálló, egymással különböző interfészekon keresztül kommunikáló programok összessége. (UNIX, Linux)
- A mikrokernelek egy kisméretű alapkészletet biztosítanak a hardver kezeléséhez, és számos alkalmazással – amiket „szervereknek” nevezünk – biztosítják a további, részletesebb funkcionalitást. A mikrokernelek azáltal, hogy az általuk nyújtott funkciók nagy részét felhasználói szintre helyezték egy plusz absztrakciós szintet biztosítanak. A mikrokernel az a minimális forma, amely csupán néhány olyan alapvető funkciót – rendszerhívást – valósít meg, ami nélkülözhetetlen operációs rendszer szolgáltatásainak realizálásához. Ilyen például a címtartomány menedzsment, szál ütemezés, folyamatok közti kommunikáció, virtuális memória kezelés. Ennek előnye,

hogy a felhasználói szinten futó programrészek hibáinak vagy működési zavarainak esetén azok nem veszélyeztetik magának a rendszermagának a működését, azaz a rendszer stabilitása nagymértékben nő. Hátránya viszont az, hogy minden új absztrakciós szint bevezetésével csökken a rendszer teljesítménye. Ezért előfordulhat, hogy egyes kritikus feladatokat (például nagyon gyors és pontos elérést igénylő hardver elemek kezelését) a rendszer nem képes kellő hatékonysággal megoldani. (AIX, Mac OS X)

- A hibrid kernel (vagy vegyes kernel) megoldásai hasonlóak a színtiszta mikrokernelnek, de több, részletesebb kódot tartalmaznak a kernelmagban, hogy nagyobb sebességet érjenek el. A „hibrid” elnevezés arra utal, hogy ezen kernelnek mind a monolitikus, mind a mikrokernel elveit és mechanizmusait alkalmazzák. Így például az üzenetcsere (message passing) és a „nem létfontosságú” kódok felhasználói szintre való áthelyezését is amellet, hogy néhány ilyen kód teljesítményi okokból mégis a kernelmagba kerül. (Windows NT, Linux, BeOS)
- Az exokernel (vagy rendszer rutinkönyvtárak) nem biztosítanak absztrakciókat vagy állandó rendszermagot, hanem egy – a futtatandó programok részére használható – rutinkönyvtárból állnak, ami kizárólag a hardver közvetlen vagy közvetett elérését biztosítja. Az exokernel elve körülbelül 20 éves múltra tekint vissza, de napi használatú gyakorlati alkalmazása még várat magára.



Összességében megállapíthatjuk, hogy a legelterjedtebb kiszolgálási mód kernel szinten is a „kliens – szerver” működés. Egy szerver jellemzően több kliens szinte egyidejű kiszolgálására is képes.

Előadás_#02.

1. Folyamatok

A multiprogramozott rendszerek előtt a tiszta szekvenciális működés volt a jellemző. Egy program (itt szándékosan nem folyamatnak nevezzük) futtatása elkezdődött, teljes egészében lefutott (eredményt képezett), majd megállt. Ezután indulhatott a következő program futása. Ütemezésre nem volt szükség, pusztán a sorrend ismerete elegendő volt.

A több folyamat (és ebből következően nyilván a több szál – amiről hamarosan szó lesz) párhuzamos megjelenését először az időosztásos (vagy időszeleteres) rendszerek (Time Sharing Systems) tették lehetővé, ahol egyszerre több felhasználó is bejelentkezhetett a rendszerbe. Alapvető kérdés volt a processzor (ami nyilván egymagos volt) idejének igazságos, de hatékony megosztása (azaz időszeletenkénti kiosztása) a felhasználók közt. Az itt szerzett tapasztalatok vezettek a klasszikus multiprogramozáshoz, amikor már egy felhasználó is a több, párhuzamos(nak érzékelt) folyamat futtatásának a lehetőségét kapta meg.

A folyamat (Process) a multiprogramozott operációs rendszerek alapfogalma. A folyamat tehát a számítási feladatok végrehajtásának alapegységét jelenti. Folyamaton így jellemzően a műveletek meghatározott sorrendben történő végrehajtását értjük. Az operációs rendszer működése folyamatok sorozata. Egy folyamat elkezdődik és a lehetséges állapotokat felvéve előbb utóbb be is fejeződik. A vonatkozó részművelet végrehajtása viszont csak akkor kezdődhet el, ha az előző részművelet végrehajtása már befejeződött.

A folyamat állapotainak egyszerűsített leírása:

- **Futásra kész**

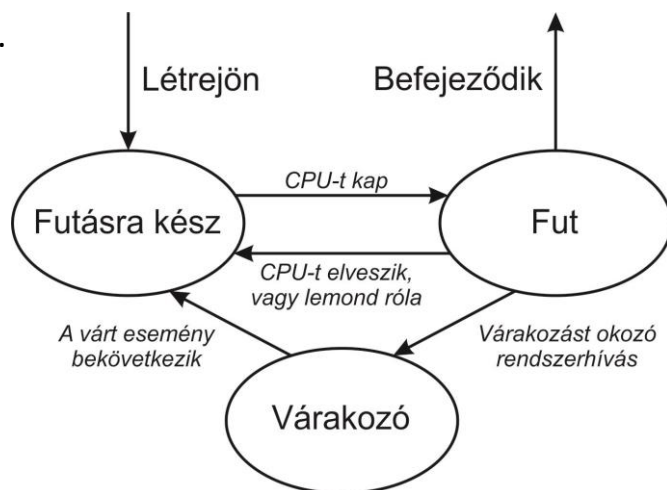
A folyamat már minden szükséges erőforrást birtokol, kivéve a CPU-t. (pont a CPU-ra vár)

- **Futó**

A folyamat fut, minden szükséges erőforrást birtokol, és ezen felül a CPU-t is birtokolja.

- **Várakozó**

A folyamat valamilyen feltétel teljesülésére vár.
(pl.: I/O művelet eredményére)



Multiprogramozott rendszerekben a processzor magok számának megfelelő számú folyamat képes tisztán egyidőben futni. (Nem szabad megfeledkezni a Hyper-threading technológiáról sem, ami nem a folyamatokkal, hanem a szálakkal kapcsolatos – így arról kicsit később lesz szó.) A nem futó, hanem a várakozó, illetve a futásra kész folyamatok számának pedig csak gyakorlati korlátja van, elvi korlátja nincs.

A folyamatkezelés (Process Management) az operációs rendszerek mindazon szolgáltatásainak, tevékenységeinek körét magába foglalja, amelyek a folyamatok végrehajtásával és az ehhez fűződő adminisztrálással kapcsolatosak.

Egy folyamat viszont közel sem biztos, hogy elkezdődik, és egy menetben simán le is fut, hiszen a lényeg pont az, hogy az operációs rendszer lehetőséget ad arra, hogy egyszerre több végrehajtás alatt álló folyamat között (időszelentenként vagy más prioritizálással) a CPU ütemező képes legyen váltani. [A CPU ütemező a rövid távú ütemező.] A módszer a váltások ellenére azt igyekszik (jellemzően sikerrel) "elhitetni" minden folyamattal, hogy ő az operációs rendszer – és ebből kifolyólag a hardver – egyedüli és osztatlan felhasználója.

Ne felejtjük tehát el, hogy normál működés közben, amikor egy folyamat megkapta és használja a CPU-t (CPU magot), biztosan el fog következni egy olyan időpillanat is, amikor nem egy másik folyamat igénye jelentkezik a CPU-ra, hanem maga az operációs rendszer igényli a CPU-t a saját belső mechanizmusainak a működtetéséhez. Ez pedig nem is ritkán jelentkező esemény, hanem az órajel alapján ciklikusan jelentkező esemény. Egy folyamat futása tehát megszakadhat egy másik (pl. magasabb prioritású) folyamat CPU igénye miatt, vagy az operációs rendszer CPU igénye miatt.

A megszakítás (Interrupt) az a műveletsorozat, amikor az éppen feldolgozás alatt lévő folyamat futásának félbehagyásával egy magasabb prioritású igény végrehajtásának indítása történik, oly módon elmenve az éppen futó folyamat környezetét (ez a fogalom hamarosan részletesen tárgyalásra kerül) egy átmeneti tárolóba, hogy az a későbbiekben folytatható legyen. A megszakítást, mint eszközt az teszi szükségessé, hogy a folyamatok végrehajtása során felléphetnek olyan események, melyek egyszerű, de hatékony kezelése csak az folyamat végrehajtás normális menetének átmeneti felfüggesztésével lehetséges.

[Magát a "megszakítás" elnevezést és terminológiát először az IBM használta, és terjesztette el. Természetesen gyorsan lettek követői, de például a Motorola a "kivétel" (exception) fogalmat használja. További érdekesség, hogy a Microsoft és a VAX egyes rendszerei mindkét fogalmat használják ráadásul nem is rokon értelmű kifejezésként, hanem mindkettőnek önálló értelmet adva.]

Kicsit közérthetőbben fogalmazva, az operációs rendszernek rugalmasan reagálnia kell a "külvilág" rendszert érintő eseményeire is. Ezt a kitűzött célt a megszakítás segítségével az operációs rendszer úgy éri el, hogy az éppen futó folyamatról a vezérlés ideiglenesen átadódik egy olyan másik folyamatra, amely képes kiszolgálni (kezelni) a bekövetkezett eseményt. A megszakítást kiszolgáló folyamat lefutása után pedig a megszakított folyamat végrehajtása a soron következő lépésétől kezdve folytatódik. Egy számítógép esetén például érzékelni kell a klaviatúra használatát, vagyis azt, hogy a felhasználó valamit begépel, vagy például azt, hogy egy periféria befejezte a kért műveletet és készen áll az eredmény vagy a kért adat átadására. Ez esetben például a néhány mikroszekundumos késleltetés/késlekedés nem jelent problémát, de a valós idejű (Real Time) és a folyamatirányítási (Process Control) rendszerek esetében az azonnali szuper gyors válaszadási idő már elsőrendű feladat.

[Real Time: Semmiképpen sem szabad azt gondolni, hogy itt valami olyan csodáról van szó, amely elvi és gyakorlati késedelem és időveszteség nélkül hajtja végre az elvárt műveleteket. Egy rendszert akkor tekinthetünk Real Time rendszernek, ha ELVÁRT sebességgel reagál. Például egy csiga Real Time irányításához vagy vezérléséhez a másodperc nagyságrendű beavatkozási illetve válaszidő is elegendő, mert közismert, hogy a legfürgébb csiga sem kapkodja el a dolgokat... A Real Time összegezve tehát azt jelenti, hogy a valós időben érkező igényeinkre valós időben kapunk választ, azaz egyszóval (még éppen) időben megérkezik a válasz illetve az eredmény ahhoz, hogy a szóban forgó folyamat zökkenőmentesen futhasson.]

A legjellemzőbb ilyen események csoportosítása:

- meghatározott külső műveletek befejezése, melyek bekövetkezésére számítani lehet, de ezek időpontja pontosan nem tervezhető (például egy periféria jelzi, hogy egy I/O művelet befejeződött)
- szándékos, azaz programvezérelt módon generált események (rendszerhívások)
- meghatározott programhibák (például 0-val való osztás)
- teljesen véletlenszerűen és váratlanul fellépő események (például hardverhiba vagy áramkimaradás)

A megszakítások két nagy csoportra bonthatók:

- Maszkolható megszakítás (Interrupt Request / IRQ [vagy IT])
Ezek végrehajtásának bekövetkezése az IRQ prioritási szintjétől függ.
- Nem maszkolható megszakítás (Non Maskable Interrupt / NMI)
Ezek jellemzően azonnal végrehajtandók (például órajel alapú megszakítás).

A folyamatok is két nagy csoportra bonthatók:

- Oszthatatlan műveleteket is tartalmazó
Ezt még az NMI sem szakíthatja meg, ha éppen oszthatatlan műveletnél tart!
- Oszthatatlan műveleteket nem tartalmazó
Tetszőlegesen megszakítható.

A végrehajtás alatt álló folyamatok közötti váltáshoz a váltandó folyamat környezetét (Context) el kell menteni, illetve az indítandó (vagy folytatandó) folyamat környezetét pedig be kell tölteni. Ez a tevékenység a környezetváltás. A környezetnek, mint információhalmaznak tartalmaznia kell a folyamat illetve a számítógép aktuális állapotát, beleértve mindazt az információt, ami a folyamat folytatásához, azaz újraindításához szükséges.

A folyamat és a számítógép aktuális állapotát egy összetett adatstruktúra a PCB (Process Control Block / Folyamatvezérlési Blokk) tartalmazza.

A PCB struktúrájának 3 fő része, és azok legfontosabb összetevői:

- A folyamatot azonosító adatok
 - Memória információ, mely tartalmazza a folyamat által használt memóriaterületek adatait, illetve a címtranszformációs táblázatot.
 - A folyamat azonosítója mellett a szülőjének és gyerekeinek azonosítói.
 - I/O státus információ, mely a folyamathoz rendelt, illetve lefoglalt I/O-eszközök listáját, és a megnyitott fájlok listáját tartalmazza.
- A CPU állapotának adatai
 - CPU-regiszterek, melyek futás közbeni processzor szintű adatokat tartalmaznak.
 - CPU-ütemezési információ, mely az adott folyamat ütemezésével, prioritásával, jogosultságaival kapcsolatos adatok összessége.
 - Elszámolási információ, mely a felhasznált CPU-idő és valóságos idő, idő-korlátok, egyéb számlálók, stb. adatainak összessége.
- A folyamat vezérlési adatai
 - A folyamat állapota. (A folyamatok bővített állapotátmeneti diagramja szerinti állapotok közül az aktuális állapot.)
 - Programszámláló, mely a folyamat soron következő végrehajtandó utasításának a címét tartalmazza.
 - Mutatók (Pointer), melyek a tárolásra használt adatszerkezet mutatói. Segítségükkel a tárolt adatok gyorsabban érhetők el.

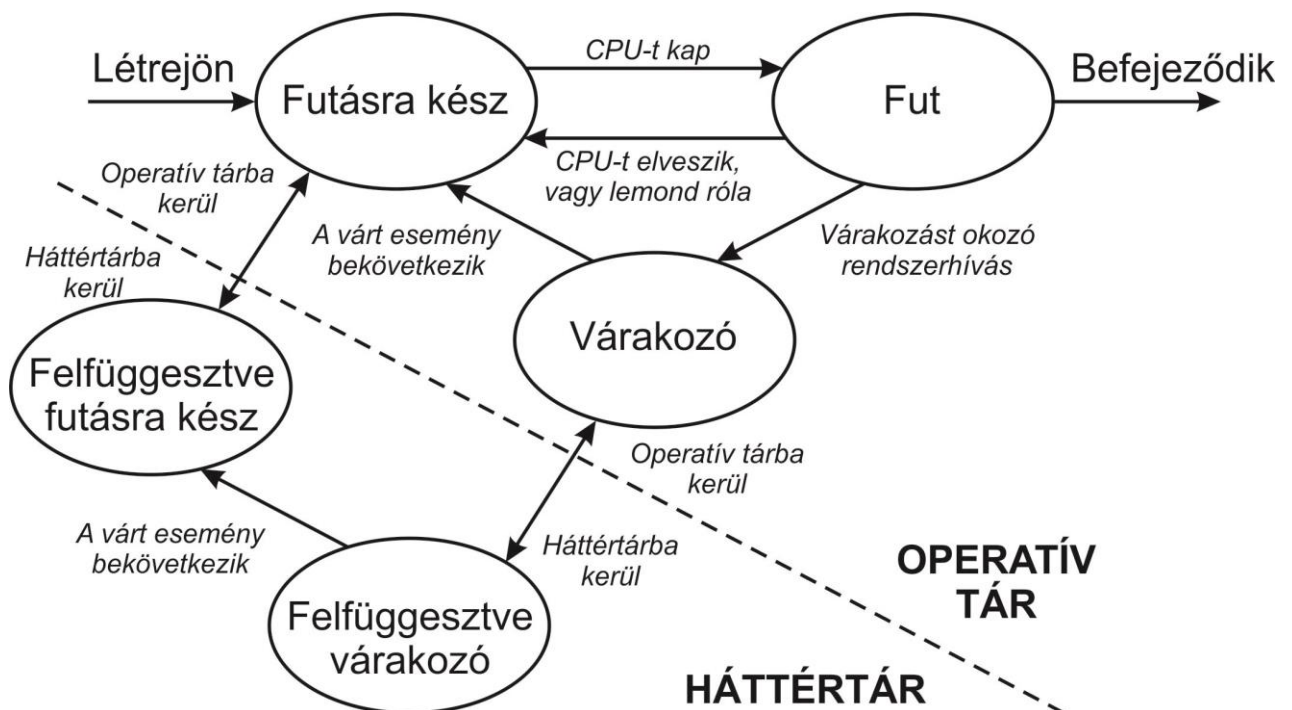
Az I/O műveletek paraméterei az (Input Output Control Block / be- és kimenetek vezérlő blokkja) IOCB-ben tárolódnak. Fontos része a rendszer struktúrájának, hogy az IOCB-k az I/O műveletet kezdeményező folyamat PCB-jére fűződnek fel. Ezzel azt is elértük, hogy az elindított I/O műveletek és az azokra várakozó folyamatok összekapcsolása is megoldottá vált. (lásd: A folyamatot azonosító adatok harmadik hivatkozása.)

2. Folyamatok modellezése

Egy folyamat vagy a CPU egy magját használja (CPU Burst / CPU löket), vagy I/O műveletet hajt végre (I/O Burst / I/O löket). Ezzel el lehetett érni, hogy a CPU és az I/O (azaz a perifériák) valóban párhuzamosan működjenek. Legelső lépésként minden folyamat CPU lökettel kell hogy induljon. Ezen löketek kiosztásáról az operációs rendszer gondoskodik, aminek természetesen a saját működéséhez is szüksége van a CPU-ra. Szintén az operációs rendszer feladata a periféria vezérlők felprogramozása. Érdekes és hasznos megoldás a DMA (Direct Memory Access), a közvetlen memória elérés, aminek segítségével az arra alkalmas és arra képes perifériák közvetlenül használhatják a memóriát. Nyilván a DMA működéshez is elengedhetetlen a CPU, de ebben a megoldásban csak addig van szükség a CPU-ra, amíg a perifériavezérlő felprogramozása tart.

Összefoglalva, tehát a rövid távú ütemező elkezd ütemezni a folyamatok végrehajtását a CPU-n, szépen ütemezetten (CPU löketenként). Ebbe a "levesbe köp bele" ciklikus rendszerességgel az operációs rendszer CPU igénye egy NMI segítségével, illetve szinte sztohasztikusan az I/O műveletek egy IRQ, és a nem várt műveletek egy IRQ vagy egy NMI segítségével.

3. A folyamatok bővített állapotátmeneti diagramja



Előadás_#03.

1. Ütemezés

Tekintsük át, hogy eddig minek a kapcsán merült fel ütemezés. Tulajdonképpen minden olyan lépés, ami állapot átmeneti változást eredményez, egyben jó eséllyel ütemezési feladatot is jelent. Minden esetben ütemezésre van szükség, ha egy (hardveres vagy szoftveres) erőforrásra több folyamat is benyújtotta igényét. Az ütemezés az ütemezendő események sorrendjének adott algoritmus szerinti meghatározását jelenti, vagyis azt, hogy melyik folyamat kapja meg az erőforrást. Az ütemezőik neve vagy közvetlenül az ellátott funkcióra, vagy az ütemezés gyakoriságára utal.

- Rövid távú ütemező (CPU ütemező)

A végrehajtás alatt álló folyamatok között képes váltani. Ütemezése minden CPU löket után lefut, tehát gyorsnak kell lennie, ezért a rövidtávú ütemező a kernel része, állandóan a memóriában tartózkodik. Egy processzor mag esetében nem kérdés, hogy mindent ezzel a rendelkezésre álló egy maggal kell megoldani, de több processzor illetve több processzormag esetén az egyik lehetséges megoldás, hogy minden mag a saját ütemezése miatt is felelős, illetve létezik olyan megoldás, amikor az egyik mag csak az össze mag ütemezését végzi, így a többi mag hatékonyabban tud a folyamatokkal foglalkozni.

Az ütemezési algoritmus lehet:

- Preemptív

Ebben az esetben a kiosztott futási jog elvehető az éppen futó folyamattól, aminek hatására a folyamat futásra kész állapotba kerül. Azért ebbe az állapotba, mert csak a CPU, mint erőforrás hiányzik neki. (A Windows NT alapú rendszerek preemptív ütemezést használnak, amit később részletesen tárgyalunk.)

- Nem preemptív

Ebben az esetben a kiosztott futási jog nem vehető el. Az éppen futó folyamat tehát vagy normál módon befejeződik, vagy önmaga mond le a futási jogáról. Ütemezés tehát futás közben csak akkor következik be, amikor a folyamat maga erre utasítást ad. (A UNIX például kernel módban nem preemptív, de user módban preemptív – ezt később részletesen tárgyaljuk.)

Az ütemezést kiváltó esemény lehet, amikor:

- a futó folyamat befejeződik,
- a futó folyamat várakozni kényszerül,
- a futó folyamat önként lemond a futási jogáról, vagy bármely okból az ütemező elveszi tőle a futási jogot,
- egy folyamat felébred és futásra készvé válik,
- illetve órajel alapú ütemezés esetén az időszelet letelte.

Homogén rendszerek esetén jellemző, hogy több CPU mag is ugyanazt a várakozási sort használja. Ebben az esetben az első szabaddá váló CPU mag kapja meg a soron következő folyamatot.

Heterogén rendszerek esetében (amikor több különböző architektúrájú és utasításkészletű CPU illetve mag együttműködéséről van szó) a folyamatok csak a nekik megfelelő utasításkészletű CPU-n tudnak futni.

Közös várakozási sorok esetében a következő megoldások alkalmazhatók:

- Szimmetrikus ütemezés
Minden CPU mag külön ütemezőt futtat. A közös várakozási sorhoz való hozzáférést, azaz annak megosztott használatát hibamentesen kell biztosítani, például kölcsönös kizárás segítségével.
- Aszimmetrikus ütemezés
Egyetlen kiválasztott CPU mag futtatja az ütemezőt, végzi a kiválasztást.

A CPU ütemezés algoritmusainak hatékonyságát több összehasonlító paraméterrel is megvizsgálhatjuk:

- CPU kihasználtság (CPU Utilization)
Azt mutatja meg, hogy a CPU az idő mekkora százalékában foglalkozik a folyamatok utasításainak végrehajtásával. A CPU-nak nyilván időt kell szakítania magára az ütemezésre, rendszeradminisztrációra, illetve az is előfordulhat, hogy nincs futásra kész folyamat, azaz a CPU tétlen (Idle).
- Átbocsájtó képesség (Throughput)
Azt mutatja meg, hogy adott időegység alatt az operációs rendszer mennyi programot (folyamat egységet) volt képes lefuttatni.
- Körülfordulási idő (Turnaround Time)
Az egy munkára fordított teljes időt mutatja meg, azaz, hogy a munka mennyi idő alatt fejeződött be. Ez tehát a futási idő és az összes egyéb (várakozással, adminisztrációval, stb.) eltelt idő összessége.
- Várakozási idő (Waiting Time)

Azt mutatja meg, hogy egy munka összesen mennyi időt töltött várakozással. Több összetevője van, mint a futás megkezdése előtti várakozás (lásd hosszú távú ütemező), a futásra kész állapotban, valamint a felfüggesztett állapotokban töltött idő.

- Válaszidő (Response Time)

Azt mutatja meg, hogy az operációs rendszer milyen gyorsan képes a felhasználói beavatkozásokra, igényekre reagálni.

- Közép távú ütemező

Amikor az operatív memória szűk keresztmetszetté válik, azaz gyakorlatilag megtelik, bizonyos folyamatok háttértárra mozgatásával szabad memóriára tehetünk szert. Ennek a memória felszabadításnak és visszarendezésnek az ütemezését végzi a középtávú ütemező.

- Hosszú távú ütemező

Az új folyamatok indításáról dönt, a folyamat indítást ütemezi. Követendő cél a CPU igényes és az I/O igényes folyamatok arányának megtartása. Relatíve ritkán fut, tehát nem kell szupergyorsnak lennie. Egy folyamat létrejötte után, megkapja az azonosítóját, valamint a legfontosabb erőforrását, a hozzá rendelt memóriaterületet. A folyamat a hosszú távú ütemező várakozási sorából akkor léphet át a rövid távú ütemező várakozási sorába, ha már minden szükséges perifériával rendelkezik, kivéve a CPU-t.

- Periféria ütemezők

Egy-egy periféria kiszolgálási sorrendjéről dönt, általában érkezési sorrend (FIFO pl. gépelés), vagy a prioritás (pl. diszk művelet) alapján. Egyes esetekben előfordulhat a fordított sorrendű (LIFO/FILO pl. LCFS / Last Come First Served szervezés esetén) kiszolgálás is.

2. Ütemezési algoritmusok

Az ütemezés többféle algoritmussal valósítható meg. Az egyes ütemezők számára más-más megoldás tekinthető optimálisnak. Az összes olyan követelménynek, amit egy "ideálisan működő" operációs rendszerrel szemben lehet támasztani, már csak a kérdésekben rejlő ellentmondások miatt sem lehet egyszerre megfelelni. A cél nem is ez, hanem a "megfelelő célra, megfelelő algoritmust" elv szem előtt tartás. A leggyakrabban megfogalmazott követelmények operációs rendszertől függetlenek, általánosságban vonatkoznak az ütemezésre, illetve annak közvetlen következményeire:

- Minden folyamat kapjon minél előbb CPU időt.
- A folyamatokat a prioritásuknak megfelelően legyenek kezelve, de részesítse előnyben a kihasználatlan erőforrásokat, illetve a fontos (olyan, amelynek a működésétől, más folyamatok függenek) erőforrásokat felhasználni kívánó folyamatokat.
- A folyamatok lehetőleg nem legyenek túl sokáig várakoztatva (éheztetés).
- Az operációs rendszer viselkedése kiszámítható legyen, a hatékonyság (a szempontjait lásd egy oldallal ezelőtt) időben és pénzben (energiaráfordítás) is kifejezhető legyen.
- Legyen maximális az átbocsájtó képessége. (Közelítse az elvi maximumot...)
- Növekvő terhelés hatására a teljesítőképesség fokozatosan csökkenjen, ne álljon elő egy túlterhelési pont esetén azonnali rendszerösszeomlás.

Az igények megismerése után nézzük, hogy milyen módon csoportosíthatjuk az ütemezési algoritmusokat:

- Egyszerű algoritmusok
 - Legrégebben várakozó (FCFS / First Come First Serve)
 Klasszikus, nem preemtív algoritmus. A futásra kész folyamatok egymás után kerülnek be a várakozási sorba, és érkezési sorrendben kerülnek kiválasztásra. (Olyan, mint mikor sorba állunk a mozijegyért...) Jellemzője a nagy várakozási idő, hiszen egy hosszan futó folyamat a mögött álló folyamatokat sokáig feltartja (konvoj hatás).
 - Körbeforgó (RR / Round-Robin)
 [RoundRobin algoritmus.xls]
 Klasszikus preemtív algoritmus, az időosztásos rendszerek így működnek. Minden folyamat egy időszletnyi futási időt kap. Az időszlet és a CPU löket hosszának viszonya sorsdöntő ebben az esetben. Amennyiben az éppen futó folyamat CPU lökete hosszabb mint az időszlet, akkor az időszlet letelte után a futási jog átszáll a következő folyamatra, a folyamatunk pedig futó állapotból visszakerül a várakozó sor végére. Amennyiben az éppen futó folyamat CPU lökete rövidebb mint az időszlet (ez jellemzően egy folyamat utolsó ütemezésekor fordul elő), akkor a CPU löket végén (új időszletet indítva) a sorban következő folyamatra száll át a futási jog.

Az időszel optimalis meghatározása nem egyszerű feladat. A túl hosszú időszel alkalmazása az FCFS működéséhez teszi az RR-t hasonlóná, a túl rövid időszel pedig a nagyszámú környezetváltás miatt lesz alacsony hatékonyságú. A tapasztalati állandó az, hogy a CPU löketek kb. 20%-a legyen csak hosszabb az időszelnél.

- Prioritásos algoritmusok

A megoldás alapját az képezi, hogy az összes futásra kész folyamat kap egy meghatározott tartományon belüli prioritási értéket. A CPU-t mindig a sorban állók közül a legmagasabb prioritással rendelkező folyamat kapja meg. A prioritási értékek meghatározása történhet az operációs rendszer által felállított saját rangsor szerint (belső kiosztás) illetve az érték érkezhethet az felhasználótól (külső kiosztás) is.

A prioritás első hallásra tökéletes megoldásnak tűnik, de jobban átgondolva felmerül egy megoldandó probléma. Amennyiben egy folyamat nagyon alacsony prioritással rendelkezik, elképzelhető, hogy véges időn belül nem jut CPU-hoz (kiéheztetés), hiszen valószínű, hogy mindig lesz nála magasabb prioritású folyamat. Azért hogy ez a forgatókönyv ne fordulhasson elő a statikus (azaz időben állandó) prioritások helyett a dinamikus prioritások használata látszik célszerűnek. A dinamizmus lényege az, hogy a régóta várakozó folyamatok prioritását az operációs rendszer ciklikusan folyamatosan megnöveli, azaz öregíti (Aging).

A prioritást az objektív fontosság mellett a CPU löketidő befolyásolja, amit viszont általában nem ismert, jellemzően becsléssel tudunk közelítő értékhez jutni. A becslésnél pontosabb információ lehet a folyamatot elindító felhasználó "önbevallása", illetve a folyamat esetleges korábbi viselkedéseiről készült feljegyzések.

Az alapvető prioritásos ütemezési algoritmusok:

- Legrövidebb löketidejű (SJF / Shortest Job First)

Az algoritmus nem preemptív, a futásra kész folyamatok várakozási sorából a legrövidebb löketidejűnek adja a CPU-t. Közel optimális az átlagos várakozási és a futási idő, és az FCFS esetében fellépő konvoj hatás nem alakul ki.

- Legrövidebb hátralévő idejű (SRTF / Shortest Remaining Time First)

Gyakorlatilag ez az algoritmus az SJF preemptív változata. Működési mechanizmusa az, hogy amennyiben a várakozó sorban következő futásra kész folyamat löketideje rövidebb, mint az éppen futó folyamat hátralévő ideje, akkor folyamatot cserél. Ez természetesen környezetváltással jár. Az ebből fakadó idővesztés ellenére is hatékony módszer.

- Legjobb válaszarány (HRR / Highest Response Ratio)

Ez az algoritmus az SJF olyan változata, ahol az öregítés segítségével elkerülhető a kiéheztetés, és az indítandó folyamat kiválasztásnál a löketidő mellett a várakozási időt is figyelembe van véve. A prioritást a következő képlettel számítja ki a rendszer:

$(\text{löketidő} + k * \text{várakozási idő}) / \text{löketidő},$

ahol a "k" egy tapasztalati konstans.

- Többszintű algoritmusok

A többszintű algoritmusok legfőbb jellemzője az, hogy a futásra kész folyamatok nem egy, hanem több önálló, egyedi prioritással rendelkező sorban várakoznak. Az egyes sorokon belül eltérő algoritmusok végzik a kiválasztást. Az ütemező egy kisebb prioritású sorból csak akkor választhat ki egy folyamatot, ha a magasabb prioritású sor(ok) már üresek.

- Statikus többszintű sorok (SMQ / Static Multilevel Queues)

Mivel statikus prioritásokkal dolgozik, nem mentes a kiéheztetéstől. Egy folyamat egyik várakozási sorból nem kerülhet át egy másik várakozási sorba. A várakozási sorok is saját állandó prioritással rendelkeznek.

A rendszer logikáját jól szemlélteti a következő példa:

1. rendszerfolyamatok (alapvetően fontos folyamatok)
2. interaktív folyamatok "a" (fontos a kellően gyors válaszidő)
3. interaktív folyamatok "b" (kevésbé kritikus válaszidő)
4. köteget feldolgozás (nem kritikus alkalmazások, fut ha van rá idő)
5. rendszerstatisztikák (nem fontos, hiszen a rendszer működésére nincs közvetlen hatással)

○ Visszacsatolt többszintű sorok (MFQ / Multilevel Feedback Queues)

A várakozási sorok az SMQ-hoz hasonlóan épülhetnek fel, de ez esetben a folyamatok képesek egyik várakozási sorból egy másik várakozási sorba átlépni. Ez a dinamizmus segít a kiéheztetés elkerülésében.

Kétféle irány létezik:

1. Egy folyamat alacsonyabb prioritású sorba kerül.

A folyamatok végrehajtása mindig a legmagasabb prioritású sorból indul. Az egyes sorokhoz tartozó időszeklet hossza viszont az alacsonyabb prioritású sorok esetében nagyobb. Amennyiben egy folyamat számára nem elegendő a rendelkezésre álló időszeklet, akkor átkerül egy alacsonyabb prioritású sorba. Fontos szempont továbbá, hogy az ütemezés a rövidebb CPU löketű folyamatokat előnyben részesíti.

2. Egy folyamat magasabb prioritású sorba kerül.

Az előző modell merev szabályait tovább finomítva időről-időre folyamatosan mérve a gyors(nak vélelmezett) folyamatokat a rendszer magasabb prioritású sorba helyezi át. Emellett a régóta várakozó folyamatok szintén egy magasabb prioritású sorba kerülnek át.

Előadás_#04.

1. Szálak (Thread)

A szálakról általános érvényű megfogalmazásokat – általában a multiprogramozott rendszerekre vonatkozóan – csak nagyon óvatosan lehet tenni, hiszen a szálkezelés tekintetében egy Windows NT alapú operációs rendszer és a UNIX abszolút másképp viselkedik, például az ütemezés szempontjából. Ugyanis a Windows NT alapú rendszerek szálütemezést használnak, míg a UNIX kizárólag folyamat ütemezést használ.

A folyamatok a végrehajtás során több (akár párhuzamosan is végrehajtható) szálra felosztva hajthatók végre. Fontos megjegyezni, hogy az erőforrásokat alapvetően a folyamatok kapják, de a folyamatoktól a szálak öröklik. Arról sem szabad megfeledkezni, hogy a folyamatok által használt erőforrások, mint olyan természetesen jelenthetnek hardveres és szoftveres erőforrásokat is.

A rendszer működését átfogó logika tehát az, hogy a felhasználó által futtatott program lesz maga a folyamat, és a folyamat miután megkapta az erőforrásokat, ha lehetősége van rá szálakra bomlik.

Például egy "IF" függvény egyes ágai mint szálak párhuzamosan feldolgozhatók, hiszen minden, a számításhoz szükséges adat közös. Abból a szempontból, hogy a folyamatoktól örökölt erőforrásokon a szálak osztoznak-e megkülönböztetünk "nehéz súlyú" folyamatokat illetve, "könnyű súlyú folyamatokat". A könnyű súlyú folyamatok szálai osztoznak az erőforrásokon. Tehát az előbb említett "IF" egy könnyű súlyú folyamat része volt, és ami képes volt szálakra bomlani. A nehéz súlyú folyamatok esetében nem beszélhetünk többes számban szálakról, csak egy végrehajtási szál van (ami így végeredményben maga a folyamat) azaz nincs ami osztozhatna az erőforrásokon.

A szálak végrehajtásának lényege tehát az, hogy a folyamat által meghatározott közös memóriát (memória címtartományt) használják, azaz a szálak váltásához nincs szükség környezetváltásra, jellemzően egy veremtár használata elegendő. Ebből az következik, hogy a szálak váltása sokkal gyorsabb, mint a folyamatok váltása. Ezért is mondhatjuk, hogy a szálak – megfelelő CPU esetében – párhuzamos végrehajtásúak.

Felsorolás szintjén hasonlítsuk össze, hogy milyen elemeket kell egy folyamatnak, és milyen elemeket kell egy szálnak tartalmaznia, nem elfelejtve azt, hogy a folyamat elemeihez a szálak hozzáférnek (hiszen öröklik azokat).

- Folyamatra jellemző elemek
 - Memória címtartomány
 - Globális változók
 - Nyitott fájlok
 - Gyermekefolyamatok (ha vannak)
 - Függőben lévő riasztások
 - Szignálok, jelzések és azok kezelői
- Szálra jellemző elemek
 - Utasításszámláló
 - Regiszterek
 - Verem
 - Szál állapotok
 - Adminisztratív információk

2. Folyamatok együttműködése

A multiprogramozott rendszerekben a folyamatok együttes lefutása, egymáshoz való viszonya, a köztük lévő kölcsönhatások, csatolások erőssége szerint a következő képpen alakulhat.

- Független folyamatok

Önmagukban szekvenciális folyamatokról van szó, melyek egymás működését nem ismerik, azt nem befolyásolják, működésük egymással nem mutat szinkronitást. Működésük tehát aszinkron, futásuk sebessége jellemzően eltér egymástól. Tulajdonképpen nincs is értelme másképp, csak önmagukban vizsgálni ezeket a folyamatokat.
- Csatolt folyamatok

Egymástól nem függetlenek, legalább egy erőforrást közösen használnak.

 - Versengő folyamatok

A versengő folyamatok sem tudnak egymásról egészen addig, amíg a versenyzés tárgyát képező erőforrás(ok) – jellemzősen periféria(k) – amelye(ke)n előbb-utóbb valahogy meg kell osztaniuk előtérbe nem kerülnek. Jellemzően user folyamatok esetében beszélhetünk versengésről.

- Együttműködő, kooperatív folyamatok

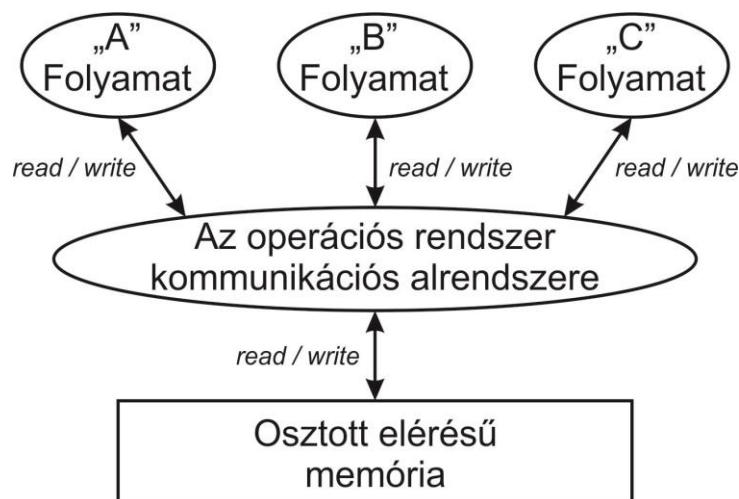
Ez esetben egy közös cél végrehajtása az ami összeköti a folyamatokat. Szükségszerű, hogy az együttműködő folyamatok képesek legyenek adatokat cserélni egymással. A kicserélt információ mennyisége az egy bit is lehet, de akár sok megabájt is elérhet. Jellemzően kernel folyamatok esetében beszélhetünk együttműködésről.

Az együttműködés tulajdonképpen az adatcsere és az erőforrások használatának összehangolását jelenti. Amennyiben ez az összehangolás időbeli, akkor szinkronizációról beszélünk. A folyamatok közötti adatcserét, információcserét röviden kommunikációnak hívjuk. (Tulajdonképpen az információ is adat.)

Ahhoz hogy két folyamat képes legyen együttműködni először is információt kell cserélniük a későbbi adatcsere időzítésével, módjával és tartalmával kapcsolatban.

Az információcsere a következő két módon történhet:

- Közös memórián keresztül



A közös memóriát (ha nem is mindig pont egyszerre) több párhuzamos futású folyamat is írja és olvassa. A működés alapja a következő kétféle közös memória modell valamelyike:

- RAM modell (Random Access Machine)

A közös memória rekeszekből áll, melyek egyenként címezhetőek. A címezés, az írás és az olvasás alapegysége tehát a memóriarekesz. Egy memória rekeszt egy időben csak egy folyamat használhat. Az írási és olvasási műveletek egy memóriablokkra, azaz rekeszre vonatkoznak. A párhuzamosság klasszikus DRAM memória modulok felhasználásával, ebben az esetben leginkább csak virtuálisan valósítható meg.

- PRAM modell (Pipelined Random Access Machine)

Ez a valódi multiprogramozott megoldás, hiszen ebben az esetben a közös memória ugyanúgy rekeszekből áll, de egy rekesz egy időben (valós időben) egyszerre több folyamat által is elérhető – akár írási akár olvasási célból. A PRAM algoritmus pusztán csak DRAM modulok felhasználásával nem oldható meg, a modell működéséhez SRAM-ok implementációja is szükséges a megfelelő működtető logikai egységgel és algoritmussal (pl. CRCW) együtt.

Egy memória rekesz szempontjából a párhuzamos működés három féle ütközést eredményezhet:

- Olvasás–Olvasás ütközés

Ez problémát nem okozhat, mindkét olvasás ugyanazt az eredményt kell hogy tartalmazza.

- Olvasás–Írás / Írás–Olvasás ütközés

A memória rekesz tartalma csak az új adattal íródhat felül. Az olvasás értéke pedig vagy a korábbi vagy az új tartalommal fog megegyezni.

- Írás–Írás ütközés

A memóriarekesz ez esetben biztosan elveszíti régi tartalmát, és új értéke pedig az egyik írási folyamat értéke lesz.

Látszik, hogy olvasások párhuzamos működése nem okoz gondot, de egy kis házárdjáték van a fenti írási mechanizmusok párhuzamos működésekor. Jellemző megoldás egyrészt az EREW PRAM (Exclusive Read Exclusive Write), ami tiltja a párhuzamos műveleteket, így visszavezet a RAM modellhez, másrészt a CREW PRAM (Concurrent Read Exclusive Write) megoldás, ami lehetővé teszi a párhuzamos olvasást, de tiltja a párhuzamos írást, valamint a CRCW PRAM (Concurrent Read Concurrent Write) megoldás, ami maga a klasszikus PRAM, azaz mindent enged párhuzamosan. CRCW PRAM esetében feltétlenül szükséges a szinkronizáció, vagy kölcsönös kizárás (ezekről a későbbiekben lesz szó). Az előbbi megoldások vegyes használata a jellemző, hiszen más és más szituációkban az egyik vagy a másik megoldás használata tűnik optimálisnak.

A Pipelined elnevezés pontosan arra utal, hogy fizikailag a memóriához egyszerre beérkező utasítások hatásukat tekintve egymás után érkező

utasításként értelmeződnek, egymást közvetlenül nem zavarják meg (nem interferálódnak), úgy viselkednek, mintha valamilyen sorba következtek volna egymás után. Az ilyen műveleteket atomi, azaz oszthatatlan műveletnek nevezzük.

- Üzenettovábbítással



Az adatcsere ez esetben nem egy közös memória rekeszen keresztül történik meg, hanem egy kommunikációs csatornán keresztül. A kommunikációs csatornát az operációs rendszer biztosítja. Ebben az esetben a két egymással kommunikálni szándékozó folyamatnak a megfelelő parancsok segítségével kell a csatornát elérnie.

- Send / Adat Küldés

A küldő folyamat ennek segítségével adja át az adatokat – a megcímezett folyamat azonosítóját, és magát az üzenetet, azaz az adattartalmat – a csatorna számára. A küldő folyamat adatai a küldő folyamat saját kizárólagos memória helyén tárolódnak. A küldött adatokat a csatorna átveszi, ha szükséges rövid ideig tárolja.

- Receive / Adat Fogadás

A címzett, azaz a fogadó folyamat ennek segítségével jelzi a csatorna részére, hogy készen áll az adatok fogadására. Amennyiben a csatorna tartalmaz a fogadó folyamat számára adatokat, akkor azokat a küldő folyamat azonosítójával együtt megkapja. A vevő folyamat adatai a vevő folyamat saját kizárólagos memória helyén tárolódnak.

Az adatforgalom ellenőrzésével, illetve a küldési és fogadási szándék jelzésével (például azzal, hogy a fogadó folyamat kér, vagy a csatorna kéretlenül is áttolja az adatot) nem most foglalkozunk.

3. Kommunikáció

A kommunikáció elengedhetetlen feltétele, hogy a két kommunikálni szándékozó folyamat "tudjon egymásról". Felmerülnek nyilvánvaló kérdések, melyeket meg kell tudni válaszolni, mielőtt bármilyen adatcserére is sor kerülhetne:

- Hogyan hivatkozzunk a kommunikációs partnerre (azaz a másik folyamatra), ismerjük-e az azonosítóját, vagy csak a funkcióját ismerjük?
- Közvetlenül fogjuk elérni a partnert, vagy valamilyen közbülső csatornán, postaládán keresztül, azaz ismerjük-e a kommunikáció adott partner szempontjából preferált vagy esetleg egyetlen lehetséges módját?
- Lehetséges-e egyszerre több partner számára elküldeni ugyanazt az üzenetet?
- Lehetséges-e valamilyen módon meggyőződni az üzenetküldés sikerességéről?
- Mi történik, ha hamarabb akarunk fogadni egy üzenetet, mint ahogyan az elküldésre (csatorna, postaláda) került?

A válaszok alapján a kommunikáció két nagy típusra választható szét:

- A partner címezése, megnevezése, elérése szerinti kommunikáció
- Az elvégzendő művelet hatása (szemantikája) szerinti kommunikáció

A partner címezése, megnevezése, elérése vonatkozásában a kommunikáció lehet:

- Közvetlen (direkt) kommunikáció

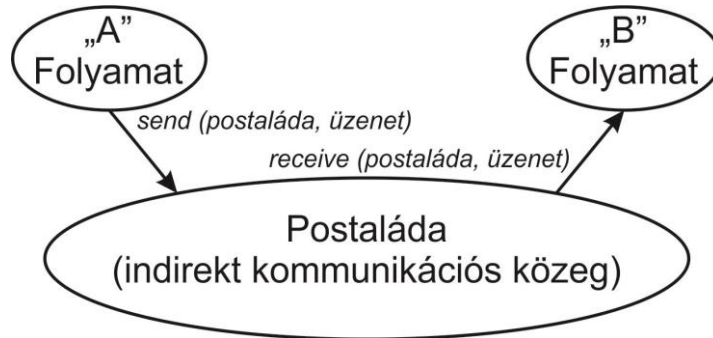


Nem véletlen, hogy itt az előző ábra ismétlődik, hiszen gyakorlatilag itt is ugyanarról van szó, csak a megközelítés más. A közvetlen kommunikáció két olyan folyamat között történhet, melyek kölcsönösen ismerik egymás azonosítóját. Azaz a küldő folyamat képes megnevezni a fogadó folyamatot, a fogadó folyamat pedig képes megnevezni a küldő folyamatot. A folyamatokra jellemző a szinkronizáció (amiről részletesen a későbbiekben lesz szó) ebben a kommunikációs formában.

A műveletek szintaktikája:

- SEND (a megcímezett folyamat azonosítója; üzenet)
- RECEIVE (a küldő folyamat azonosítója; üzenet)

- Közvetett (indirekt) kommunikáció



Ebben az esetben a kommunikálni szándékozó folyamatok, ugyan ismerik egymás folyamat azonosítóját, mégsem közvetlenül egymást nevezik, azaz címszik meg, hanem a közvetítő csatornát, vagy a postaládát. A postaláda jellemzően FIFO szervezésű, meghatározott számú (jellemzően két) folyamat használhatja, és elvileg sem végtelen kapacitású. Jellemző a több postaláda jelenléte a rendszerben. Nem szükséges a szinkronizáció ebben a kommunikációs formában.

A műveletek szintaktikája:

- SEND (a postaláda azonosítója; üzenet)
- RECEIVE (a postaláda azonosítója; üzenet)

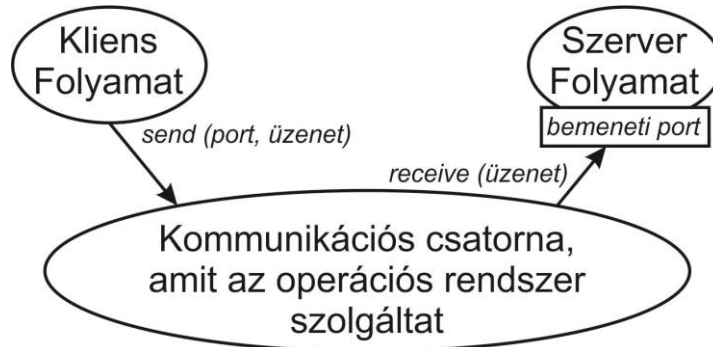
- Aszimmetrikus kommunikáció

Ebben az esetben a kommunikációban részt vevő folyamatok egyike (a küldő vagy a fogadó) kommunikációs portot használ. A port megnevezés általános megközelítésben, például TCP/IP hálózati kapcsolat esetében egy adott hoszt konkrét feladatot végrehajtó folyamatát vagy részfolyamatát jelenti. Ez esetben is a port egy feladat tipizációt, megjelölést jelent. Vételi port használata esetén a küldő oldal a vételi oldalon elvárt feladat típust (folyamat típust) jelöli csupán meg. Ebből az következik, hogy a küldő és a fogadó folyamatnak közvetlenül nem kell egymást ismerne.

Mit is jelent mindez a gyakorlatban? Ha egy folyamat például nyomtatni szeretne, és annak az információnak a birtokában van, hogy (ebben az esetben például) a 99-es porton nyomtatók várják a feladatokat, akkor elegendő megcímeznie a 99-es portot, és az első szabaddá váló nyomtató ki

fogja nyomtatni az anyagot. (Nyilván ez a példa akkor igaz, ha egy helységben több, egyforma nyomtató várakozik az anyagok kinyomtatására. Például ilyen eset lehet egy nagyvállalatnál a céges bérjegyzék nyomtatása.)

A kliens-szerver kiszolgálási modell:

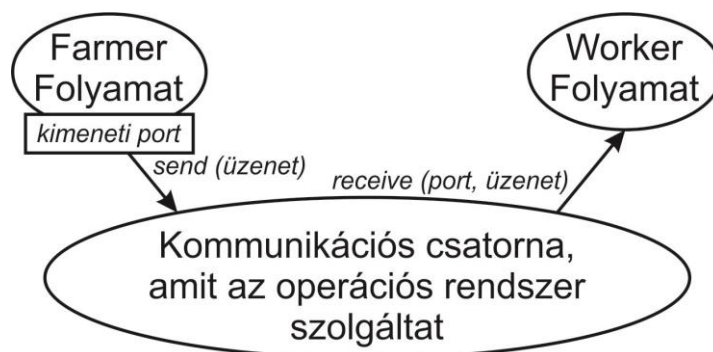


Vevő oldali port használat esetében a kliens-szerver kiszolgálási modell érvényesül, azaz egy szerver, egy kiszolgáló várja a sorban beérkező kéréseket.

A műveletek szintaktikája vételi port esetében:

- SEND (a vételi oldal portjának azonosítója; üzenet)
- RECEIVE (üzenet)

A farmer-worker kiszolgálási modell:



Küldő oldali port használat esetében a farmer-worker kiszolgálási modell érvényesül, azaz a munkákat sorban kiosztó farmer (szerencsésebb a menedzser szó használata) várja a feladatokra jelentkező munkásokat.

A műveletek szintaktikája küldő oldali port esetében:

- SEND (üzenet)
- RECEIVE (a küldő oldal portjának azonosítója; üzenet)

- Csoportkommunikáció

Jellemzően a küldő oldal ez esetben a vevő oldalon nem egy folyamatot céloz meg, hanem folyamatok csoportját. A kijelölt csoport minden tagja meg kell hogy kapja az üzenetet (multicasting). Létezik olyan eset is, amikor az összes folyamat címzetté válik, azaz az össze folyamat megkapja az üzenetet (broadcasting).

A csoportkommunikáció szintaktikája:

- SEND (csoport azonosító; üzenet)

Az elvégzendő művelet hatása (szemantikája) alapján a kommunikáció további tagolása már túlmutat az operációs rendszerek tantárgy tematikáján. Amit ezzel kapcsolatban meg kell említeni, az a műveletet végrehajtásának ellenőrzése, ami történhet állapotkódok cseréjével vagy hiba megszakításokkal és az azt követő hibakezelési rutin végrehajtásával. Hibajelzés történhet bizonyos időkorlátok lejárta, a kommunikáció megszakadása, illetve csak részbeni teljesülése esetén. Az időkorlátok használata alapvető igény, hiszen egyik folyamat sem várakozhat végtelen ideig. Az adatcsere nyugtázása abból szempontból is fontos, hogy a küldő folyamat az érintett memória rekesz tartalmát nyilván csak azután írhatja felül, ha már meggyőződött arról, hogy az ott tárolt és onnan továbbított adatokra a továbbiakban neki nincs szüksége, azaz azt a megcímezett folyamat sikeresen felhasználta.

A kommunikációs csatorna jellemezhető az átviteli közeg típusával, a kapcsolat lehetséges irányával (Simplex, Half Duplex, Full Duplex), illetve a csatornát használó folyamatok számával. Ezen jellemzően logikai paramétereken túl fizikai paraméterek is jellemzik a csatornát (amely így már túlmutat a felhasználó számítógépén). További fontos paraméter, hogy a csatorna milyen hosszú ideig és maximálisan mekkora méretű üzenetek tárolására alkalmas, más szóval mekkora a csatorna kapacitása, és mekkora a csatorna megbízhatósága. Ezen kívül szempont a csatorna késleltetésének minimális értéke is, ami azt mutatja meg, hogy optimális esetben mekkora késleltetést okoz egy elvileg azonnali (tehát tárolás nélkül továbbított) üzenet továbbítása.

Távoli kommunikáció esetén az időkorlátok használata elkerülhetetlen. Szintén szükség lehet az átvitel ellenőrzésére is. (Ez a témakör innentől már a Számítógép Hálózatok tantárggyal mutat több kapcsolatot...)

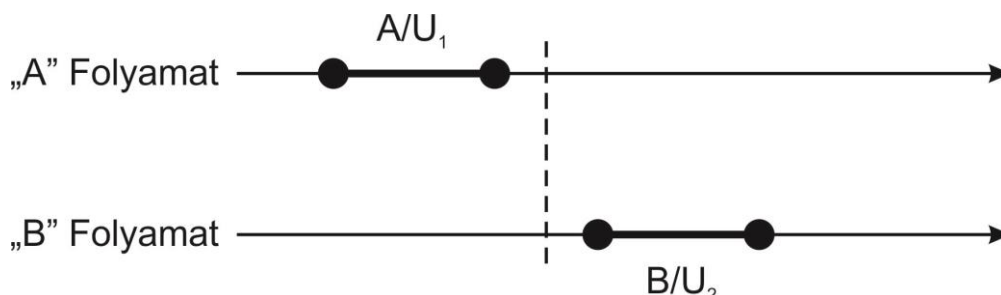
Előadás_#05.

1. Szinkronizáció

Azok a folyamatok, melyek egymástól nem függetlenek, azaz valamilyen függőség fennáll közöttük, ahhoz, hogy a legkevesebb késedelemmel legyenek képesek adatot cserélni egymással összehangoltan kell együttműködniük. A másik folyamathoz való alkalmazkodás kompromisszumokkal és korlátozásokkal jár, de több a járulékos előny ebből, mint a hátrány. A folyamatok műveleteinek összehangolását nevezzük tehát szinkronizációnak.

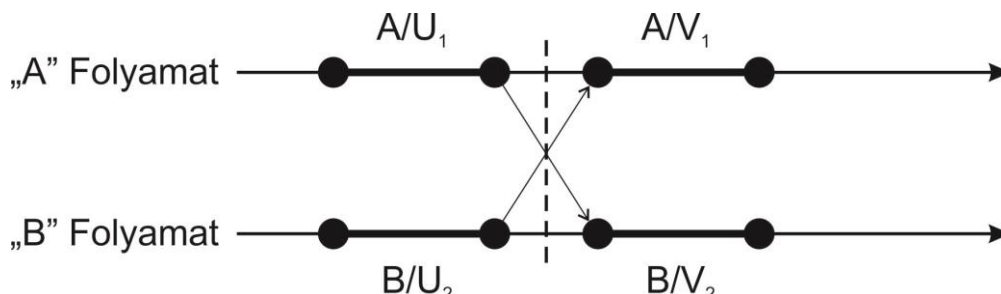
Az összehangolt folyamatok lefutása a következők szerint történhet:

- Előidejűség (Precedencia)



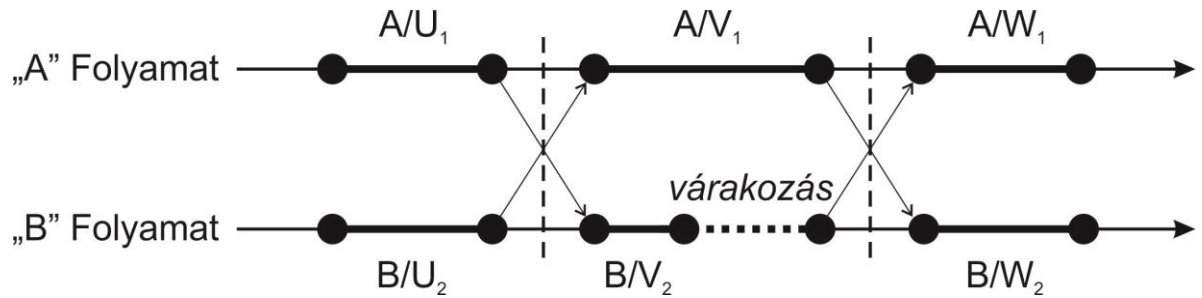
Az egyes folyamatok meghatározott műveletei között előírt végrehajtási sorrend van meghatározva. Egy azt jelenti, hogy ez egyik folyamat adott utasításának feltétlenül meg kell előznie a másik folyamat adott utasítását. Nem csak pusztán megelőzésről van szó, hanem arról is, hogy a második folyamat adott utasítása csak az első folyamat adott utasításának a befejezése után kezdődhet el, azaz semmiképpen nem lehet időbeli átfedés. Legjellemzőbb eset az, amikor a második folyamatnak szüksége van egy, az első folyamat által előállított adatra.

- Egyidejűség (Rendezvény)



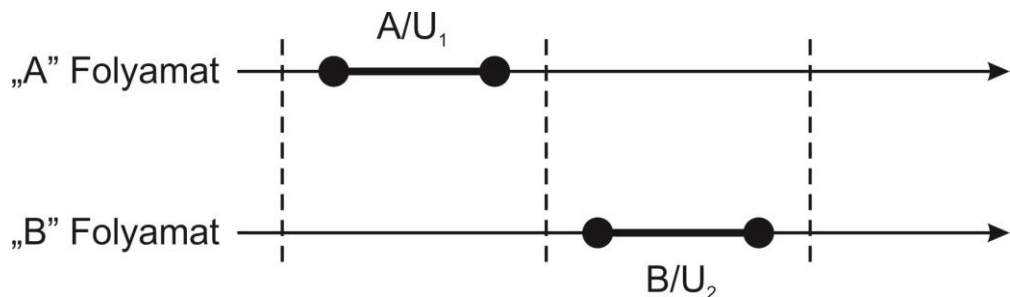
Az egyes folyamatok meghatározott műveletei között előírt végrehajtási mód lehet az, hogy a műveleteknek be kell várniuk egymást, és egyidejűleg kell végrehajtódniuk. Az egyidejű végrehajtás az után kezdődhet meg, hogy az azt megelőző folyamatok közül leghosszabb is befejeződött.

A randevú speciális esete a meghosszabbított randevú.



Ez gyakorlatilag randevúk egymás utáni sorozatát jelenti. Jellemző, hogy ugyan a folyamatok egyszerre indulnak, szükségképpen mégsem biztos, hogy egyszerre fejeződnek be. Az ez után következő randevúra csak a lassabban lefutó folyamat befejeződése után kerülhet sor.

- Kölcsönös kizárás (Mutual Exclusion)



Az egyes folyamatok meghatározott műveletei között előírt végrehajtási mód lehet az, hogy a műveletek semmiképpen se egyszerre hajtsódjanak végre. Sőt a műveletei még részlegesen sem fedhetik át egymást.

Könnyen belátható, hogy több együtt futó folyamat esetében léteznek olyan program szakaszok, melyek semmi esetre sem hajtódhatnak végre együtt. Legjellemzőbb eset egy közösen használt erőforráshoz való hozzáférés. A folyamatoknak azon utasítás sorozatait, melyek végrehajtása egyidejűleg nem megengedett, kritikus szakasznak nevezzük. Egy folyamat csak akkor léphet be a kritikus szakaszba, azaz csak akkor kezdheti meg a kritikus szakaszként kijelölt utasítássorozat végrehajtását, ha a kritikus szakaszban másik folyamat nem tartózkodik. Amennyiben egy másik folyamat foglalja a kritikus szakaszt, meg kell várni annak a (rész)folyamatnak a befejeződését, azaz a kritikus szakasz felszabadulását. Párhuzamosan futó folyamatok összetartozó kritikus szakaszai tehát kölcsönösen kizárják egymást.

A kölcsönös kizárás eredeti tipikus esete a nyomtatás egy megosztott nyomtatón, (fontos megjegyezni, hogy ez esetben a Windows előtti, azaz a nyomtatási sorkezelő nélküli, közvetlen elérésű sornyomtató vagy mátrixnyomtató szerepel a példában) hiszen nem lenne szerencsés, ha egy

lap szöveges tartalmának nyomtatása közben az egyik sor az egyik, a másik sor pedig egy másik folyamatból származna. Kölcsönös kizárással a második folyamat csak az első folyamat nyomtatásának befejezése után kezdhet maga is nyomtatni.

A kölcsönös kizárás ma is az egyik leggyakrabban használt szinkronizációs eszköz mind szoftveres, mind hardveres erőforrások közös használata esetén. Jelentősége azonban már nem a nyomtatás, hanem a fájl és port műveletek esetében van.

[term_fogy_modell.xlsx]

A kritikus szakasz gyakorlati megvalósításánál ügyelni kell arra, hogy a következő három feltételt teljesíteni tudjuk.

- A kölcsönös kizárás biztosítása
A kritikus szakaszban egyszerre csak egy folyamat tartózkodhat.
- A haladás biztosítása
Amennyiben a kritikus szakasz nem foglalt, és van legalább egy olyan folyamat, amely be szeretne lépni, akkor ezek közül egyet be kell engedni,
- A várakozási idő elviselhető szinten tartása
Törekedni kell az éheztetés elkerülésére.

A kritikus szakaszra való belépés engedélyezése történhet szoftveres vagy hardveres úton. A szoftveres út az egyszerűbb, de időigényesebb megoldás. Az operációs rendszernek ismernie kell a kritikus szakasz elejét (Entry) és végét (Exit). Az érintett folyamatoknak képesnek kell lenni egyrészt a belépési szándék jelezésére, másrészt a kritikus szakasz elhagyásának jelezésére. Feltétlenül szükség van legalább egy változóra, amely jelzi a kritikus szakasz effektív foglaltságát.

A hardveres megoldásnak a processzor utasításkészletébe épülve kell képesnek lennie a foglaltság ellenőrzésének az elvégzésére. Ehhez speciális, megszakíthatatlan, egyidejűleg több művelet végrehajtására is alkalmas megoldásokra van szükség.

- TestAndSet
A megnevezés azért van egybeírva, hogy ez is utaljon arra, hogy ez esetben egy lépésről van szó. Amennyiben egy érintett folyamat azt olvassa ki ebből a változóból, hogy szabadon beléphet, akkor belép és ezzel a TestAndSet logikai változó értékét szabadról foglaltárra állítja. A megoldás tehát összesen egy darab logikai változót használ, viszont igényli azt, hogy a PRAM modell mindenképpen az "Olvas-Ír" oszthatatlan műveleti sorrendet kövesse.

- Swap

Az elnevezés találó, hiszen ez esetben valóban egy cseréről van szó. Minden érintett folyamat rendelkezik egy kulcsnak nevezett logikai változóval. A kulcs 1-es értéke azt jelzi, hogy az adott folyamat be szándékozik lépni a kritikus szakaszba. Maga a kritikus szakasz szintén rendelkezik egy logikai változóval, aminek lakat az elnevezése. Amennyiben a kritikus szakaszban nem tartózkodik folyamat, a lakat értéke 0.

A kritikus szakaszba való belépéshez az összes érintett program meg kell, hogy vizsgálja a lakat változó értékét. Amennyiben egy folyamat a lakat értékét 0-nak találja, akkor a lakat értékét lecseréli (Swap) saját kulcs változójának értékére (ami 1 volt), és belép a kritikus szakaszba. A többi folyamat a lakat vizsgálata során már 1-es értékkel találkozik, azaz nem léphet be. A kritikus szakasz elhagyása után a folyamat visszacseréli a lakat és a kulcs változók értékét, ezzel lehetővé téve egy másik folyamat belépését a kritikus szakaszba.

Meg kell jegyezni, hogy a fent ismertetett két megoldás, a három teljesítendő feltétel közül csak az első kettőt tudja maradéktalanul megvalósítani. A harmadik feltétel (a várakozási idő elviselhető szinten tartása) csak lényegesen bonyolultabb eljárással valósítható meg.

- Szemafor

A szemafor algoritmust Edsger Wybe Dijkstra (1930-2002, holland matematikus-informatikus) dolgozta ki, 1968-ban. A megoldás működési mechanizmusa egy egyvágányos vasúti sínszakasz védelmének modelljét vette alapul. A szemafor a szinkronizációs problémák megoldásának univerzális eszköze, hiszen a kölcsönös kizárás megvalósításán túl az előidejűség és az egyidejűség esetében is használható.

A működés megvalósításához szükségünk van:

- egy nem negatív értékeket hordozó változóra, ennek neve "s" (létezik bináris szemafor is ez esetben az "s" értéke csak 0 vagy 1 lehet)
- három atomi, azaz nem megszakítható műveletre, melyek elegendőek a modell működéséhez:
 - "Init" [beállít] a szemafor inicializálása, alaphelyzetbe állítása, azaz "s"=1 (kölcsönös kizárás) vagy "s"=0 (előidejűség) illetve "s" 1-nél nagyobb értékre állítása, mely érték a kezelt erőforrások számát mutatja meg.

- "Wait(s)" [vár] Ez a művelet "s" értékét csökkent 1-el, amennyiben "s" értéke nagyobb mint 0. Nevével ellentétben a wait művelet nem minden esetben várakoztat, csak az "s"=0 esetben történik várakoztatás. (Eredeti elnevezése "P" a *proberen* holland szó után [kipróbálni])
- "Signal(s)" [jelez] Ez a művelet "s" értékét 1-el megnöveli. (Eredeti elnevezése "V" a *verhogen* holland szó után [megnövelni])

A működés mechanizmusa a következő.

Amennyiben "s" értéke 1, ez azt jelenti, hogy a modell a kölcsönös kizárást valósítja meg. A kritikus szakaszba belépni szándékozó folyamat egy "P" műveletet hajt végre, aminek hatására "s" értéke 0 lesz, és belép a kritikus szakaszba. A kritikus szakaszban végrehajtott programrész befejezése után egy "V" műveletet hajt végre a folyamat, azaz "s" értékét 1-el megnöveli, jelen esetben visszaállítja 1-re. A folyamat elhagyja a kritikus szakasz, lehetőséget teremtve másik folyamatok számára a belépésre.

Amennyiben "s" értéke 0, ez azt jelenti, hogy a kritikus szakasz használatban van. A kritikus szakasz felszabadulása után "s" értéke 1 lesz. Fontos megjegyezni, hogy nem mindegy, hogy az "s" értéke az inicializálást követően lesz 1 vagy egy "V" művelet után.

Amennyiben "s" értéke 2 (vagy ennél is több), akkor több folyamat egymás mellett képes belépni, mert az "s" értéke ez esetben azt mutatja meg, hogy a szemaforhoz tartozó erőforrásból mennyi áll rendelkezésre. Belépéskor a folyamatok sorban egymás után csökkentik (P), kilépéskor pedig növelik (V) "s" értékét.

[Szemafor.xlsx]

Működés közben sok folyamat teszteli az "s" értékét, ami legtöbbször felesleges aktív processzor használatot jelent. Lassú folyamatokat nem célszerű futásra kész állapotban tartani, hiszen a processzor, mint erőforrás ezen folyamatoknak még hiányzik, de számos más erőforrást viszont feleslegesen tartanak foglalásban. Célszerűbb a lassú folyamatok esetében altatni, azaz passzív várakozást alkalmazni, hogy elkerüljük az erőforrások pazarlását.

2. A "Monitor" típusú programszerkezet

A szemaforral ellentétben a monitor egy magasabb szintű, objektum elvű, objektum orientált művelet. Ezt a programszerkezetet C.A.R Hoare angol programfejlesztő dolgozta ki 1974-ben. A monitor objektum egy több szál által használt eljárás nem párhuzamos végrehajtását teszi lehetővé, azaz nem folyamat szintű művelet. Ötvözi az objektum orientált programozást a szinkronizációs metódusokkal.

A monitor objektum a következőkből áll:

- osztott adat
- ezeket az adatokat feldolgozó eljárások
- monitort inicializáló metódusok

A megvalósításhoz ez esetben is három dolog kell, egy feltételváltozó és két művelet (Wait és Signal), de a kritikus adatokhoz csak szinkronizált műveleteken keresztül lehet hozzáférni.

Minden eljárás halmazt egy monitor kontrolál. A többszálás alkalmazás futásakor, a monitor egyetlen szálnak engedélyezi egy adott időpontban az eljárás végrehajtását. Ha egy szál éppen egy monitor által kontrolált eljárást akar futtatni akkor az lefoglalja a monitort. Abban az esetben ha a monitor már foglalt, akkor a belépni szándékozó szál várakozik amíg a monitort lefoglaló szál befejezi az adott eljárás végrehajtását és felszabadítja a monitort. Előnye, hogy a közösen használandó erőforrások és az erőforrásokon végrehajtható műveletek egyetlen szintaktikus egységben helyezkednek el.

A monitor és a szemafor összehasonlítása:

- A monitor magasabb szintű eszköz. Magas szintű, objektumorientált nyelvek esetében (pl. Java) roppant egyszerűen használható.
- A szemaforok segítségével rugalmasabban kezelhetők a kritikus szakaszok, hiszen nem kell beágyazott módon, előre elkészíteni az összes kritikus szakaszt.
- A szemaforokkal több kritikus erőforrás használatát könnyebb általában összekombinálni egy kritikus szakaszon belül.

Előadás_#06.

1. Holtpont, Éheztetés

A holtpont részletes tárgyalása előtt nagyon fontos leszögezni a következőt:

- Az éheztetés folyamat szintű probléma.
Egy folyamat akár a végtelenségig is várhat erőforrásra (pl. öregítés nélkül).
- A holtpont operációs rendszer szintű probléma.
Akár az egész rendszer megállhat, lefagyhat, (de ez nem minden esetben következik be, maradhatnak futó folyamatok) legrosszabb esetben azonban semmi sem tud futni. Amennyiben az összes erőforrás holtpontba kerül, a rendszer működése biztosan leáll.

A hosszú távú ütemező indítja sorba a folyamatokat, melyek ezután az indulásukhoz szükséges erőforrásokra igényt jelentenek be. Azokat a folyamatokat, melyeknek csak a CPU – mint erőforrás hiányzik, az összes induláskor ismert erőforrás biztosítása után a rövid távú ütemező várakozó sorba helyezi, azaz ezek a folyamatok futásra kész állapotba kerülnek. Amint a folyamat megkapja a CPU-t is a folyamat futása elkezdődik. Erőforrásigény azonban futás közben is keletkezhet. Például ha egy folyamat sikeres befejezését egy adott dallam lejátszásával is kívánjuk jelezni, nyilván a hang megszólalásáért felelős erőforrást (hardveresen a hangkártyát, szoftveresen a hangkártya meghajtó programjait – a hangkeverés lehetőségétől most ebben a példában tekintsünk el) nem célszerű már a futás elején lefoglalni, hiszen a folyamatunk futása közben más folyamatok hasonló célból, különböző hangjelzésekkel tudathatják lefutásukat.

A preemtivitás, mint tulajdonság az erőforrások szempontjából is az egyik csoportosítási szempont.

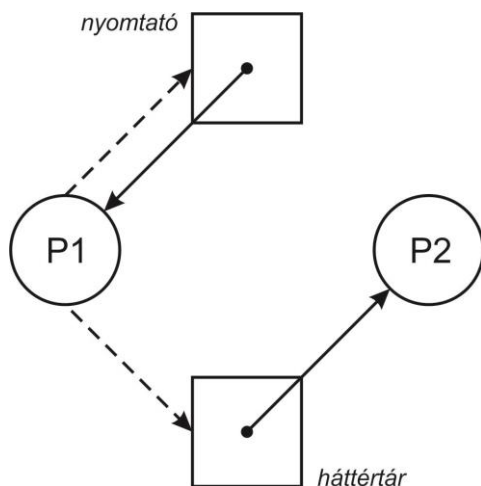
- Megszakítható (Preemptív) erőforrás
pl. processzoridő, memória
- Nem megszakítható (nem preemptív) erőforrás
pl. állományok, nyomtató, rajzgép, szalagos egység.

A nem megszakítható hozzáférési mód olyankor szükséges (esetleg elengedhetetlen), amikor egy folyamat egy erőforráson olyan műveletet végez, amelyet nem szeretne, hogy egy másik folyamat megzavarjon. Normális esetben egy erőforrás csak akkor szabadul fel, ha a folyamat önként lemond róla, vagy a folyamat futása befejeződik.

A valóságban az erőforrások osztályokba sorolhatók, melyek az adott osztályon belül egyenértékűek, csereszabatosak. Az erőforrás osztályok, azaz egy másfajta csoportosítás:

- az egypéldányos osztályban csak egy erőforrás példány van (pl. nyomtató, plotter)
- a többpéldányos osztályban olyan erőforrás van, amit egyszerre több folyamat is használhat (pl. memória)

Probléma alapvetően a kizárólagosan használható, nem megszakítható erőforrásokkal van, vagyis a probléma (azaz az előre nem kalkulálható várakozás) akkor keletkezik, amikor a „P1” folyamatnak – már futása közben – szüksége van egy másik folyamatra a „P2” által használt, kizárólagosan használható erőforrásra. Amennyiben „P1” prioritása kellően magas, magasabb, mint „P2” prioritása, akkor a kérdés az, hogy a „P2” folyamat állapottere menthető, vagy nem menthető. Ha nem menthető, és a folyamatot kilőjük (Kill) – hogy hozzáférjünk az általa lefoglalt kizárólagosan használható erőforráshoz – akkor az biztos, hogy adatvesztéssel jár, a „P2” adatai, eredményei elvesznek, így azokat a későbbiekben nyilván pótolni kell, amennyiben az egyáltalán lehetséges. Ha menthető, akkor a középtávú ütemező a „P2” folyamatot kitüntetett várakozó állapotba teszi, és „P1” végeztével „P2” folytatja futását.



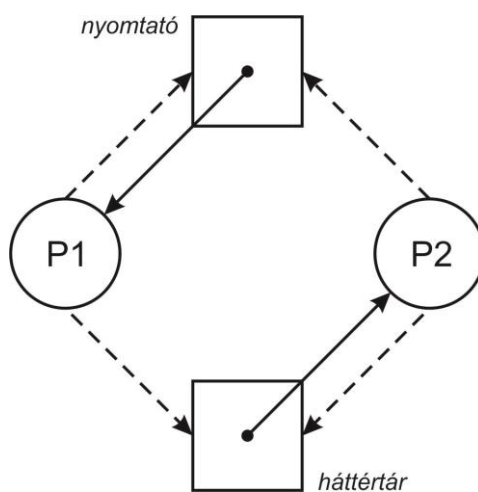
A szaggatott vonalú nyíl, ami egy folyamattól indul és egy erőforrás határvonalára mutat, az úgynevezett "kérési él", azaz az erőforrás foglaltságának szándékát jelzi. A folyamatos vonalú nyíl, ami egy erőforrás belsejéből indul és egy folyamat határvonalára mutat, az úgynevezett "foglalási él", azaz az erőforrás adott folyamat általi foglaltságát mutatja.

A fenti eset holtponthoz nem vezet, de előre nem kalkulálható várakozást igen, sőt esetleg adatvesztést is (lásd „P2” menthetősége vagy újratekintése).

[Szerver gépek esetében gyakorlatilag minden HW erőforrás többpéldányos és jellemzően cache-elhető is.]

Amennyiben a létező véges számú, jellemzően nem megszakítható erőforrás birtoklásáért egyszerre több folyamat is versenyben van, elő tud állni egy olyan állapot, amelyben lesz olyan „Px” folyamat, melynek az egyik általa kért erőforrás nem biztosítható, mivel azt már egy másik „Py” folyamat használja. Ez esetben a „Px” folyamat várakozni kényszerül. Szerencsés esetben relatíve rövid várakozás után hozzá tud férni az áhított erőforráshoz, amint az felszabadul. Szerencsétlen esetben megtörténhet az, hogy „Py” is várakozik, hiszen az általa óhajtott erőforrást egy harmadik folyamat, a „Pz” birtokolja, aki pedig a „Px” egy másik erőforrására vár – sőt akár sokkal hosszabb hasonló láncolat is kialakulhat.

A legszerencsétlenebb eset a körkörös várakozás, ami a holtpont egyik oka. A körkörös várakozás kialakulásához akár két folyamat is elegendő lehet.



A fenti esetben mindkét folyamat nyomtatni szeretne, amihez az adatokat a háttértárról olvasnák be. Jelen esetben a folyamatok az erőforrásokat nem azonos sorrendben igényelték. A „P1” folyamat először foglалást jelentett be a nyomtatóra, amit meg is kapott. A „P2” folyamat először foglалást jelentett be a háttértárra, amit meg is kapott. Ezután a „P1” hiába jelent be foglалást a háttértárra, azt már birtokolja a „P2”, és hasonló az eset a „P2” szempontjából is, hiszen a „P2” hiába jelent be foglалást a nyomtatóra, azt már birtokolja a „P1”. Azonos prioritás esetén nincs remény a továbblépésre.

Elvi szintű, azaz elvileg (főleg a ma jellemző memória konfiguráció mellett csak nehezen) elképzelhető az a példa is, amikor a holtpont azért alakul ki, mert egy folyamatnak valamilyen driver (azaz meghajtó program) használata szükséges egy adott erőforrás eléréséhez, de nincs a folyamathoz rendelt memóriában elegendő hely a meghajtó program betöltéséhez és végrehajtásához.

Az erőforrások használatba vételének és használatának lépései:

1. Igénylés.

Amennyiben az igény nem teljesül, a folyamat várakozni kényszerül.

2. Az erőforrás kizárólagos használatának megkezdése.

3. Az erőforrás felszabadítása a folyamat végeztével.

Ahhoz hogy a holtponthoz ki tudjon alakulni négy szükséges feltételnek kell egyszerre teljesülni:

1. Kölcsönös kizárás

Ezen feltétel szerint egy erőforrást egy időpillanatban csak egy folyamat használhat. Amennyiben egy folyamat igényt tart erre az erőforrásra, várakozni kényszerül.

2. Foglaltság és várakozás

Ez a feltétel azt jelenti, hogy legalább egy olyan folyamatnak kell léteznie, amely már rendelkezik egy erőforrás foglaltsággal, de további olyan erőforrásokra vár, melyeket más folyamatok foglaltságba tartanak, használnak.

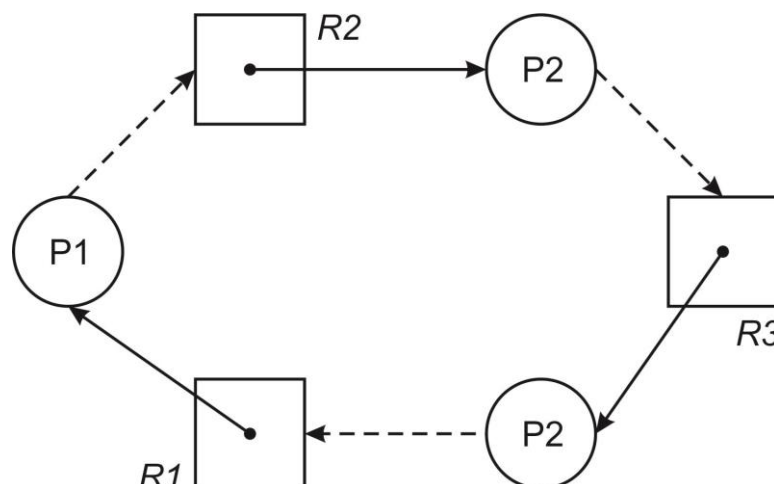
3. Kiszorításmentesség

Ez a feltétel azt jelenti, hogy a lefoglalt erőforrást az azt foglaltó folyamattól másik folyamat elvenni nem tudja, azaz az erőforrás nem preemptív, az erőforrás csak a folyamat befejezése után szabadul fel.

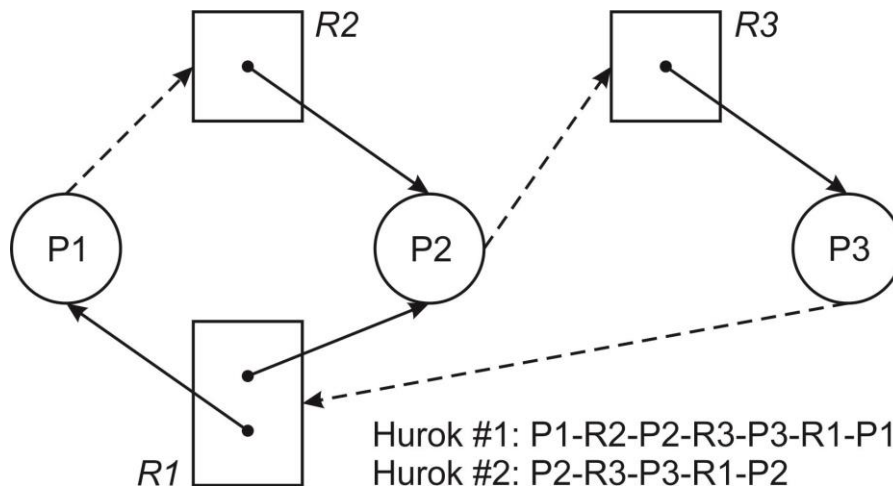
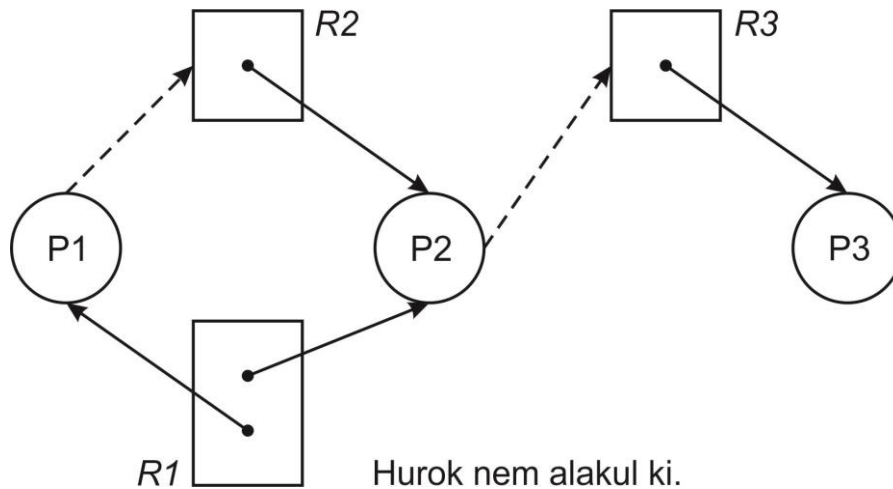
4. Körkörös várakozás

Ez a feltétel szerepelt a 2. oldalon („Px” „Py” „Pz” folyamatokkal modellezve).

Az erőforrás gráfok használatával tudjuk legkönnyebben átlátni a helyzetet. Általában az erőforrás-használati gráfban a hurok szükséges feltétele a holtponthoz. Amennyiben valamennyi erőforrás osztály csak egyetlen erőforrást tartalmaz, a hurok elégséges feltétel is egyben.



Azok az erőforrás osztályok, melyeknek az erőforrásaihoz egy időben több folyamat is csatlakozhat, jelentősen csökkentik a hurok kialakulásának valószínűségét. Amennyiben az erőforrás használati gráf nem tartalmaz hurkot, akkor biztosan nincs holtponthelyzetben.



A bármely okból kialakult holtpont feloldásához, megszüntetéséhez mindenképpen arra van szükség, hogy a holtpontot okozó erőforrás foglaltsága valahogy megszűnjön.

Az adott probléma (holtpont elkerülése) megoldásakor tekintettel kell arra lenni, hogy a probléma milyen áron (mennyi közvetlen adatvesztéssel, adott folyamat vagy folyamatok teljes újraindításával) oldható meg. A holtpont jellemzően nem számolható fel veszteség nélkül.

A holtpontot egyszerűbb megfelelő adminisztrációval megelőzni, mint a már kialakult holtpontot kezelni, megszüntetni. Nézzük meg, hogy a fent ismertetett négy szükséges feltétel közül melyik az, amelyik kiküszöbölhető:

1. Kölcsönös kizárás

Nem megosztható erőforrások esetében nem küszöbölhető ki.

Megosztható erőforrások viszont nem is követelik meg a kölcsönös kizárást.

2. Foglалás és várakozás

Ez kizárható, hiszen azt jeleni, hogy egy folyamat nem kérhet új erőforrást, amíg lefoglalva tart egy másikat. A megvalósításnak két módja van:

- Futás elején lefoglalja az összes erőforrást, és csak akkor futhat, ha valamennyi erőforrás a rendelkezésére is áll.
- Az erőforrás-foglalás előtt a foglalt erőforrásokat felszabadítja, vagyis a folyamat csak akkor igényelhet egy erőforrást, ha már más erőforrást nem használ.

A módszer biztos, de nem hatékony, hátránya, hogy rossz kihasználtságot eredményez, és nagy a kiéheztetés esélye is.

3. Kiszorításmentesség

A nem preemtív erőforrás közvetlenül nem vehető el a folyamatból, de ha a folyamat további erőforrásra vár, azaz várakozik, futni képtelen, akkor célszerű az egész folyamatot megállítani, aminek hatására maga lemond az összes birtokolt erőforrásáról. A folyamat csak akkor lesz újraindítva, ha minden erőforrás a rendelkezésére fog állni.

A módszer hátránya, hogy fennáll annak a kockázata, hogy a folyamatot folytatni nem lehet, csak teljesen újraindítani, mivel az addigi eredményei, részeredményei elveszhetnek. A kiéheztetés ez esetben is elő fordulhat.

4. Körkörös várakozás

Ez a feltétel is kizárható, amennyiben az erőforrásokat hierarchikusan megszámozzuk, és az erőforrások csak ebben a sorrendben foglalhatók le. Ezzel elérhetjük azt, hogy egy folyamat csak akkor kaphat meg egy adott sorszámú erőforrást, ha az adott erőforrásnál magasabb sorszámú erőforrást nem használ.

A módszer hátránya, hogy a hierarchikus sorszám nem képes az igények változási dinamikáját követni, mivel statikus megoldás.

Mindezek ismeretében, akkor hogyan is kerülhető el a holtpont kialakulás, hatékony módon, miközben a holtpont kialakulásának feltételei továbbra is adottak. A megoldást az erőforrások kiosztásának módja képes biztosítani. A módszer akkor használható, ha:

- Biztos, hogy valamennyi olyan folyamat, amelynek erőforrás igényei maximálisan ki vannak elégítve, véges idő alatt lefut, s ezután felszabadulnak az általa lefoglalt erőforrások.
- Valamennyi folyamatról előre tudható, hogy milyen erőforrás típusból maximálisan mennyit igényel egyidejűleg lefoglalva.
- Az erőforrás igényeket csak akkor elégítjük ki, ha rendszer "biztonságos állapotban marad", azaz található olyan folyamat végrehajtási sorrend, melyben az egymást követő folyamatok maximális erőforrásigénye kielégíthető.

A biztonságos állapot tehát biztosan holtpont mentes. A nem biztonságos állapot még nem jelenti a holtpont automatikus kialakulását, de le kell szögezni, hogy a holtpont csak a nem biztonságos állapotban belül képes kialakulni.

A legkézenfekvőbb megoldás az úgynevezett bankár algoritmus.

2. Bankár Algoritmus

A Bankár algoritmust Edsger Wybe Dijkstra (1930-2002, holland matematikus-informatikus) dolgozta ki [az ő nevéhez fűződik a Szemafor, valamint a strukturált programozás fogalma is].

A legfontosabb alapelv az, hogy a Bank sohasem kölcsönzi ki egyszerre az összes pénzét, pláne nem egyetlen ügyfélnek, hanem ügyfelenkénti limitált hitelkereteket használ.

Az operációs rendszer csak azokat a folyamatokat engedi (egyszerre) futni, melyek erőforrásigénye (egyszerre) kielégíthető. A gyakorlatban sajnos a folyamatok nem mindig képesek pontosan előre jelezni az összes HW és SW erőforrás igényüket.

[Bankár algoritmus_v2.xls]

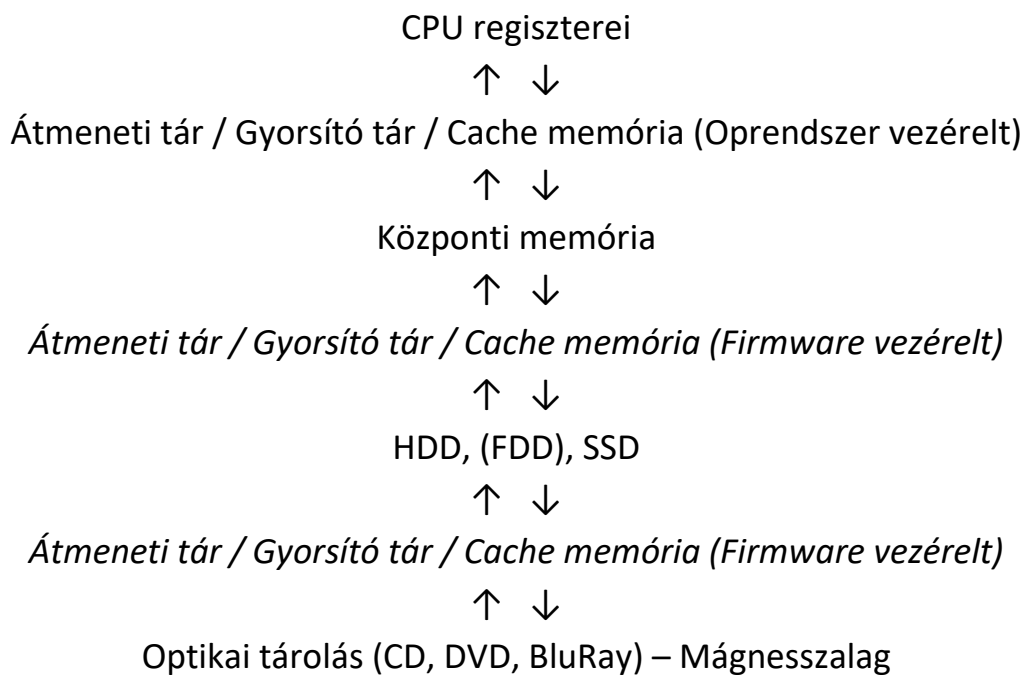
Az elvárható gondosság, mint fogalom, azt vélelmezi, hogy egy jól felépített rendszer megelőzési és elkerülési eljárásokat alkalmaz. Amennyiben ezek az eljárások hiányoznak, akkor csak eseti manuális módszereket lehet bevetni.

Előadás_#07.

1. Tárkezelés

A tárolóeszközök több szempont szerint is csoportosíthatóak (sebesség, kapacitás, mozgó alkatrészek aránya, stb.), mégis a legfontosabb csoportosítási szempont az, hogy az adott tároló a számítógép kikapcsolása után is megőrzi-e tartalmát, vagy a számítógép kikapcsolásával elfelejti azt. Így megkülönböztetünk „nem felejtő”, azaz permanens (Persistent, Non-Volatile), illetve „felejtő”, azaz nem permanens (Non-Persistent, Volatile) tárolókat.

A tárolóeszközök hierarchiája a tárolókapacitás illetve a sebesség viszonylatában alkot egységes rendszert. A két előbb említett paraméter jellemzően fordított arányosság szerint viszonyul egymáshoz. Az egységnyi adat tárolásának költsége is egyre csökken a tárterület méretének növekedésével. A következő felsorolás az első sorában a leggyorsabb, de legkisebb kapacitású tárolók, míg az utolsó sorában a leglassabb elérésű, de legnagyobb kapacitású tárolók családjai találhatók.



Más megközelítésben: A központi tár után következnek a belső permanens tárolók, a külső permanens tárolók, illetve a biztonsági másolatok különböző megvalósításai. Léteznek ezen kívül nem „oprendszer közeli” tárolási megoldások: hordozható eszközök (Pen Drive, mobil HDD), internet, felhő alapú tárolás. (Bár pont a Windows Vista és az azt követő rendszerek képesek bizonyos típusú pendrive-okat rendszergyorsításra használni, de ez inkább lakossági (azaz kommersz) megoldás, mint vállalati (azaz professzionális), tekintettel a megoldás hibátűrésére.

Az egyes szintek között (az adott szintek kiszolgálásához megfelelő) gyorsító tárákat, cache memóriákat is találhatunk. [A „cache” szó angol/francia eredetű, jelentése rejtkehely. A hangzás összezseng a „cash” – készpénz szóval, támogatva azt a (jelen esetben téves) logikát, hogy a készpénz gyorsan rendelkezésre áll...]

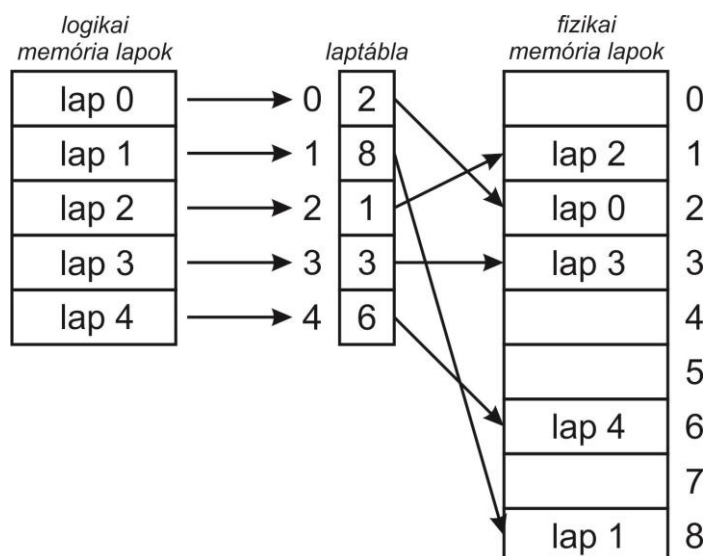
Az operációs rendszer, illetve az egyes intelligens eszközök firmware-e ezen átmeneti tárák segítségével olyan módon képes az adatátvitelt gyorsítani, hogy megkísérli egy megfelelő algoritmus segítségével azokat az adatokat előre áttölteni a gyorsító tárba, melyekre nagy valószínűséggel a közeljövőben szükség lesz. Egy adott címen elhelyezkedő adat esetében például az adott cím környezetében található adatok is betöltésre kerülnek (hátha jók lesznek valamire...). A gyorsítás tehát azon alapul, hogy a gyorsító tár gyorsabb, mint az egyik irányból hozzá kapcsolódó, gyorsítandó tár. Így ha a gyorsítandó (rész)tartalom korábban már bekerült a gyorsító tárba (valamely cache vezérlő algoritmus szerint), az ilyen adatokat már nem a lassú működésű területről, hanem a gyors cache tárolóból lehet előhívni.

Például egy CPU esetében a keresés a leggyorsabb helyen, azaz az L1 cache-ben indul, az L2-ben majd az L3-ban folytatódik. Legrosszabb esetben az adat a RAM-ban lesz megtalálható, illetve legeslegrosszabb esetben háttértárból kell előhívni.

Az operációs rendszer feladata a hatékony memória gazdálkodás, az operatív memória hatékony kezelése illetve kiosztása a folyamatok részére. A multiprogramozott rendszerek esetében, ahol egyszerre több program is a memóriában tartózkodik a hatékony memóriakezelés kiemelt fontosságú feladat.

A memória kezelés legfontosabb elvei:

- A programok fizikai tárigényének csökkentése
 - A fizikai és a logikai címtartományok összerendelése.



- A programok bizonyos részei, eljárásai csak akkor töltődnek be a memóriába, amikor azokra szükség van. (pl. hibakezelő rutin)
- A több program által használt eljárások elég, ha csak egy példányban, egy fizikai címtartományban tárolódnak. Így a programok különböző logikai címtartományai ugyanarra a fizikai címtartományra hivatkoznak. Ezen elv gyakorlati megoldásai például az Overlay (Egymást átlapoló programrészek) és a DLL (Dynamically Linked Library / Dinamikusan kapcsolódó [kölcson]könyvtár).

Az overlay technika biztosítja az olyan programok használhatóságát is, melyek egyszerre nem férnének el a számítógép belső memóriájába. Az overlay átlapolást jelent. A gyakorlatban az overlay területre a programnak mindig csak az éppen szükséges (futás alatt álló) része töltődik be, felülírva az esetleges előzőleg bent lévő adatokat.

A DLL technika megvalósulásának alapját a *.dll rendszerfájlok képezik, melyek dinamikusan, azaz csak akkor töltődnek be az operatív memóriába, ha azokat a program meghívja, vagyis ilyenkor dinamikusan szerkesztődik össze a hívó program és a DLL-ből meghívott eljárás. Az eredeti angol „library” azaz könyvtár kifejezés arra utal, hogy a fájl adatokat tárol, amelyeket a programok szükség esetén „kikölcsönöznek”. (Kissé zavaró lehet, hogy a könyvtár az informatikában más megközelítésben mást [is] jelent...)

- A memória hézagmentes kitöltése
 - Egy számítógép működése közben programok indulnak, futnak majd befejeződnek, azaz végül az adott program által használt memóriaterület felszabadul. Különböző méretű programok lefutása után a legkülönbözőbb méretű, felaprózódott szabad helyek keletkeznek a memóriában. Célszerű tehát a darabokból álló szabad helyek összevonása egybefüggő szabad helyé. Elvileg és gyakorlatilag egy egybefüggő logikai címtartomány nem igényel egybefüggő fizikai címtartományt (mesterséges folytonosság).
- A háttértárak dinamikus használata a memória kiterjesztésére
 - Amennyiben egy program futása valamely okból jelentős ideig várakozni kényszerül (pl. egy másik program eredményére vár), akkor célszerű az operatív tárból a háttértárba áthelyezni. Így az operatív tárból felszabadult helyen más programokat futtathatunk.

A társzervezés módjai és megvalósulásai esetében fontos megjegyezni, hogy a társzervezési algoritmusok szükség képpen mindig veszteségesek, de a veszteség mértéke nem mindegy, hogy mekkora. Cél a minél kisebb veszteség elérése. A címtranszformáció a logikai és a fizikai címek kapcsolatának megvalósítását jelenti.

- Egypartíciós rendszer

Ez esetben az operációs rendszer által használt tárterületen túl, egy darab egybefüggő címtartomány létezik. Az operációs rendszer védelmét egy határregiszter biztosítja, ami a program(ok) által használható legkisebb címet tartalmazza. Az előző mondatban a zárójel azért szerepel, mert ez a megoldási mód jellemzően egyetlen program futása esetén volt használatos.

- Többpartíciós rendszer

A multiprogramozás igényeinek kiszolgálására használatos rendszer, mely több program egyidejű operatív tárban történő tárolását, futtatását teszi lehetővé, minden program számára külön memória partíciót biztosítva. A megfelelő védelemhez partíciónként kettő (egy alsó és egy felső) határregiszterre van szükség.

Ez nagyon szépen hangzik, de fix méretű partíciók esetén nagyon rossz a kihasználtság hatékonysága. Ha a programok nem használják ki a rendelkezésükre álló memóriaterületet, akkor az belső tördelődéshez (Internal Fragmentation) vezet. Azaz kihasználatlan memória területek lesznek az adott partíción belül.

Dinamikusan kiosztott, azaz a programok igényeihez méretezett partíciók esetében viszont külső tördelődés (External Fragmentation) állhat elő. Azaz az adott partíciók között kihasználatlan memória területek keletkezhetnek, melyek összes területe jelentős lehet.

A külső tördelődés okozta veszteségek minimalizálására több algoritmus is rendelkezésünkre áll.

- Legjobban illeszkedő (Best Fit)

Ebben az esetben a foglalás a legkisebb, de még elégséges méretű területből fog megtörténni, így a maradék, azaz a fel nem használt, kihasználatlan terület a lehető legkisebb lesz.

- Első megfelelő (First Fit)

Ebben az esetben a hely keresése a tár elejéről indul, és az első megfelelő méretű terület kiválasztása a cél, függetlenül attól, hogy az mennyivel nagyobb a lefoglalandó területnél. A megoldás jellemzően inkább gyors, mint terület hatékony.

- Következő megfelelő (Next Fit)

Ez az előző algoritmus egy módosított változata, melyben a keresés nem a tár elejétől indul, hanem közvetlenül az utoljára lefoglalt tartomány után, így szintén főleg a sebességre nézve hatékony.

- Legkevésbé illeszkedő (Worst Fit)

Ebben az esetben a foglalás a legnagyobb rendelkezésre álló szabad területből fog megtörténni. A választás logikája az, hogy a rendelkezésre álló legnagyobb szabad területből a választás (azaz a foglalás) után még kellően nagy, azaz másik folyamatok számára is választható méretű terület marad.

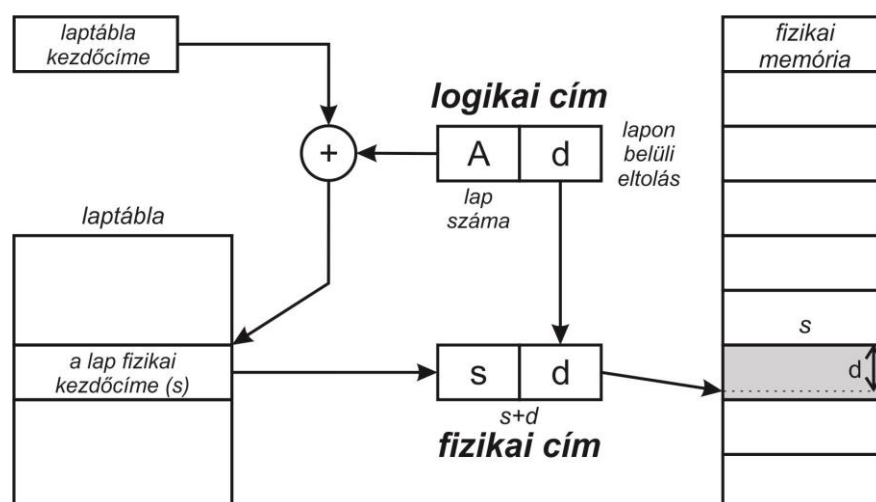
Amennyiben a dinamikus kiosztás folyamatos átméretezéssel, dinamikus áthelyezhetőséggel is társul, akkor a tördelődések szinte teljesen elkerülhetők, de ne felejtjük el az ehhez szükséges erőforrásigényt, és a ráfordítandó időt. A gyakorlatban hatékonyabb működést eredményez az, ha beletörődünk némi memória veszteségbe (és kellően nagyméretű memóriánk van ennek elviselésére).

- Tárcsere (Swapping)

A tárcsere esetében az operatív tárat időben osztjuk meg a folyamatok között. Azaz egy program vagy az operatív tárból vagy valamilyen háttértárból tartózkodik. Erre jellemzően akkor van (klasszikus esetben volt...) szükség, amikor egyszerre csak egy futó program fér el az operatív tárból.

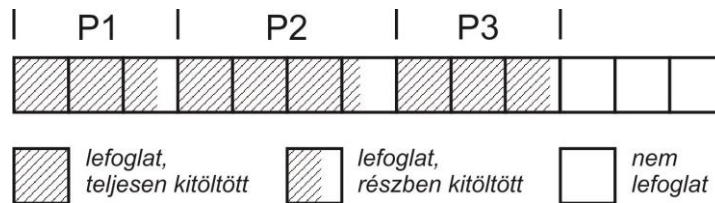
- Lapszervezés

A logikai és a fizikai címtartomány egyaránt azonos, rögzített méretű lapokra van osztva. Jellemző a kettő egész hatványai (azaz az informatikai szempontból kerek számok) szerinti kiosztás. Fix méretek esetében (pl. kész szoftverek) célszerű a használata.



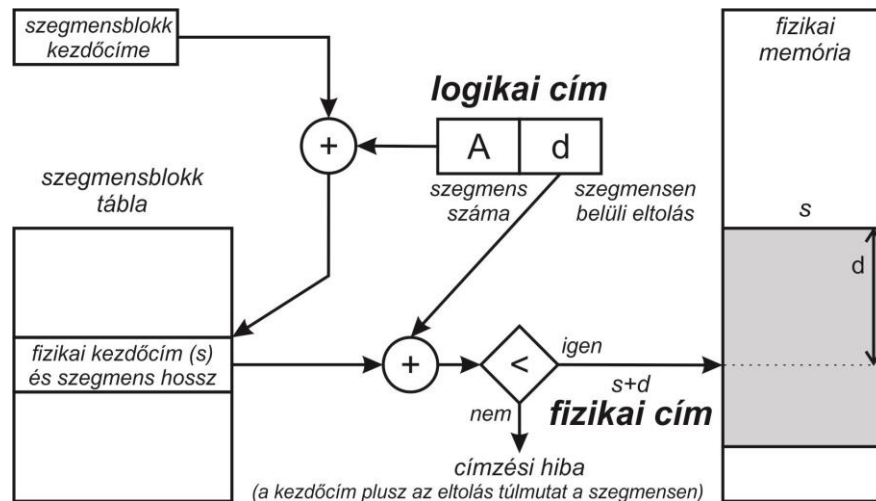
A logikai címtartományban egy címet, azaz egy tároló rekeszt a logikai lapcím és egy eltolás érték jelenít meg. Szükség van ezen kívül egy laptáblára is, mely a logikai lapcímek sorrendjében a hozzájuk tartozó fizikai lapok kezdőcímeit tartalmazza. Mivel a lapméret adott, így eleve nem definiálhatunk azon túlmutató eltolási értéket. Ez esetben a címtranszformációhoz csupán egy összeadás szükséges.

A lapszervezés kísérőjelensége a belső tördelődés, hiszen az egy folyamathoz tartozó fix méretű lapok közül a legutolsó nagyon ritkán van pont tele. A lapokon belül a kihasználatlan részek más folyamat részére nem oszthatók ki.



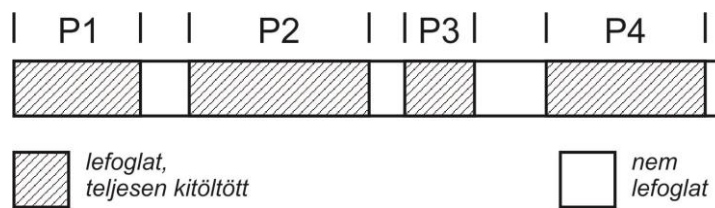
- Szegmensszervezés

A logikai és a fizikai címtartomány különböző méretű lapokra van osztva. Előnye, hogy a lapok az adott programok igényeinek, logikai szerkezetének megfelelően alakíthatóak ki. Változó méretek esetében (pl. adatállományok) célszerű a használata.



A logikai cím ez esetben is két komponensből áll. A tábla kezdőcímét a folyamat környezetét leíró adatszerkezetben tárolja a rendszer. A változó szegmensméret miatt a szegmensen belüli eltolás maximális hossza is szegmensenként eltérő. Ez esetben a címtranszformáció kicsit bonyolultabb, elvégzéséhez két összeadás, és egy méretellenőrzés szükséges.

A szegmensszervezés kísérőjelensége a külső tördelődés, hiszen a változó méretű szegmensek nem feltétlenül képesek a memóriát teljesen kitölteni.



- Kombinált szervezés

A szegmensszervezés és a lapszervezés együttes használata, mely a ma használatos rendszerekre jellemző.

Kombinált szervezés – illetve szegmensszervezés és lapszervezés esetén is – minden egyes folyamat logikai címtartománya 0-tól (a saját nullájától!) indul, de az azonos logikai címekhez nyilván más-más fizikai címek tartoznak, a saját táblák szerint.

2. Virtuális memóriakezelés (VM)

A virtuális memória egy, az operációs rendszer és/vagy a számítógép hardvere által nyújtott szolgáltatás, amit jellemzően egy külső tárolóterület igénybevételevel, a futó program(ok) számára transzpárens módon biztosítja, hogy a program végrehajtáskor a központi vagy operatív memória fizikai korlátai észrevétlenek legyenek. Ez gyakorlatilag azt jelenti, hogy a folyamatok által felhasználható logikai címtartományoknak csak a háttértárak kapacitása szab határt.

Az operációs rendszer úgy szabadít fel operatív memóriát az éppen futó program számára, hogy a memóriában tárolt, de éppen nem használt lapokat kiírja a külső tárolóra, amikor pedig ismét szükség van rájuk, visszaolvassa őket. Mivel a merevlemez sebessége töredéke a memória sebességének, nagyon sok múlik azon, hogy a virtuálismemória-kezelő milyen stratégiát alkalmaz az operatív memóriából kimozgatandó lapok kiválasztásakor.

Az alapvető cél, hogy „minden” az operatív tárban legyen objektíve elérhető, főleg több folyamat esetén. Valójában elég, ha csak az éppen futó folyamat futásához éppen szükséges adatok vannak az operatív tárban, de emellett minden más folyamat úgy hivatkozhat logikai memória címekre, mintha azok fizikai címek volnának. A tárolt információk (folyamatos) mozgatása az operatív tár és a háttértár között valósíthatja meg ezt a modellt.

A fizikai címekhez kötött logikai címek használata alapvető a modern operációs rendszerek működése szempontjából.

Elvárások:

- a programok gyorsabban töltődjenek be, mert induláskor csak a szükséges kódrészletek töltődnek az operatív tárba
- több folyamatot futtathatunk egy időben, ha csak az van a fizikai memóriában, ami tényleg szükséges
- tudjunk olyan folyamatokat is futtatni (egyszerre akár többet is), melyek több memóriát igényelnek, mint a rendelkezésre álló fizikai memória
- a folyamatok legyenek képesek osztozni közös erőforrásokon, adatokon és adatszerkezeteken, illetve kódokon
- a folyamatok megoszthatnak memóriaterületeket egyedi hozzáférésekkel (írás, olvasás), ezekről nyilvántartást kell készíteni

A laphiba, mint fogalom azt jelenti, hogy olyan memórialapra hivatkozunk, amely nem az operatív tárban, hanem valamelyik háttértáron van a hivatkozás pillanatában. A laphiba mentes időszakok hossza a memória méretétől, illetve a futtatni kívánt programok számától és méretétől függ.

Probléma akkor keletkezik, ha a fenti feltétel nem teljesül. Az operatív memóriában nem található lapra történő hivatkozáskor következik be a laphiba. A laptábla bejegyzés azt jelzi, hogy az elérni kívánt lap nincs a fizikai memóriában, illetve a logikai cím nagyobb egy előre megadott korlátnál, vagy a folyamat olyan memóriacímet próbált elérni, amihez nem tartozott memória, vagy tiltott módon akar hozzáférni a memóriához (pl. csak olvasható részt írni akar), akkor a program a futását megszakítja.

A laphibát megszakításként kell értelmeznie az operációs rendszernek, és kezelnie kell. Például: Ha az adott lap nincs a memóriában, de a HDD-n megtalálható, akkor azt onnan be kell olvasni, majd az operációs rendszernek vissza kell adnia a vezérlést a megszakított folyamatnak.

Amennyiben a fizikai memória az előbbi esemény bekövetkeztekor éppen teljesen tele van, akkor legelőször is helyet kell felszabadítani. [lásd: Lapcsere algoritmusok] Eközben a folyamat természetesen várakozik.

Lapozási stratégiák:

- Igény szerinti lapozás:
 - Csak laphiba esetén, és csak a laphibát megszüntető lapot hozza be a fizikai memóriába.
 - Csak a szükséges lapok vannak/legyenek a fizikai memóriában.
 - Az új lapra vonatkozó hivatkozás jellemzően hosszú várakozást eredményez.

- Előretekintő lapozás:
 - Előre tekintve (becslés) az operációs rendszer megpróbálja „kitalálni”, hogy mely lapokra lesz szükség, és azokat is behozza. (Azt, hogy a fizikai tár mekkora része használható fel erre a célra, korlátozni kell.)
 - Feltétel a megfelelő mennyiségű szabad erőforrás (CPU, HDD, fizikai memória).
 - Ha a behozott lapokra tényleg szükség lesz (sikeres a becslés), akkor csökken a laphibák száma.
 - Ha nem, akkor feleslegesen használunk erőforrásokat, és lassítottuk a rendszert.

Lapcsere algoritmusok:

Az alaphelyzet az, hogy tele van a fizikai memória, és laphiba történik. Melyik lapot írjuk ki a háttértárba? Legalább egyet választanunk kell, hiszen ennek a helyére kerül majd be a kívánt adat a háttértárból. (áldozat kiválasztás / Victim Selection)

- Optimális algoritmus
(gyakorlatilag nem realizálható az abszolút optimális választás, de viszonyítási alapnak mindenképpen jó)
- Legrégebbi lap (FIFO)
(a múlt alapján próbálja meghatározni a jövőt, a tényleges használatot nem veszi figyelembe, csak a sorrendet)
- Újabb esély algoritmus (Second Chance)
(mivel tárol egy úgynevezett „Reference bit”-et a korábbi használatokról, szintén a múlt alapján dönt, de a korábbi használatok alapján)
- Legrégebben nem használt (Least Recently Used, LRU)
(egy úgynevezett „last used” időbélyeget tárol)
- Legkevésbé használt (Least Frequently Used, LFU)
(szintén egy számlálót használ a döntéshez)
- Utóbbi időben nem használt (Not Recently Used, NRU)
(kétszintű prioritást használ a döntéshez, két bittel: „hivatkozott” illetve „módosított”)

Minél több tárolt adat segíti a döntést, annál inkább HW támogatást igényel a módszer, különben a futási ideje megnőhet.

Fontos szempont az eddig felsoroltak mellett a terheléelosztás, azaz amikor az operációs rendszer és egy háttértár terheltsége is alacsonyabb, olyankor lehetőség nyílik a virtuális memória lapjainak rendezésére, azaz olyan lapok mozgatására is, melyek cseréje csak hosszabb távon elengedhetetlen.

Az operatív tár, azaz a fizikai memória és a virtuális tárkezelés stratégiai algoritmusai nem csak egy folyamat szempontjából vizsgálható meg, hanem a rendszer egésze szempontjából is. A lapcsere algoritmusok végrehajtása közben merül fel az a kérdés, hogy egy adott folyamat laphibája esetén a végrehajtás milyen hatással lehet a többi folyamatra. A döntési helyzet pedig az, hogy a lapcsere algoritmus csak a laphibát produkáló folyamat saját lapjai közül (lokális memória-gazdálkodás) válasszon, vagy a rendelkezésre álló teljes lapkészletből (globális memória-gazdálkodás).

A globális memória-gazdálkodás egyenletesebb terhelés eloszlással jár, de szükségképpen előtérbe helyezi a nagyobb, több hivatkozással bíró folyamatokat, tulajdonképpen folyamatosan kiszorítva a kicsi folyamatokat. A lokális memória-gazdálkodás közel egyenlő esélyt biztosít a kicsi és a nagy folyamatok részére, viszont így túl statikus ahhoz, hogy a terhelés ingadozásokat hatékonyan legyen képes kezelni.

A két együttesen megvalósítandó cél – a multiprogramozás fokának javítása (és ezzel hatékonyabb erőforrás kihasználás biztosítása), valamint a virtuális memória lapok számának csökkentése (amivel egyre csökken az egy folyamathoz rendelt lapok száma, és így a laphibák gyakorisága növekszik, a CPU feleslegesen terhelődik a laphiba megszakításokkal) – egyszerre és együttesen nem valósítható meg, valamilyen kompromisszumra van szükség.

Az optimális szint eléréshez először is el kell dönteni, hogy melyik paraméter optimumát próbáljuk megvalósítani. A multiprogramozás fokának van elvi optimuma, de gyakorlatilag azt mondjuk, hogy minél nagyobb, annál jobb. A másik paraméter, az egyes folyamatokhoz tartozó lapok száma viszont egyértelműbben optimalizálható. Minden folyamatnak maximum annyi lapra lehet szüksége, hogy a legösszetettebb, legtöbb lapra hivatkozó utasítása is le tudjon futni, illetve a lapok száma nem kell hogy több legyen, mint a folyamat logikai címtartománya.

Az optimumot, azaz az egyensúlyi állapotot úgy fogalmazhatjuk meg, hogy az egyes folyamatok esetében a két laphiba közötti átlagos futási idő haladja meg a laphiba lekezelésének idejét. Másképp fogalmazva, egy folyamatnak legalább annyi lapot kell biztosítani az egyensúlyi állapothoz, amennyi lapra a laphiba lekezelésének ideje alatt hivatkozik. Ez az érték a működő lapkészlet (Working Set). Az elmélet természetesen ez esetben is szebb, mint a gyakorlat, hiszen egy programnak a futása közben, különböző futási szakaszaiban a működő lapkészlete más és más méretű lehet.

Az operációs rendszer a lokális és a globális memória-gazdálkodás között az egyensúlyt egy úgynevezett dinamikus lokális memória-gazdálkodásban találhatja meg, melyben a laphibákat lokálisan kezeli, de a folyamatok igénye szerint képes a lapok globális átcsoportosítására is. Ha számottevő az eltérés egy folyamat működő lapkészlete és a folyamat által a valóságban használt lapok számával, akkor célszerű a lapokat átcsoportosítani. Tekintettel arra, hogy a laphiba gyakoriságának mérése lényegesen egyszerűbb, mint működő készlet meghatározása, a kiindulási alap a laphiba gyakorisága. Túl sok laphiba esetén a folyamathoz kiosztott lapok száma nem elegendő, a lapok számát növelni kell. A másik esetben, amikor szinte elenyésző a laphibák száma, felmerül a kérdés, hogy a folyamat nem birtokol-e esetleg túl sok lapot – másik futó folyamatok kárára.

3. Hasznos linkek

[Mi a virtuális memória? \(Microsoft\)](#)

[A virtuális memória méretének módosítása \(Microsoft\)](#)

Előadás_#08.

1. Állományrendszerek

Rövid hardver áttekintés a lemez alapú háttértárakról:

FDD 18 szektor, 80 sáv, 2 oldal, 1-1 fej

HDD nagyszámú szektor és sáv, ZBR, lemezenként 1 oldal és 1 fej

ODD spirál vagy ZBR, lemezenként 1-2 oldal, 1-2 réteg és 1 fej

Leggyakoribb fájlrendszerek:

- FAT, FAT12, FAT16, FAT32, exFAT

FAT: File Allocation Table / Fájl Allokációs Tábla

Az egyik legelső (a kódot 1977-ben készítette Marc McDonald), fix méretű blokkokból álló, láncolt lista szerkezetű fájlrendszer, az informatika nagy túlélője. Egyes változatai még ma is használatosak a FDD és HDD technológiákban.

[A FAT16 és a FAT32 összehasonlítása \(Microsoft, angol nyelvű\)](#)

[Az exFAT és FAT32 összehasonlítása \(NTFS.com, angol nyelvű\)](#)

- HPFS

HPFS: High Performace File System / Nagyteljesítményű Fájl Rendszer

Az IBM által kifejlesztett rendszer, amely kb. 200MB lemezkapacitás felett mutat érdemi előnyöket, pl. évekig nem kell töredezettség mentesíteni (B+FA stuktúra), a főkönyvtár fizikailag nem a lemez elején, hanem a közepén helyezkedik el. Sajnos a Microsoft a HPFS-t csak rövid ideig támogatta, végül folyamatosan kiszorította a támogatott fájlrendszerek közül.

[A FAT, a HPFS és az NTFS fájlrendszer áttekintése \(Microsoft\)](#)

- NTFS

NTFS: New Technology File System / Új Technológiájú Fájlrendszer

Ezzel egy hamarosan következő előadás részletesen foglalkozik.

- UFS

UFS: Unix File System / Unix Fájl Rendszer. Első verzióját 1984-ben fejlesztették ki a Berkley egyetemen. Paraméterei minden szempontból jobbak mint az akkori FAT rendszereké. Egyedi tulajdonsága például a szuperblokk, ami egy úgynevezett "magic number"-t (és még jó pár paramétert) tartalmazott a használt lemez geometriájával, statisztikáival és finombeállításával kapcsolatban. Arról sem szabad elfeledkezni, hogy a Unix mindet (beleértve a fizikai eszközöket, perifériákat is) fájlként kezel.

- NFS
NFS: Network File System / Hálózati Fájlrendszer.
A Sun Microsystem fejlesztése, ami hálózatos (pontosabban TCP/IP feletti) rendszerek között biztosít transzparens állomány elérést, első verziója 1984-ben jelent meg. Valójában nem fizikai fájlrendszer hanem egy, a megosztást biztosító szoftverréteg. Működése szerver-kliens alapú, és mára elmondható, hogy minden elosztott fájlrendszer alapja az NFS.
- VFS
VFS: Virtual File System / Virtuális Fájlrendszer
Valójában nem fizikai fájlrendszer hanem egy új absztrakciós réteget biztosító szoftveres virtualizációs megoldás, első verziója 1985-ben jelent meg. A VFS tulajdonképpen egy interfész a kernel és a konkrét fizikai fájlrendszer között, melynek segítségével egy virtuális gépen futó alkalmazások anélkül érik el a fizikai tárolókat, hogy azoknak a különböző fizikai fájlrendszereit ismernék.
- JFS
JFS: Journal File System / Naplózó Fájlrendszer
A JFS az IBM fejlesztése a HPFS továbbfejlesztéseket, első verziója 1990-ben jelent meg. A JFS a naplózó technológia nyílt forráskódú alapja. A naplózás segítségével akár a rendszerösszeomlás, illetve az áramkimaradás okozta adat konzisztencia problémák is szinte minden esetben automatikusan javíthatók.
- EXT, EXT2, EXT3, EXT4
EXT: Extended File System / Kiterjesztett Fájlrendszer
Az első EXT fájlrendszert 1992 áprilisában írta Rémy Card a Linux kerneléhez. A Linux kernelben már a 0.96c verzió óta meglévő VFS első érdemi, és széles körben elterjedt felhasználása az EXT fájlrendszer. Az EXT3 óta a JFS-ben megismert naplózási képesség is a fájlrendszer része.
- LTFS
LTFS: Linear Tape File System / Szalagos Fájlrendszer
Az LTO rendszerű mágnesszalagok jellemzően archiválásra használt fájlrendszere, amely a HDD-hez hasonló adatszerkezetet és adatkezelést biztosít. Első verziója 2000-ben jelent meg.
- CDFS, UDF
CDFS: Compact Disc File System / Kompakt Lemez File Rendszer
UDF: Universal Disc Format / Univerzális Diszk Formátum
Az optikai lemezek lezárt illetve nem lezárt fájl rendszerei, lemezformátumai.
[Milyen CD- vagy DVD-formátumot érdemes használni? \(Microsoft\)](#)

Fájl (File) – magyar megfelelője az állomány – alatt az összetartozó információk gyűjteményét értjük. Tartalma lehet program, adat, szöveg, ábra, vagy bármely kódolt információ. A fájlok perifériákon tárolódnak az operációs rendszer által használt fájlrendszernek megfelelően, jellemzően könyvtár struktúrában. A felhasználó egy fájlra a fájl egyedi azonosítójának (elérési út, név) segítségével hivatkozhat. Egy fájl használatához a felhasználónak - ha az elérési utat és a fájlnevet ismeri – nem szükséges ismernie a fájl tárolásának fizikai paramétereit: azaz hogy melyik fizikai eszközön, illetve az eszközön belül pontosan hol található a fájl; az eszköz milyen csatornán illeszkedik a rendszerhez; mi a tárolás adategysége.

A fájlrendszer tehát egy olyan szoftveres réteg, ami azt biztosítja, hogy a felhasználó a fizikai tárolót megfelelő jogosultsággal elérhesse, az abban használatos könyvtár struktúrában adatokat tudjon tárolni, illetve a tárolt adatokat vissza tudja nyerni. A fájlok a felhasználó egyéni céljai szerint, illetve az operációs rendszer szempontjai szerint könyvtárakba (Directory) csoportosíthatók. A könyvtárak és fájlok csoportosításának következő lépcsője a kötet (Volume).

A fájlkezelő (állomány kezelő) rendszer a következő feladatokat kell hogy ellássa:

- adatcsere, információátvitel a fájlok és a folyamatok tárterülete között
- műveletek fájlokon, könyvtárakon és köteteken
- osztott fájlkezelés, azaz a fájlok akár több folyamat egyidejű rendelkezésére is lehetnek bocsájtva
- a hozzáférések, jogosultságok szabályozása
- a tárolt információ illetéktelen hozzáférés elleni védelme, titkosítása
- a tárolt információ sérülések elleni védelme

A fájlkezelő rendszer az operációs rendszer I/O (be- és kimeneti) rendszerének szolgáltatásaira épül, gyakorlati megvalósítása további egymásra épülő szoftver rétegek segítségével történik. Az adatok az egyes rétegek szolgáltatásain keresztül, rétegenként kisebb-nagyobb átalakítások és konverziók során jutnak el a háttértárból az operatív tárba illetve az operatív tárból a háttértárba.

A fájlkezelő rendszer rétegei (a legalsó rétegtől elindulva):

- Készülék kezelő (Device Driver) réteg feladatai:
 - közvetlenül a tárolóeszköz (háttértár) vezérlése
 - az operatív tár és a háttértár közötti adatátvitel lebonyolítása
 - jellemző a megszakítások (IRQ) illetve a közvetlen memóriaelérés (DMA) használata
- Elemi átviteli műveletek réteg feladatai:
 - egy adott fájl leképzése optimális számú és méretű blokkra
 - konverzió a központi tár illetve a háttértár címzési rendszere között
 - az operatív tár és a háttértár sebesség különbségének kezelése gyorsítótárak (cache) igénybevételével
- Állományszervező réteg feladatai:
 - fizikai és logikai blokkok, fájlok, könyvtárak nyilvántartása
 - szabad területek nyilvántartása
 - blokkok logikai összefűzése
 - dinamikus helygazdálkodás
 - új fájlok létrehozásakor, vagy meglévő fájlok méretének növelésekor szükséges adminisztráció elvégzése, szabad hely biztosítása
 - fájl törlése után a hely felszabadítása
 - fájl elérés időbeli ütemezése, szinkronizálása, a hozzáférés szabályozása
 - jogosultságok kezelése

Az operációs rendszer által egységként kezelt lemezterület a blokk. A blokk jellemzően néhány szektort (ZBR szervezésű tárolás esetén természetesen zónát) tartalmaz. A fájlok jellemzően több blokkból állnak.

A szektorméret a tárolóeszköz fizikai paramétere, az viszont, hogy egy blokk hány szektort tartalmaz az paraméterezhető. A blokkméret meghatározását szervezési, tárolási és hatékonysági szempontok befolyásolják. Minél nagyobb blokkokat használunk, annál kevesebb az egységnyi információ átviteléhez szükséges többlet művelet illetve idő. Másrészt viszont, ha a nagyméretű blokkokat az információ nem tölti ki teljesen, akkor feleslegesen foglalunk területet mind a háttértárban, mind az operatív tárban, ami belső tördelődési veszteséghez vezet.

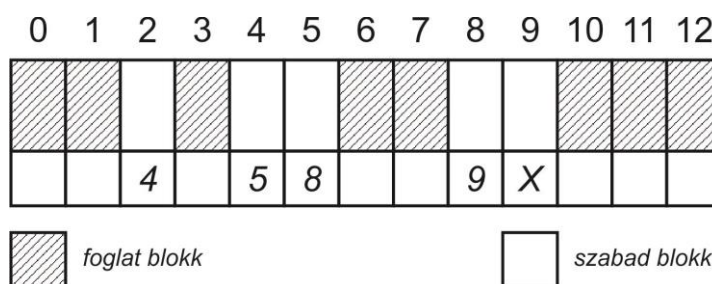
A szabad helyek nyilvántartása (ami a fentiek szerint alapvetően az állományszervező réteg egyik fontos feladata) többféleképpen is megvalósítható:

- Bittérképes ábrázolással

Ebben az esetben a háttértár (jellemzően merevlemez) minden egyes logikai blokkjához egy bit tárolja, hogy az adott blokk szabad-e. Ezen a bitek segítségével foglaltsági vektor állítható elő, ami a lemez egy erre fenntartott helyén tárolódik. Ezzel a tárolási formával az egymás melletti szabad blokkok kiválasztása egyszerű (de sajnos csak ez). A hatékonyságot és a gyorsaságot növelendő, célszerű az egész vektort az operatív tárba letükrözni.

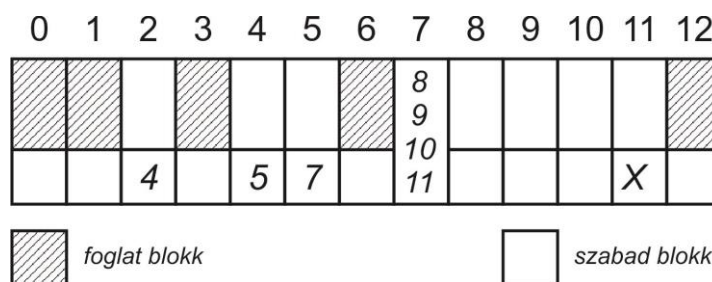
- A szabad blokkok láncolt listájával

Ebben az esetben minden egyes szabad blokk utolsó néhány bájtyát kiemeljük a klasszikus adattárolási funkcióból, ugyanis ezen a néhány bájton a következő szabad blokk sorszámát tároljuk. A módszer annak ellenére sem túlzottan hatékony, hogy a lista bejárásához elegendő az első szabad blokk ismerete, hiszen a lista bejárása lassú, sok lemezműveletet igényel.



- A szabad blokkok csoportjának láncolt listájával

Ebben az esetben egy szabad blokk jellemzően nem csak egy, a soron következő blokk címét tartalmazhatja, hanem „n” darab blokk címét, melyek közül „n-1” darab ténylegesen szabad. A címgyűjtemény listáját tartalmazó blokk technikailag foglalt (de az operációs rendszer szempontjából természetesen szabad). Azaz ciklikusan egy teljes blokkot a nyilvántartás céljaira foglalunk le. Teljesítménye, hatékonysága jobb, mint a szabad blokkok láncolt listájának hasonló paraméterei.



- Az egybefüggő szabad területek leírásával

Ebben az esetben a szabad blokkokra nem egyesével hivatkozunk. A tárolás lényege, hogy az egybefüggő terület első blokkjának címe mellett a terület hosszát tároljuk el. Ezen tárolási forma nyilván akkor hatékonyabb, ha a szabad területek átlagos hossza minél nagyobb.

A szabad helyek nyilvántartása mellett nyilván fontos feladat a fájlok tárolásának a megoldása, azaz a logikailag összefüggő foglalt helyek és azok sorrendjének nyilvántartása. Ez a nyilvántartás is többféle módon valósítható meg.

Allokációs módszerek:

- Folytonos terület használata

Ebben az esetben az összetartozó információk csak egymást követő blokkokban tárolódhatnak. A nyilvántartáshoz elegendő ismerni az első blokk sorszámát, valamint a tároláshoz használt blokkok darabszámát.

- Előnyei:

Ez a módszer akkor célravezető, ha a szabad területek átlagos hossza jelentős számú blokkból áll. A tárolt információ elérése egyszerű, az összetartozó adatok sorban követik egymást. Ugyan a rendszer szemléltetést rugalmatlan, de például tárcsere esetén – mivel ismerjük az elmentendő címtartomány méretét, és a blokkok sorban követik egymást, így azok egyszerre, egy lépésben mozgathatók – hatékony és gyors megoldást biztosít.

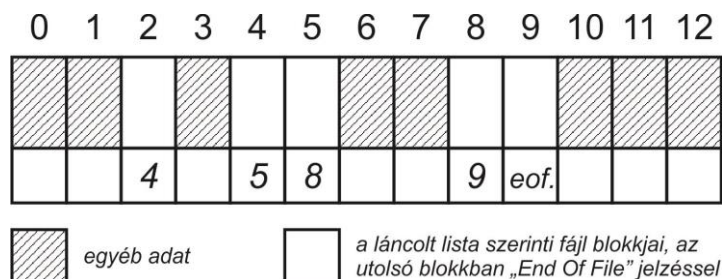
- Hátrányai:

A külső tördelődés a módszer egyik kellemetlen velejárója, aminek a hatásait a változó méretű partícióknál megismert algoritmusokkal (Best Fit, First Fit, Next Fit, Worst Fit) lehet ellensúlyozni. Amennyiben a külső tördelődés közvetlenül kezelhetetlen szintet ér el, a megoldás az állományok átmásolása egy másik háttértárra, ezután a terület teljes felszabadítása, végül az állományok új struktúrában történő visszamásolása.

További hátrány az, hogy a gyakorlatban a szükséges blokkok száma csak megbecsülhető, de pontosan előre nem ismert. Amennyiben a lefoglalt terület kicsinek bizonyul, és közvetlenül a terület után nem található szabad blokk, akkor az operációs rendszer kénytelen az egész területet egy nagyobb területre átmozgatni. Amennyiben ez nem lehetséges (és a tár más módon sem szervezhető át belátható időn belül), akkor legrosszabb esetben a programunk futása le fog állni.

- Láncolt lista

Ebben az esetben (a szabad területek nyilvántartásánál már megismert módon) a tárolás szekvenciális és mindig a sor elejéről indul, oly módon, hogy maga a blokk tartalmazza a következő blokk számát, ami hasznos területet vesz el a blokkból.



A nyilvántartáshoz a fájlok első és utolsó blokkjának sorszámát elegendő tárolni.

- Előnyei:

Ebben az esetben külső tördelődés nem lép fel. Az adatszerkezet dinamikus, a fájl méret, azaz a felhasznált blokkok száma tetszőlegesen (egyéb, a fájlrendszer szabta korlátok ismeretében) növelhető.

- Hátrányai:

A szekvenciális tárolásból fakadóan egy fájl eléréséhez a teljes listát lépésről lépésre végig kell járni. A blokkok tartalma két részre van osztva (adat illetve a következő blokk címe), ennek adminisztrálása erőforrást igényel. Egyetlen blokk meghibásodása a tárolt adatok részleges vagy teljes elvesztését okozhatja.

- Láncolt lista központi láncelem táblával

Fájl allokációs tábla (File Allocation Table / FAT)

Ez a módszer a láncolt tárolás olyan megvalósítása, ahol a fájlok adatait és a láncelemeket (a következő blokk sorszámát) elkülönítve tároljuk. A láncokat az úgynevezett fájl allokációs tábla tárolja. A táblában a tároló (lemez) minden egyes blokkjához tartozik egy bejegyzés (pointer), ami abban az esetben, ha a szóban forgó blokk egy fájlhoz tartozik a fájl következő blokkjára mutat. Az állományleíróban csak az egyes fájlok első blokkjának sorszámát kell tárolni, hiszen a többi sorszám a FAT-ban tárolódik.

Az ábrán egyrészt az látszik, hogy az első két bejegyzés (000, 001) szabvány szerint nincs használatban. Az itt

FAT tábla		
blokk	foglalt	köv.
000	0	-
001	0	-
002	1	003
003	1	005
004	0	000
005	1	006
006	1	011
007	1	009
008	1	007
009	1	eof.
010	0	000
011	1	008

ábrázolt fájl első eleme a 002-es blokkban található (ez az infó kell, hogy az állományleíróban szerepeljen), a többi blokk sorrendje pedig: 003, 005, 006, 011, 008, 007 és végül az utolsó a 009.

A FAT tábla a foglalt és összetartozó, valamint a szabad blokkok tárolására egyaránt alkalmas. A FAT tábla jellemzően a tároló (lemez) elején helyezkedik el, ráadásul – adatbiztonsági megfontolások miatt – két példányban, esetenként jelentékeny tárolókapacitást elfoglalva.

- Előnyei:

Az adatvesztés valószínűsége jelentősen csökkenthető, hiszen maga a FAT tábla két példányban tárolódik. A tárolt információ elérése egyszerűbb és gyorsabb, hiszen elegendő a FAT táblában keresni, nincs szükség a teljes adatblokkok beolvasására a kereséshez. FAT tábla memóriában történő tárolásával a keresés tovább gyorsítható.

- Hátrányai:

Maga a FAT tábla nyilvánvaló hogy területet hasít ki magának a rendelkezésre álló tároló kapacitásból. Mivel a FAT szekvenciális szerkezetű, a tárolókapacitás növekedésével a FAT tábla (illetve a FAT tábla második példánya) számára lefoglalt hely szintén növekszik, nyilván az adattárolásra felhasználható tárolókapacitás rovására.

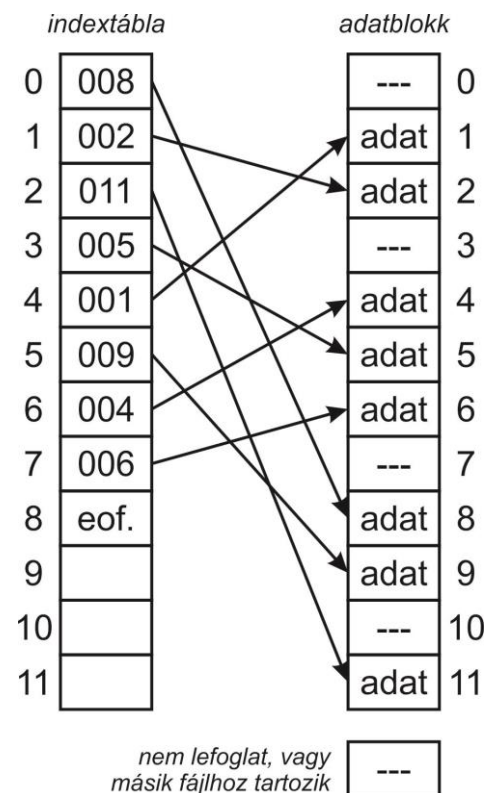
- Indexelt tárolás

Ebben az esetben az egy fájlhoz tartozó blokkok tárolása egy erre a célra fenntartott külön adatterületen, egy úgynevezett indextáblában történik. Egy fájl eléréséhez elég az állományleíróban eltárolt (első) indextáblát tartalmazó blokk címére hivatkozni.

Az itt ábrázolt fájl első eleme a 008-as blokkban található, a többi blokk sorrendje pedig az indextáblában haladva: 008, 002, 011, 005, 001, 009, 004 és végül az utolsó a 006.

- Előnyei:

Mivel az indextábla segítségével minden blokk közvetlenül megtalálható, maga a közvetlen hozzáférés metodikája is egyszerű.



Ráadásul lehetőség van olyan állományok tárolására is amelyeknek nem minden blokkja tartalmaz információt, azaz az állományon belül üres blokkok fordulnak elő. Ez esetben nem szükséges ezen üres blokkokhoz fizikailag is lemez blokkot rendelni, elegendő ezeknek a helyét az indextáblában üresen hagyni.

- Hátrányai:

Viszonylag pazarló helygazdálkodás jellemzi, hiszen az egy fájlhoz tartozó blokkok címeinek tárolásához akkor is legalább egy teljes indexblokkot kell lefoglalni, ha az állomány kis méretű és így csak kevés blokkot tartalmaz. Az indextábla mérete nem határozható meg előre, ezért mindenképpen biztosítani kell azt, hogy maga az indextábla mérete dinamikusan követhesse az igényeket. Ez a szempont megoldható az indexblokkok láncba fűzésével (például a szabad helyek csoportjainak tárolásához használt módon), vagy több szintű indextáblák használatával. Nyilván a két módszer kombinálva is használható, a kis fájlok esetében jellemzően egyszintű, a nagy fájlok esetében jellemzően többszintű indextáblák segítségével.

- Kombinált módszerek

Az optimális tárhely gazdálkodás jellemzően egy, kitüntetett módszer segítségével nem valósítható meg, hiszen a fájlok mérete, viselkedése, címezési módja több szempontból is eltérő. Az operációs rendszernek minden felmerülő esetet (közel) optimálisan és minimális tárolókapacitás veszteséggel kell tudnia kezelni.

- A hozzáférés módja szerint

A célszerűség a gyakorlati tapasztalatok alapján ez esetben az, hogy a soros hozzáférésű fájlok esetében a láncolt, míg a közvetlen hozzáférésű fájlok esetében az folytonos listák használata az optimális választás.

- A fájlok mérete szerint

A célszerűség a gyakorlati tapasztalatok alapján ez esetben az, hogy a kis méretű fájlokat folytonosan, a nagy méretű fájlokat pedig indexelt módon tároljuk és kezeljük.

Az állományokat a belső szerkezetük illetve a hozzáférési módok alapján is csoportosíthatjuk.

Tárolási szempontból egy fájl tulajdonképpen egy bitsorozat, pontosabban bájtsorozat, hiszen a jellemző fájlmanipulációs műveletek bájtonkénti és nem a

bitenkénti szervezésűek. A fájlok belső szerkezete főleg a fájl típusától. funkciójától függ. A futtatható fájlok és az adatfájlok szerkezetüket tekintve eltérőek. A futtatható fájlok utasításokat tartalmaznak, az adatfájlok pedig adatokat.

(Nyilván a tartalom a valóságban ennyire nem egyszerűsíthető le, hiszen például egy *.EXE állomány a végrehajtandó program utasításai mellett számos adatot és grafikus információt is tartalmazhat. A fájlok belső szerkezetének mostani, alapszintű tárgyalásakor csak a numerikus adatokat tartalmazó adatfájlokra helyezzük a hangsúlyt.)

A felhasználó a tárolandó adatokat a fájlokon belül önálló egységekbe csoportosíthatja. Az adatfájlok belső szerkezetük szerint tartalmazhatnak adatmezőket (Field), valamint az adatmezők összetartozó csoportjait, melyeket rekord-nak (Record) nevezünk. A mezők illetve a rekordok hossza lehet fix illetve változó is.

Az operációs rendszereknek jellemzően nem célja a fájlok belső szerkezetének közvetlen ismerete és annak közvetlen értelmezése. A gyakorlatban a különböző felépítésű (a fájl kiterjesztése jellemzően utal a fájl funkciójára) fájlok kezelése, manipulálása megfelelő cél-szoftverrel valósul meg. Egy-egy ilyen cél-szoftvert csak speciális esetben érdemes beleintegrálni az operációs rendszerbe, hiszen ez egyrészt növeli az operációs rendszer helyigényét, másrészt fix funkció lévén csökkenti az operációs rendszer rugalmasságát.

Egy fájl (állomány) a hozzáférési mód, az elérés szerint lehet:

- **Soros azaz szekvenciális**
Ebben az esetben az adatokat csak a tárolt bájtok sorrendjében lehet olvasni illetve írni. Szükség van egy mutatóra (File Pointer) ami az aktuális elérési pozíciót tárolja.
- **Közvetlen (Direct) azaz tetszőleges (Random)**
Ebben az esetben az elérni kívánt információ-elem, az állományon belüli címének (sorszámának) ismeretében, közvetlenül elérhető.
- **Indexelt, index-szekvenciális (Index Sequential Access Method / ISAM)**
Ebben az esetben a rekordok közül indexállományok segítségével igény (tartalom) szerint tudunk választani. Az ehhez szükséges kulcsok egy-egy index állományban rendezett módon vannak tárolva.
- **Partícionált**
Ebben az esetben egy fájl soros rész-állományok alkotják, és maga a fájl tartalmaz egy nyilvántartást arról, hogy ezek a partíciók hol helyezkednek el a fájlban.

A fájlkon végrehajtandó műveletek – a fentiek szerint – vonatkozhatnak a fájl egészére, vagy csak a fájl egy részére. A legáltalánosabb fájl műveletek a következők:

- Attribútumok módosítása
 - A fájlhoz rendelt logikai és numerikus információk, közvetlenül, azaz a fájl tartalmának megváltoztatása nélkül is felülírhatók.
- Adatátvitel: írás és olvasás
 - Közvetlen elérés esetében az információ címét, helyét kell ismerni.
 - Soros, szekvenciális elérés esetben a kezdő pozíciót kell ismerni, illetve az aktuális pozíciót kell tárolni.
- Hozzáírás, bővítés
 - Az állomány végéhez új információt írunk, aminek hatására az állomány mérete növekszik, esetlegesen új blokk lefoglalására kerül sor.
- Pozícionálás
 - Soros hozzáférés esetén megadhatjuk az aktuális pozíciót.
- Állomány megnyitása
 - az állomány megkeresése
 - jogosultságok ellenőrzése
 - az elvégezendő műveletek megadása
 - osztott állománykezelés szabályozása
 - soros hozzáférés pozíciójának beállítása
- Állomány lezárása
 - Puffer használata esetén a ki nem írt információ kiírása, osztott állománykezelés esetében az állomány felszabadítása.
- Programállomány végrehajtása
 - Az operációs rendszer létrehoz egy új folyamatot, a programállományt betölti a folyamat tárterületére és elindítja azt.
- Új állomány létrehozása
 - Szükséges az új allokációs bejegyzés(ek) létrehozása, valamint a szabad blokk(ok) lefoglalása.
- Állomány törlése
 - Szükséges a meglévő allokációs bejegyzés(ek) törlése, valamint a foglalt blokk(ok) felszabadítása.

Az osztott állománykezelés több folyamat esetén jellemzően a kölcsönös kizárás segítségével valósítható meg, ami vonatkozhat:

- a teljes állományra megvalósítható
 - a kizárólagos használat
 - több folyamat is csak olvashatja (azonnal vagy csak a lezárás után)
 - több folyamat is írhatja
(a gyakorlatban sokkal inkább csak az állomány egy részére mód)
- az állomány egy részére megvalósítható
 - a rekord szintű kizárás
 - a holtponthoz és kiéhezés elkerülése

A hozzáférés szabályozásakor nem feledkezhetünk meg arról, hogy a fájlokat (állományokat) minden esetben valamilyen folyamat hozza létre, így a létrehozott fájlok jogosultságai a folyamat, illetve a folyamat felhasználójának (gazdájának) jogosultságaitól fog függeni.

A hozzáférési jogosultságok, fájlokhoz, könyvtárakhoz vagy elérési útvonalakhoz tartoznak. Ezeket a jogosultságokat felhasználónként illetve felhasználói csoportonként lehet definiálni.

Tipikus jogosultságok:

- fájl szinten
 - átnevezés, írás, olvasás, végrehajtás, hozzáírás, törlés
- könyvtár szinten
 - átnevezés, módosítás, listázás, keresés, új file létrehozás a könyvtárban belül, könyvtár törlés

A könyvtárak tulajdonképpen fájlok és más könyvtárak (alkönyvtárak) gyűjteménye, melynek tartalmát a könyvtár nyilvántartás írja le, nyilvántartás bejegyzések (Directory Entry) formájában.

A nyilvántartás bejegyzés a következő elemekből áll:

- az állomány neve
 - A fájlnev vagy azonosító könyvtáranként egyedi, ugyanaz a név több könyvtárban is szerepelhet.
 - Az fájlnev szerkezetileg lehet:
 - homogén karaktersorozat
Például a UNIX esetében a fájlnev korlátozás nélkül bármilyen nyomtatható karaktert tartalmazhat.

- önálló részekből álló karaktersorozat
Például Windows rendszerek esetében megkülönböztetjük a fájlnevet és a kiterjesztést. Egyéb rendszerek esetében további önálló részek is lehetnek, például verziószám.
- az állomány fizikai elhelyezkedését leíró információk
 - hossza
 - hozzá tartozó háttértár blokkok leírása
 - a hozzáférés módja
- Az állománykezeléssel kapcsolatos logikai információk
 - típusa (ha van ilyen)
 - tulajdonosának, létrehozójának azonosítója
 - különböző fájlhoz rendelt időpontok
Leggyakoribb megvalósulásai: létrehozás, utolsó módosítás, utolsó hozzáférés, argumentumok utolsó módosítása, érvényességi idő.
 - hozzáférés jogosultságok
 - hivatkozás számláló
Akkor használatos, ha egy fájlra több különböző néven, illetve több különböző elérési úton lehet hivatkozni.

A nyilvántartás bejegyzésben tárolt információkat a UNIX két részre bontja, ugyanis külön tárolja a kötet-nyilvántartást (Volume Directory) és a fájl nyilvántartást (File Directory).

A háttértáron tárolt információk mellett az operációs rendszerek az operatív tárban is tárolnak állománykezeléssel kapcsolatos információkat.

- az átvitel állapota (folyamatonként)
 - soros hozzáférés aktuális pozíciója
 - megengedett műveletek listája
Egy fájl saját jogosultságainál egy folyamatnak az adott fájllal kapcsolatos műveleti jogosultságai magasabb prioritással bírnak.
- osztott kezeléssel kapcsolatos információk
 - mennyi folyamat kezeli egyidejűleg
 - a kölcsönös kizárást milyen módon, mekkora tartományra kell biztosítani
 - mely folyamatok várakoznak az állomány felszabadulására

Az állományok nagy és esetenként folyamatosan változó száma indokoltá teszi a könyvtárak hierarchikus szervezését. A könyvtár rendszerek legismertebb megoldásai a következők.

- Kétszintű könyvtárszerkezet

Az egyes felhasználók állományai kerülnek egy könyvtárba. Ez a megoldás nem problémamentes, hiszen a felhasználó más felhasználók állományait, illetve az operációs rendszer állományait is szándékozhat használni.

- Fa struktúra

A fa struktúra gyakorlatilag egy irányított gráf, két pont között egyetlen lehetséges útvonallal. A könyvtárak a fájlok mellett további alkönyvtárakat is tartalmazhatnak. A fa struktúrával kapcsolatos fogalmak:

- aktuális könyvtár (Current Directory) folyamatonként

- gyökér könyvtár (Root Directory)

A hierarchia kiindulási pontja, ebből nyílnak a további könyvtárak.

- elérési út (Path)

A gyökér könyvtárból az aktuális könyvtárig vezető útvonal leírása, a közbeeső könyvtárak segítségével. A fa struktúrából következően minden (al)könyvtárhoz csak egyetlen elérési út tartozhat.

- keresési útvonalak (Search Path)

Azoknak az elérési utaknak a gyűjteménye, amely elérési utakon egy adott fájl megtalálása érdekében a rendszer az keresést végez.

- Körmentes irányított gráf

Ellentétben a fenti irányított gráffal, ez esetben két pont között több útvonal is lehetséges, és a gráf szigorúan hurokmentes. Az egyes állományokat a gyökérből így több útvonalon is elérhetjük. Az állományra történő hivatkozás (Link) lehet:

- fizikai

A nyilvántartás bejegyzésben az állományt leíró információt tároljuk.

- szimbolikus

A nyilvántartás bejegyzésben az állomány (egy vagy több) elérési útvonalát tároljuk.

A körmentes irányított gráf használatakor egyes esetekben problémát jelent az, hogy egy adott állományra több elérési úton is lehet hivatkozni, hiszen például mentéskor, vagy használati statisztikák készítésekor csak egy útvonalra lehet hivatkozni. Hasonló helyzet áll elő egy állomány törlésekor is.

- Általános gráf

Ez a struktúra a gyakorlatban nem használatos, hiszen az általános gráfban található hurkok az addig ismertetett problémák mellett végtelen ciklusokat is okozhatnak egy állomány keresésekor. Az ok, amiért egyáltalán megemlítsük ezt a struktúrát az, hogy a körmentes gráfok sérülés esetén (szerencsétlen esetben) részben vagy egészben általános gráffá alakulhatnak UNIX rendszerekben.

A könyvtárakon jellemzően a következő műveleteket hajthatjuk végre:

- Attribútumok módosítása

A könyvtárhoz rendelt logikai és numerikus információk felülírása

- Könyvtár létrehozása

- Könyvtár törlése

Három lehetséges megvalósítási megoldása létezik:

- a könyvtár akkor törölhető, ha üres
- csak a könyvtárban tárolt fájlok törlődnek, de a könyvtárban lévő (al)könyvtárak, és azok tartalma nem
- az összes tárolt információ fájl és könyvtár törlése

- Keresés

- egy nyilvántartásban
 - rendezetlen keresés, teljes bejárással,
 - rendezett keresés, felező eljárással,
- a könyvtárszerkezetben, bejárva a megadott elérési útvonalat

- Új bejegyzés létrehozása illetve törlése

- nyilvántartás méretének dinamikus változása
- blokkok lefoglalása illetve felszabadítása
- rendezettség megtartása

Törlés esetén a törölt bejegyzés első lépésként csak logikailag törlődik, azaz az állományt tartalmazó blokk „foglalt” bejegyzése „törölt” bejegyzésre változik. Ez azt jelenti, hogy egészen a fizikai törlésig (amikor az állományt tartalmazó terület felülíródik új adatokkal) elvileg (és időnként gyakorlatilag is ☺) visszaállítható a törölt anyag.

Előadás_#09.

1. Bemeneti- és kimenetei rendszer (I/O)

Egy számítógépnek a tiszta aritmetikai műveletek végzése mellett, képesek kell lennie a külvilággal való kommunikációra, adatcserére. A számítógép operációs rendszere, az azon futó programok, és a programok által igényelt adatok is valahogy be kell hogy jussanak a működési területre. A programok jellemzően futásuk közben a feladatuk elvégzéséhez szükséges bemenő adatokat igényelhetnek, futásuk végeztével pedig kimenő adatokat produkálnak.

Periféria alatt azokat a hardver elemeket értjük, melyek nem beágyazott részei a számítógépnek, hanem a számítógéphez lettek csatlakoztatva. A számítógép tehát képes a perifériák nélkül is működni, de a perifériák, a számítógép képességeit és lehetőségeit bővítve, kényelmesebb és hatékonyabb működést eredményeznek.

A perifériákat három nagy csoportra bonthatjuk feladatköreik alapján:

- ember – gép kapcsolattartás (HID / Human Interface Device)
billentyűzet, egér, touchpad, monitor, touchscreen, hangszóró, mikrofon, kamera, nyomtató, plotter, szkennер, ujjlenyomat olvasó, joystick, stb.
- gép – gép kapcsolattartás
LAN, WLAN, WWAN, Infra, Bluetooth, NFC, stb.
- adattárolás
mágneses, optikai és flash alapú tárolók

A perifériák működését, viselkedését és használhatóságát meghatározó jellemzők:

- csatlakozási szabvány
soros, párhuzamos, USB, FW, eSATA, SCSI, iSCSI, FC, UTP/FTP (CAT5e, CAT6), DSUB, DVI, DisplayPort, HDMI, Memóriakártyák (SD, CF, MS, xD, MM), stb.
- vezérlőegység
A perifériák általában beépített vezérlőegységet tartalmaznak, de előfordulnak kivételek is. Például: GDI nyomtatás, xD kártya.
A GDI ([Windows] Graphical Device Interface) nyomtatók közbeiktatott lapleíró nyelv nélkül képesek a (Windows) alkalmazásokból nyomtatni.
- működési sebesség
Gyakorlatilag mindig lassabb, mint a CPU sebessége, különös tekintettel a HID eszközökre.

- szinkronizáció

A szinkronizáció azért kiemelten fontos, mert ha a számítógép nem képes a periféria által elvárt mennyiségű, ütemű és sebességű adatátvitelre, akkor az elvárt adatátvitel egy jellemzően lassabb (kisebb adategységű) ismétléssel lesz csak sikeres. A perifériákat a számítógép mellett egymáshoz is szinkronizálni kell, hiszen a perifériák a számítógéppel illetve egymással is képesek a párhuzamos működésre.

A fentiek ismeretében látszik, hogy a különböző perifériákat egyedi, a saját paramétereikhez legjobban megfelelő algoritmusokkal, módszerekkel (IRQ, DMA) kell használni.

Az operációs rendszer közvetlenül a következő feladatokat kell hogy ellássa az I/O eszközök kezelése során:

- egységes alkalmazói felület kialakítása (API / Application Interface)
 - műveletek egységesítése
Fájl alapú periféria elérés esetén csak az utolsó szinten konkretizálódik, hogy mi lesz az adat sorsa. A periféria szempontjából megkülönböztetünk adatfájlokat és vezérlőfájlokat.
 - logikai periféria kezelés
Perifériafüggetlen programozással, azaz az egyes perifériák sajátosságainak ismerete nélkül, általános periféria meghajtók (Device Driver) segítségével kisebb CPU terheléssel lehet kezelni a perifériákat.
 - műveletek átirányíthatósága

- egységes csatlakozófelület kialakítása (DI / Device Interface)

- eszközmeghajtók illesztése, hatékony periféria működtetés

A logikai perifériakezelésnél ismertetett általános periféria meghajtókkal szemben speciális, egyedi periféria meghajtó (Device Driver) használatával egyedi és hatékony perifériakezelés valósítható meg. A párhuzamos perifériakezelés szintén a hatékonyságot növeli. Multiprogramozás használatával lehetővé válik, hogy folyamatok I/O-kra várakozzanak, miközben más folyamatok végrehajtás alatt állnak.

Az operációs rendszer szempontjából az I/O műveletek tulajdonképpen adatblokkok mozgatása a memória vagy a háttértár illetve a perifériák között. Az adatblokkok paramétereit az IOCB tartalmazza. Ezen adatblokkokkal alacsony szintű I/O kezelés esetén a következő három művelet végezhető el:

- indítási művelet (Start)

A logikai periféria név ismeretében kiválasztásra kerül a fizikai periféria, amely ha már aktív, akkor kész az adatcserére, ha nem aktív, akkor a kezelőprogram egy Start_Device utasítással indítja el az eszközt. Az operációs rendszer csak az indítást végzi, nem várja meg az I/O művelet teljesülését illetve végét, hiszen eközben a CPU-t más feladatra is tudja használni.

- várakozás/tesztelés (Wait)

Szinkronizációs várakozás, illetve műveleti időkorlátok használatának lehetősége. Az IOCB állapotjelzője ez alatt az átvitel pillanatnyi állapotának megfelelően vehet fel értékeket (elkezdődött, várakozik, fut, befejeződött). Az I/O művelet befejezése után visszatér az I/O műveletet elindító folyamathoz a megfelelő állapotjelzéssel.

- kényszerített leállítás, hiba esetére (Stop)

Az adott várakozási sorból az IOCB-t csak akkor lehet eltávolítani, ha nincs aktív átvitel. A már elindított, de (például az időkorlátig) be nem fejeződött, vagy programhiba miatt megállt I/O műveletek kényszerített megállítása a Stop_Device utasítás használatával lehetséges.

Az alacsony szintű, egyszerű I/O műveletek lehetővé teszik a szinkron (blokkolt, várakozó) illetve az aszinkron (nem blokkolt, továbbfutó) programhívást is.

Az adatmozgatás három legfontosabb paramétere:

- forrás
- cél
- adatmennyiség

A forrás és a cél egyaránt lehet fizikai vagy logikai memóriacím, illetve fizikai vagy logikai periféria név. Az adatmennyiség az adatcsere során küldött, illetve fogadott bájtok számát jelenti.

2. Elosztott rendszerek

Elosztott rendszer alatt jellemzően az úgynevezett lazán csatolt rendszert értjük, nem feledkezve meg arról, hogy az együttműködésnek nem csak ez az egy módja ismert. Lazán csatolt rendszer esetén a feladatok végrehajtása több, egymástól független, önálló CPU-val, lokális memóriával és saját órajellel rendelkező számítógép között oszlik meg. Szorosan kapcsolt rendszer esetén feladatot végrehajtó számítógépek közös órajellel és osztott memóriával rendelkeznek, jellemzően közös operációs rendszer használata mellett. Az erőforrások viszont mindkét esetben közösek, illetve a műveletvégzés is osztottan történik mindkét esetben.

Az elosztott rendszer nem keverendő össze egy általános célú, számítógépekből kialakított hálózattal. Ugyan mindkét megoldás autonóm számítógépekből áll, de a gépek közti viszony azonban eltérő. Egy számítógép hálózat minden egyes gépe egyértelműen azonosítható, címezhető.

Az elosztott rendszer is hálózattal összekötött autonóm számítógépek összessége, de ezen felül rendelkezik valamilyen elosztott szoftverrel is. Az elosztott szoftver (ami akár elosztott operációs rendszer is lehet) két legfontosabb funkciója a feladatok szétosztása és az erőforrások megosztása. Egy elosztott rendszer akkor teljesíti a vele szemben támasztott elvárásokat, ha a felhasználó azt egyetlen, integrált erőforrásnak látja, azaz az elosztottság a felhasználó számára rejtve marad.

Az elosztott rendszerek tehát:

- önálló, autonóm számítógépekből állnak,
- melyeket valamilyen kommunikációs csatorna köt össze,
- és az egyes hardver komponensek összehangoltan működnek.

Elosztott rendszerek esetében a következők a megvalósítandó célok:

- hatékony erőforrás gazdálkodás
- több felhasználó egyidejű kiszolgálása
- a részfeladatok gyors, párhuzamos megoldása
- megfelelő szintű biztonság elérése, akár redundancia igénybevételével
- a stabilitás biztosítása egyes gépek kiesése esetén is

Az elosztott rendszerek által használt hálózati megoldások, topológiák részletes ismertetésére a számítógép hálózatok tárgy keretein belül lesz lehetőség.

Az elosztott rendszerek tehát lehetővé teszik olyan informatikai környezetek kialakítását, amelyeknek jellemzői túlmutatnak az egyéni, azaz egygépes rendszer nyújtotta szolgáltatásokon. Az elosztott rendszerek fő jellemzői:

- erőforrás megosztás (Resource Sharing)
- nyílt rendszer (Open System)
- konkurens működés (Concurrency)
- méretezhetőség (Scalability)
- hibatűrés, megbízhatóság (Fault Tolerance, Reliability)
- átlátszóság (Transparency)

Ezen tulajdonságok együttes megléte az elosztott struktúrának köszönhető. Ez azonban nem jelenti azt, hogy minden elosztott rendszer jellegéből fakadóan teljesíti is ezeket a jellemzőket. A rendszer hardveres és szoftveres komponenseinek ismeretében, a topológia figyelembe vételével az elosztott rendszereket testre kell szabni, oly módon, hogy a fenti jellemzők mindegyike a megfelelő mértékben valósuljon meg. Az egyedi igényeknek megfelelően, az egyes rendszerekben ezek a jellemzők különböző prioritással vannak jelen.

Erőforrás megosztás

Az elosztott rendszerek esetében is mind a szoftveres, mind a hardveres erőforrások megosztása a cél. A hardveres erőforrások megosztása költséghatékony működést eredményez (például csökkenthető a nyomtatók darabszáma). A szoftveres erőforrások megosztása pedig a feladat végrehajtás hatékonyságát növelik, jellemzően több felhasználó esetében.

Azt sem szabad elfelejteni, hogy egyes erőforrások, például a memória abszolút elosztott kezelésének már operációs rendszer szinten is vannak korlátai, hiszen a memória használata részben dedikált módon történik.

Az osztott erőforrásokhoz jellemzően a rendszer bármelyik pontjából hozzáférhetünk, miközben maga az erőforrás fizikailag bármely másik ponthoz kötődhet. Az erőforrások megosztásáért az egyes számítógépeken futó erőforráskezelő (Resource Manager) folyamatok felelnek.

Az erőforráskezelő legfontosabb feladatai:

- az erőforrás megnevezésének és tulajdonságainak publikálása
- kommunikációs interface biztosítása az erőforrás eléréséhez
- az erőforrás párhuzamos elérésének szabályozása

Például egy nyomtató esetében ismerni kell, hogy a nyomtató melyik számítógép melyik portjára van csatlakoztatva, milyen lapleíró nyelv segítségével küldhetők nyomtatási feladatok a nyomtatóra. Kezelní kell a nyomtató nyomtatási sorát az egymás után, vagy egyszerre beérkező feladatok átmeneti tárolásával.

Az elosztott rendszerek az erőforrások kezelésére leggyakrabban a kliens-szerver modellt alkalmazzák. A kliensek egy előre meghatározott protokoll segítségével érik el a szervereket kéréseikkel. A szerverek a szolgáltatás megvalósulásáról, annak aktuális státuszáról hasonló módon tájékoztatják a klienseket. A kliens-szerver modell mindig az adott feladatra (illetve az erőforrás fizikai helyére) vonatkozik, nem magára valamelyik számítógépre. Például az egyes gépeken található fájlok megosztása kapcsán mindegyik gép viselkedhet szerverként és kliensként is, az adatkérés irányának megfelelően.

Nyílt rendszer

A nyílt rendszer elnevezésben a nyíltság azt jelenti, hogy a rendszer szoftveres illetve hardveres bővítése egyszerűen megoldható, azaz nyitott a változtatásokra. Hardveres bővítés alatt további perifériák hozzáadását, a rendszer hardver elemeinek bővítését (például memória, háttértár) értjük. A szoftveres bővítés az adott gép szoftverének újabb funkciókkal történő kiegészítése (például egy addig csak egy időben egy felhasználó által elérhető erőforrás párhuzamos elérését biztosító szoftver telepítése).

A nyílt elosztott rendszerek jellemzői:

- A folyamatok közötti kommunikációs megoldások egységesek, a folyamatok elhelyezkedésétől függetlenek.
- Az osztott erőforrások publikus interface-eken keresztül érhetők el.
- Heterogén rendszerek kialakításának támogatása, különböző gyártók szabványos eszközeiből.
- Szabványos kommunikáció használata az egyes hardver elemek között, előre definiált interface-ek segítségével.

A UNIX operációs rendszer volt az első olyan rendszer, amelyik támogatta a nyílt rendszerarchitektúrát. További rugalmasságot biztosít az, hogy a UNIX egy közkedvelt magas szintű nyelven (C nyelv) íródott, így egyszerűen lehetett új rendszerhívásokkal bővíteni, melyek lehetőséget adtak új perifériák, vagy új szolgáltatások implementálására.

Konkurens működés

A konkurens működés a folyamatok párhuzamos futtatásának az alapja az elosztott rendszer erőforrásainak esetében is. Ez esetben valós párhuzamosságról beszélünk, hiszen minimum a rendszerben lévő számítógépek számának megfelelő számú CPU mag áll egy időben rendelkezésre. (Több CPU-s gépek, illetve többmagos CPU-k esetében a rendelkezésre álló magok száma nyilván csak több lehet.)

Amennyiben az elosztott rendszert kezelő szoftvernek (vagy operációs rendszernek), lehetősége nyílik egy feladatot párhuzamosan végrehajtható részekre bontania, akkor a feladat végrehajtása jelentősen fel tud gyorsulni. Természetesen a konkurens végrehajtás esetén szükség van a folyamatok végrehajtásának időbeli szabályozására, azaz a folyamatok szinkronizációjára, különös tekintettel a közösen használt erőforrások elérésével kapcsolatos műveletekre.

A konkurens végrehajtás szerver és kliens folyamatokra egyaránt vonatkozhat, azaz egy időben több párhuzamosan futó szerver illetve kliens folyamat lehet a rendszerben, melyek hasonló szolgáltatást nyújtanak, illetve vesznek igénybe.

Méretezhetőség

A rendszer méretezhetősége azt jelenti, hogy a rendszer mérete – és ezzel együtt a kapacitása is – lényeges strukturális változtatások nélkül is megváltoztatható, növelhető. Egy jól kialakított rendszer képes követni a működése közben felmerült felhasználói igényeket, a rendszer így szükség szerint rugalmasan bővíthető.

Érdekes példa erre a telefonszámok hatjegyűről hétjegyűre történt változtatása, mely az előzetes tervezés fontosságára hívja fel a figyelmet. A rendszer tervezésekor meg kell találni az egyensúlyt, azaz a műszaki megvalósítás és a költségek helyes egymáshoz való viszonyát, valamint kalkulálni kell a bővítés lehetőségével is.

Egy másik jellemző példa maga a fájl megosztás, hiszen ennek kapcsán lehetőségünk van meghatározni az egyidejűleg létesíthető kapcsolatok számát. Ez esetben az indokolatlanul alacsony szám a rendszer lassulásához vezet, az indokolatlanul magas szám pedig feleslegesen köt le olyan erőforrásokat, melyeket csak a valós igények megjelenésekor célszerű igénybe venni.

Hibatűrés, megbízhatóság

Hiba alatt olyan működési állapotot értünk, mely kezelői beavatkozás nélkül nem szüntethető meg. Meg kell különböztetni a zavartól, amely mint fogalom a kezelő beavatkozása nélkül megszüntethető problémák összefoglaló elnevezése.

Fontos belátni és megérteni azt, hogy egyes hibák bekövetkezése objektíve elkerülhetetlen (például egy HDD fizikai meghibásodása). Amennyiben valamilyen redundáns tárolást használunk (például RAID1), akkor a bekövetkező probléma az elromlott HDD szempontjából valóban egy HIBA, viszont az egész rendszer szempontjából már csak egy ZAVAR. Azaz a rendszerünk továbbra is képes feladatainak ellátására. A meghibásodott HDD-t természetesen mielőbb cserélni kell, hogy a rendszer hibatűrése visszaállhasson az elvárt szintre.

A hibatűrés megvalósításához elengedhetetlen bizonyos redundanciák használata, mind a szoftver, mind a hardver tekintetében.

Egy informatikai rendszert akkor tekinthetünk hibatűrőnek, ha képes a működés során jelentkező hibák felismerésére, kezelésére oly módon, hogy közben a rendszer alapvető működése jelentős mértékben nem változik meg.

Az elosztott rendszerek egyik előnyös tulajdonsága az, hogy akár hardver hibák jelenlétében is nagyfokú rendelkezésre állást biztosítanak. Egy többfelhasználós, de nem elosztott rendszerben egyetlen hiba hatására jellemzően a rendszer elérhetetlenné válik az összes felhasználója számára. Ezzel szemben, amikor egy elosztott rendszerben meghibásodik valamelyik komponens, a hiba csak a meghibásodott komponensen folyó munkát érinti.

Homogén rendszerek, azaz azonos felépítésű és funkciójú gépek esetében egyes gépeket eleve tartaléknak állíthatunk be. Azaz, ha a rendszerben egy gép meghibásodik, a tartalék gép azonnal a helyére léphet.

Léteznek olyan abszolút kritikus rendszerek is, ahol a redundancia egy még magasabb szintje az úgynevezett szavazó rendszer kiépítése indokolt. Ebben az esetben három (minimum 3, és mindenképpen páratlan számú) számítógép párhuzamosan ugyanazt a feladatot hajtja végre. A rendszer végső kimenete, azaz a futtatás eredménye többségi elven keletkezik. Ez azt jelenti, hogy vagy mind a három gép ugyanazt az eredményt produkálja, vagy legalább kettő azonos eredményt produkál. Három különböző eredmény esetén meg kell ismételni a számítást, és/vagy el kell kezdeni a hibás gép(ek) kiszűrését. Ezt az elvet Neumann János dolgozta ki 1956-ban. Ne feledkezzünk meg az akkori és a mostani hibaarányokról sem.

A hibatűrés szoftveres támogatásának jellemző megvalósítása a javító blokkok (Recovery Block) módszere. Ez esetben az érintett szoftver modulokra kell bontani. Minden egyes modul több példányban kerül lefuttatásra, az eredményeket pedig egy önálló programmodul, egy elfogadási teszt ellenőrzi. Célszerű olyan szoftver komponenseket tervezni, amelyek segítségével az adatok jelentős része a meghibásodás észlelésekor visszaállítható, a számítások pedig visszagörgethetők (Recovery, Roll-Back).

Átlátszóság

Az átlátszóság ez esetben azt jelenti, hogy egy elosztott rendszerben a komponensek elosztott természete, az egyes szolgáltatások és erőforrások fizikai helye a felhasználó számára (közvetlenül) nem érzékelhető. A felhasználó a rendszert egy egységes egészként látja. A szolgáltatások igénybe vétele független a szolgáltatások fizikai helyétől, illetve a szolgáltatást biztosító hardvertől és szoftvertől. Ennek eléréséhez hálózati, kommunikációs, menedzsment és integrációs technikákra van szükség.

Az elosztott rendszerek az alábbi a kilenc féle átlátszósággal rendelkeznek:

- Hozzáférés átlátszóság (Access Transparency)
A felhasználó (és tulajdonképpen a futó programok) előtt rejtett módon teszi lehetővé azt, hogy azonos módon történjen a helyi és a távoli erőforrások azonos műveleteket használó kezelése.
- Hely átlátszóság (Location Transparency)
A felhasználó (és tulajdonképpen a futó programok) előtt rejtett módon teszi lehetővé az információs objektumok elérését, az objektumok helyének ismerete nélkül.
- Hálózati átlátszóság (Network Transparency)
Ez egy összefoglaló terminológia, a hozzáférés átlátszóság és a hely átlátszóság együttes értelmezése.
- Konkurencia átlátszóság (Concurrency Transparency)
A felhasználó (és tulajdonképpen a futó programok) előtt rejtett módon teszi lehetővé a folyamatok konkurens együttműködését, osztott információs objektumok használatának segítségével.
- Másolat átlátszóság (Replication Transparency)
A felhasználó (és tulajdonképpen a futó programok) előtt rejtett módon biztosít több másolatot az információs objektumokból, így növelve a rendszer teljesítményét és megbízhatóságát.

- Hiba átlátszóság (Failure Transparency)
A felhasználó (és tulajdonképpen a futó programok) előtt rejtett módon biztosítja a meghibásodások kezelését, lehetővé téve a rendszer használatát hardver- illetve szoftver meghibásodások esetén is.
- Vándorlási átlátszóság (Migration Transparency)
A felhasználó (és tulajdonképpen a futó programok) előtt rejtett módon, az érintett programok működését nem befolyásolva biztosítja az információs objektumok szabad mozgását a rendszerben.
- Teljesítmény átlátszóság (Performance Transparency)
A felhasználó (és tulajdonképpen a futó programok) előtt rejtett módon teszi lehetővé azt, hogy a terhelések változására a rendszer folyamatos ön-átkonfigurálással legyen képes reagálni, így tartva fenn (esetleg növelve) a rendszer teljesítményét.
- Skálázási átlátszóság (Scaling Transparency)
A felhasználó (és tulajdonképpen a futó programok) előtt rejtett módon biztosítja a rendszer bővíthetőségét, oly módon, hogy közben a rendszer struktúrája és algoritmusai változatlanok maradhatnak.

A gyakorlatban a fenti átlátszóságok közül a hozzáférés- és a hely átlátszóság (azaz a hálózati átlátszóság) meglete a legkritikusabb az osztott erőforrások használhatóságának szempontjából, illetve a többi átlátszóság alapját is jellemzően ezek képezik. Nyilván a többi átlátszóság is fontos, de az átlátszóságok egy része eseti jellegű. (Például a skálázási átlátszóság meglete indifferens, ha nem bővítjük a rendszerünket.)

3. Az elosztott rendszerek operációs rendszerei

A valamilyen módon összekapcsolt rendszereken futó operációs rendszereket két fő csoportra oszthatjuk:

- hálózati operációs rendszerek
 - nem biztosítanak átlátszóságot
 - ez egyes csomópontok egyenként elérhetőek
 - programok futtathatók közvetlenül távoli csomópontokon
 - adatok közvetlenül mozgathatóak az egyes csomópontok között
- elosztott operációs rendszerek
 - átlátszóság biztosítása
 - az átlátszóság miatt
 - az egyes csomópontok egyenként nem címezhetőek
 - a programok futtatásának, futásának helye közvetlenül nem határozható meg
 - adatok közvetlen mozgatására a csomópontok között nincs lehetőség
 - számításvándorlás és folyamat vándorlás megvalósítható a terhelésnek megfelelően
 - a felhasználó nem feltétlenül ismeri fel, hogy elosztott rendszert használ

Elosztott fájl rendszerek

Elosztott fájl rendszernek jellemzően azt az operációs rendszerrel szorosan együttműködő szoftver réteget értjük, amely képes biztosítani a távoli fájlrendszerek tartalmának elérését és használatát a helyi fájlrendszer tartalmának elérésével megegyező módon. Azaz ez esetben röviden a fájlok elérésének átlátszóságáról van szó.

Az elosztott fájl rendszerek használatakor a következő tulajdonságokat várjuk el:

- Hozzáférési átlátszóság
A helyi és a távoli fájlok azonos eljárással kezelhetők.
- Elhelyezkedési átlátszóság
A fájlnevek nem hivatkoznak, nem mutatnak a fájlok fizikai elhelyezkedésére.
- Vándorlási átlátszóság
A fájlok a rendszerben az elnevezésük megváltoztatása nélkül mozgathatóak.

- Méretezési átlátszóság

A fájlrendszer maga is méretezhető, azaz amennyiben ezt a terhelés indokolja új komponensekkel a mérete illetve a teljesítménye is növelhető.

- Hibatűrő fájlrendszer

A fájlrendszer egyes komponenseinek hibái esetén a fájlrendszer legalapvetőbb funkcióinak továbbra is elérhetőnek kell maradniuk, ami esetenként csak kompromisszumok árán (csökkent funkcionalitás mellett) valósítható meg.

- A felhasználók mobilitása

Egy adott jogosultságú felhasználó az összes olyan fájlt, melyet jogosult elérni, a rendszer bármely tetszőleges pontjáról valóban el is érheti.

- A fájlok mobilitása

A fájlok a rendszerben futás közben is áthelyezhetők.

A fájlok elnevezése, azaz a fájlok állapottere elosztott fájlrendszerek esetében kétféle megvalósítású lehet:

- Egységes (uniform)

Ebben az esetben az azonos fájlok azonos névvel érhetők el a különböző hosztokon.

- Nem egységes (nem uniform)

Ebben az esetben az azonos fájlok különböző névvel érhetők el a különböző hosztokon.

4. Védelem és biztonság

Az erőforrás megosztás és a biztonság kérdései

A SETI program valóban „magasztos” céloktól vezérelt elképzelése volt az erőforrás megosztás önkéntes módja. De, ez lett az alapja a ma egyik jelentős támadási módszerének, az erőforrás lopásnak. (trójai → majd az előadás vége felé jön...)

SW illetve HW védelem kapcsán egy kis kitekintés klasszikus irányba:

Decimus Iunius Iuvenalis (Aquinum, kb. i.u. 50 – ?, kb. i.u. 130) Számos szókapcsolata máig szállóigeként él, mint például a „Quis custodiet ipsos custodes?” - „Ki őrzi az őrzőket?” vagy a „Mens sana in corpore sano” - „Ép testben ép lélek”, illetve a „Panem et circenses” - „Kenyeret és cirkuszt”.

A „Ki őrzi az őrzőket?” más megközelítésben: Például egy szultán szempontjából az, hogy a háremhölgyek vigyáztak egymásra az egy szoftveres védelem, de a háremhölgyet őrző eunuch már hardveres védelmet jelent ☺.

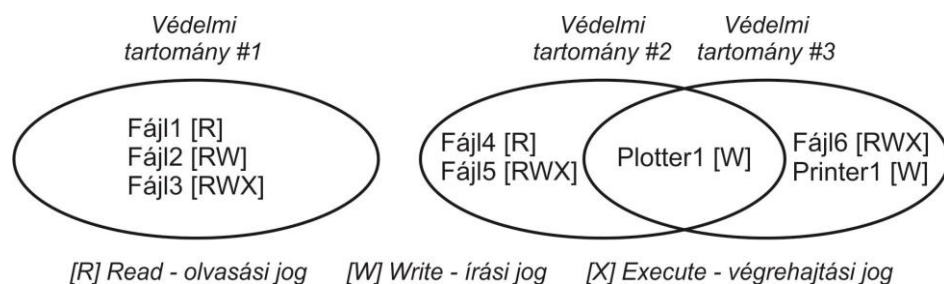
A védelem nem jelent mást, mint az erőforrások elérésének egy előre megállapított szabályrendszer szerinti elérési módját. A biztonság, mint fogalom, a rendszerben tárolt adatok sérthetlenségét fejezi ki. Minkét esetben szintekről beszélünk.

Védelem illetve a biztonság mindig több tényező:

- szoftveres tényező
- hardveres tényező
- emberi tényező

Ha van védelem, nyilván kell lennie fenyegetésnek is, mely lehet belső, ha a rendszerben futó (esetleg kéretlen) folyamatokból fakad, illetve lehet külső, ha rendszertől független tényezők a forrásai.

Bármely védelem alapja tehát egy szabályrendszer, a jogosultságok hierarchikus rendszere. Ideális esetben minden folyamatnak illetve erőforrásnak csak az éppen szükséges jogosultságokkal kell rendelkeznie. Minden elemnek egyéni azonosítója van, és előre definiált műveletekkel érhető el. A valóságban az elosztott rendszereken belül védelmi tartományokat (Domain) használunk a szabályzásra, statikus vagy dinamikus elérési mátrix-szal. A statikus mátrix a folyamat élelciklusa alatt nem változik, a dinamikus képes bővülni, szűkülni. Védelmi tartomány megadja, hogy az adott folyamat az adott állapotában mely erőforrásokhoz férhet hozzá és azokon milyen műveleteket végezhet.



Dinamikus védelmi tartományos esetén szabályozni kell a tartomány váltás vagy megváltoztatás módját.

- Védelmi tartomány váltás (Switch)
Azt kell definiálni, hogy mely tartományokból melyik másik tartományba lehet átlépni azaz védelmi tartományt váltani.
- Elérési jogosítványok másolása (Copy, Transfer, Limited Copy)
Ebben az esetben egy folyamatnak lehetőséget adunk arra, hogy adott védelmi tartományban lévő védelmi jogosítványát egy másik védelmi tartományba átmásolhassa. Ez inentől további lehetőségeket is felvet: a jogot átadjuk (Transfer) vagy átmásoljuk (Copy), illetve, hogy a másolt jog továbbmásolható lesz-e.

- Objektumok tulajdonlása (Owner)

Az erőforrás tulajdonosa más tartományokban is adhat vagy törölhet az erőforráshoz tartozó védelmi jogokat, műveleteket. A mátrixban az oszlopok mentén történő módosítások tartoznak ide.

- Vezérlő (Control)

A mátrixban a sorok mentén történő módosítások tartoznak ide. Ha egy tartomány ilyen módon egy másik tartományt is vezérel, akkor annak sorában adhat vagy törölhet jogosultságokat.

Az elérési mátrix

A mátrix sorait tartományok, oszlopait statikus védelmi tartományok esetén (első táblázat) csak objektumok, illetve dinamikus védelmi tartományok esetén (második táblázat) tartományok is alkotják, az egyes cellákban pedig a kapcsolódó jogok helyezkednek el.

Tartományok	Objektumok							
	Fájl1	Fájl2	Fájl3	Fájl4	Fájl5	Fájl6	Printer1	Plotter1
1	olvasás	írás, olvasás	írás, olvasás, végrehajtás					
2				írás	írás, olvasás, végrehajtás			írás (nyomtatás)
3						írás, olvasás, végrehajtás	írás (nyomtatás)	írás (nyomtatás)

Tartományok		Objektumok								Tartományok		
		Fájl1	Fájl2	Fájl3	Fájl4	Fájl5	Fájl6	Printer1	Plotter1	1	2	3
	1	olvasás	írás, olvasás	írás, olvasás, végrehajtás							váltás	
	2				írás	írás, olvasás, végrehajtás			írás (nyomtatás)	váltás		váltás
	3					írás, olvasás, végrehajtás	írás (nyomtatás)	írás (nyomtatás)	írás (nyomtatás)	váltás		

Jellemző a nagyszámú védelmi tartomány és a nagyszámú objektum, melyekhez relatíve kisszámú jogosultság tartozik. Ezek alapján a mátrix mérete szélsőségesen nagy is lehet, miközben kitöltése jellemzően ritka, ezért célszerű a várható méretéhez és a várható tartalmához optimalizálni a mátrix tárolási módját.

- Globális tábla

A legegyszerűbb tárolási mód, ezért mérete nagy, feldolgozása hosszú. Egy listába gyűjtve tartalmazza a következő hármas adatcsoportokat: „tartomány; objektum; művelet végzési jog”.

- Objektum elérési lista

A mátrixot az oszlopai mentén fel kell darabolni, majd objektumonként a következő kettes adatcsoportot kell eltárolni: „tartomány; művelet végzési jog”. Ez a globális táblánál gyorsabb feldolgozást tesz lehetővé.

- Tartományok jogosítványainak listája

A mátrixot a sorai mentén fel kell darabolni, majd tartományonként a következő kettes adatcsoportot kell eltárolni: „objektum; művelet végzési jog”. Ez a globális táblánál gyorsabb feldolgozást tesz lehetővé, különösen a tartományok szerinti összehasonlításban.

- Zár-kulcs módszer

Az előző két módszer elvi ötvözete. Minden objektumhoz egyéni bitminták tartoznak, ezekre mint „zár” hivatkozunk. A védelmi tartományokhoz pedig egy vagy akár több bitminta is tartozhat, melyekre mint „kulcs” hivatkozunk. Amennyiben egy „kulcs” illik az adott „zár”-ba, akkor az adott védelmi tartományban lévő folyamat képes végrehajtani a vonatkozó műveletet.

Védelem a kártékony programok ellen

A kártékony programok egy része észrevétlenül jut a rendszerbe („A” eset), másik részét a felhasználó maga engedi be („B” eset).

A védelemnek átfogónak kell lennie. Ez egy egyszerű példán keresztül könnyen megérthető. Vegyünk egy embert, akinek kancsal a szeme, fáj a foga és el van törve a lába. Mire van ennek az embernek szüksége? Szemüvegre, fogfájás csillapítóra, és arra, hogy begipszeljék a lábát, de nyilván a szemüvegtől nem múlik el a fogfájása... Egyik eszköz/folyamat sem helyettesíti a másikat.

Ennek analógiájára, a tűzfal képes portot blokkolni, de a vírust nem ismeri fel. Az adathalászat elleni program megvédi pl. a bankkártya számunkat, de nem véd meg a címhamisítás ellen, stb.

Ezért a védelemnek mindenképpen több lábon kell állnia.

A kártevőket a tevékenységük, a szaporodásuk illetve terjedésük módja szerint is csoportosíthatjuk.

„A” eset

- Vírus

Ez a klasszikus támadás egy számítógép ellen. Az első vírus címért hivatalos források szerint dúl a harc. 1982-ben, a 15 éves iskolás Rich Skrenta egy számítógépes partin (ez még LAN nélküli party volt) szeretne volna megtréfálni barátait. A gépeken Apple II operációs rendszer futott, és az

Elk Cloner nevet kapott vírus floppy lemezen terjedt. Működése alapján ezt a vírust tekinthetjük a boot vírus elődjének. Amikor a számítógép a fertőzött floppy lemezről indult, a vírus bekerült a memóriába, s amikor egy fertőzés mentes lemez került a meghajtóba, akkor oda kimásolódott, így terjedt gépről-gépre. (már akkor is jelen volt a programok illegális másolása, ezért tudott terjedni) A megfertőzött gép minden 50-dik indításnál egy poénos szöveget írt a képernyőre. Az első igazán komoly vírust (állítólag) palesztin informatikusok írták egy izraeli hadi gyár megzavarására. De kapott vírus már iráni uránium centrifuga is...

A vírusok általában file (lehet akár adat file, vagy futtatható állomány, stb.) vagy boot szektor szinten fertőznek, de előfordul, hogy olyan HDD helyre íródnak be, amihez a felhasználó nem is fér hozzá. A vírus egyik legfontosabb tulajdonsága a szaporodás. A vírus aktiválódhat azonnal, de aktivizálódása akár dátumhoz, vagy bekapcsolások számához, stb. is köthető.

- Féreg (Worm)

A féreg a vírus közvetlen leszármazottja. Szinte minden alapvető tulajdonságban egyezik a vírussal, ám van egy nagy különbség, ami a férgeket rendkívül sikeressé teszi: a férgek – a vírusokkal ellentétben – képesek „önálló” akciókra, a felhasználó közreműködése nélkül is tudnak terjedni. Nem fertőznek meg a gépen állományokat, a céljuk a gyors terjedés, ezért különösen szeretik a levelezőprogramokat, és benne a személyes levelezőlistákat. (mostanában a közösségi oldalakat is kezdik felfedezni maguknak) Jellemző példa, hogy amikor egy féreg megérkezik – általában egy e-mail csatolmányba rejtve – és mi azt megnyitjuk, azaz aktiváljuk a férget, a féreg automatikusan továbbküldi magát a listánkban található összes címzettnek.

- Adathalászat (Phishing)

Személyes adatok, bankkártya információk ellen irányul. Jellemzően web-es alkalmazások, vagy weboldalak hamisításával csapják be a felhasználót.

„B” eset

- Trójai és Erőforrás halászat

Trójai faló → Homérosz, Iliász, Odüsszeusz, Trója, Laokoon ismert története alapján. Az óvatlan felhasználó maga juttatja a gépébe a kártékony programot, leggyakrabban nyilvános vagy demo programokhoz kapcsolódóan. Létezik olyan változat is, ami önálló programnak látszik, de funkcióit a felhasználó felé csak nagyon kis részben (főleg csak az álcázás

miatt teljesíti). Pl. aktuális, hogy az NSA-nak létezik olyan megfigyelő programja, ami Firefox böngészőnek látszik, és szinte ugyanúgy is viselkedik! De létezik szinte minden népszerű programnak az álcázott verziója...

Az erőforrás halászat a trójai programok speciális működése. A már tárgyalt erőforrás megosztás kényszerű és agresszív esete. A felhasználó annyit érzékel, hogy lassabb lett a gépe, az okot nem ismeri. Valójában a gépe erőforrásain már nem egyedül rendelkezik, sőt, neki csak a maradék jut.

- **Megtévesztő programok (Misleading Application)**

Igazából nem létező hibára, fertőzésre hívja fel a felhasználó figyelmét, segítő szándéknak tűnő módon. Sőt, megoldást is kínál, miszerint az általa ajánlott alkalmazás „megtisztítja” a gépet...

- **Címhamisítás (Address Spoofing)**

Ebben az esetben például olyan email érkezik hozzánk melynek a feladója látszólag egy ismerősünk (aki a címlistánkban is szerepel) vagy egy ismert közintézmény, de valójában az igazi feladó, aki nem tiszta szándékkal küldte a levelet, amiben esetleg segítséget, pénzt, egy adott oldalra való belépést stb. kér, vagy belépési kódot ajánl fel, ismeretlen.

- **Pszichológiai manipuláció (Social Engineering)**

Összefoglaló néven így hivatkozunk a felhasználó bizalma ellen intézett támadásokra. A következő (nem is feltétlenül csak számítógép segítségével végrehajtott) tevékenységeket soroljuk ide:

- Zsarolóvírus (Ransomware)
- Üzenetküldésen keresztüli adathalászat (Messaging, Smishing)
- Személyre szabott csaló támadás (Personalized Scams)
- Vállalatvezető nevében elkövetett csalás (CEO Fraud)
- Telefonhívásos támadás, csalás (Phone Call Attack, Scams)
- Közösségi oldalon keresztül elkövetett csalás (Scamming You Through Social Media)
- Információ elcsúszása (Shoulder Surfing)
- Kattintásvadász oldalak (Clickbait)

Előadás_#10.

1. A Windows NT kialakulása

A Windows NT (New Technology) a Microsoft cég új generációs, eredetileg vállalati felhasználókat megcélzó operációs rendszerének az elnevezése. A Windows NT operációs rendszerrel a Microsoft a DOS-, illetve Windows rendszereket kívánta felváltani. Szoftver fejlesztés azonban nem csak a Microsoft-nál történt, az IBM már 1987. áprilisában bemutatta az OS/2 nevű operációs rendszerének első verzióját. Igazi siker azonban csak az 1992-ben bemutatott valódi 32-bites OS/2 2.0 lett (mely képes volt Windows 3.1-es programok futtatására is), majd az 1994-ben bemutatott OS/2 3.0 Warp lett, mely hatékony és konzisztens rendszer volt. Egy többé kevésbé kényszerű együttműködés után a Microsoft 1990-ben szakított az IBM-mel és 1993. júliusában bemutatta saját 16-bites Windows NT 3.1 nevű operációs rendszerét. Ezek után a Microsoft fejlesztései is a 32-bites irányba indultak el, 1996. júliusában került bemutatásra a Windows NT 4.0. Az első említésre méltó 64-bites rendszer a Windows XP 64-bit volt, 2003 márciusában. A Microsoft asztali- és szerver operációs rendszerei ma is az NT technológiára épülnek.

Az eddig megjelent NT alapú Windows verziók:

Verzió	Build	Megnevezés	Megjelenés
3.1	528	Windows NT 3.1	1993.07.27
3.5	807	Windows NT 3.5	1994.07.21
3.51	1057	Windows NT 3.51	1995.05.30
4.0	1381	Windows NT 4.0, 32-bit	1996.07.29
5.0	2195	Windows 2000	2000.02.17
		Windows 2000 Datacenter	2000.09.26
5.1	2600	Windows XP	2001.10.25
5.2	3790	Windows XP, 64-bit	2003.03.28
		Windows Server 2003	2003.04.24
		Windows Server 2003 R2	2005.12.06
6.0	600x	Windows Vista	2006.11.30
		Windows Server 2008	2008.02.27
6.1	760x	Windows 7	2009.10.22
		Windows Server 2008 R2	2009.11.22
		Windows Home Server 2011	2011.04.06
6.2	9200	Windows 8	2012.10.26
		Windows Server 2012	2012.09.04
6.3	9600	Windows 8.1	2013.10.18
		Windows Server 2012 R2	2013.10.18
10.0	10240	Windows 10	2015.07.29
1607	14393	Windows Server 2016	2016.10.12
1809	17763	Windows Server 2019	2018.11.13
11.0	22471	Windows 11	2021.10.05

A Windows NT megalkotásakor a Microsoft két alapvető szempontot kellett hogy figyelembe vegyen. Egyrészt az aktuális piaci igényeket, követelményeket ki kell elégíteni (ez lássuk be nem minden verziónál ment zökkenőmentesen), másrészt a továbbfejlesztési lehetőségek, és a saját (a piacot irányító, befolyásoló) célkitűzések is meg kell hogy jelenjenek a termékben.

2. Az első Windows NT kiadásokkal szemben támasztott követelmények

Egy valós 32-bites (csak az Windows NT 4.0 óta), bármikor megszakítható (Preemptív) és újrarahívható (Reentrant) [a rendszerhívásokat több alkalmazás is meghívhatja egyszerre, és ezek nem blokkolódnak, akkor sem, ha már valakit éppen kiszolgál az adott rendszerhívás], valamint virtuális memóriakezelésen alapuló operációs rendszer megvalósítása volt az alapvető cél. A legfontosabb követelmények a következők voltak:

- Fusson különböző hardver architektúrákon.
- Fusson multiprocesszoros környezetben.
- Legyen skálázható az összes rendelkezésre álló erőforrás viszonylatában.
- Vegye figyelembe a DCE (Distributed Computing Environment) ajánlásait, azaz legyen képes elosztott hardver környezetben is futni.

[A DCE egy, a 90-es évek elején tett kísérletet a különböző hardver architektúrák szoftveres kompatibilitásának irányába.]

- Legyen alkalmas a 16-bites MS-DOS és a Windows 3.1-es alkalmazások futtatására. (Ez a követelmény már a korai 32-bites időktől is csak kompromisszumok árán, és csak részben teljesült, a 64-bites verziók alatt pedig már közvetlenül nem is oldható meg.)
- Teljesítse a POSIX 1003.1 szabványt.

[Portable Operating System for UniX / formális neve: IEEE1003, hivatalos elnevezése ISO/IEC 9945).

- Teljesítse az ipari környezetnek megfelelő biztonsági szabványokat.
- Használjon UNICODE-ot a karakterek és stringek ábrázolására.

[A UNICODE a karakterek gépi ábrázolásának szabványa. Mivel minden karaktert 16-biten ábrázol, így szinte minden nyelv ábécéjének teljes karakterkészletét lehetséges azonos kódolással használni. A UNICODE így lehetővé teszi az alkalmazások nyelvterülettől független elkészítését. Az első olyan, Microsoft által gyártott operációs rendszer, amely binárisan is egységes az egész világon, a Windows 2000 volt.]

Az UNICODE előnyeit kihasználva, az NT 4.0 a belső karakterábrázolásánál a Microsoft az UNICODE használta mellett döntött. Tekintettel arra, hogy az

UNICODE használata még egyáltalán nem volt általános, így a Windows NT 4.0 alatt futó alkalmazások jelentős része még nem használt UNICODE-ot. Ez az oka annak, hogy a Win32 API-ban (Application Programming Interface / Alkalmazásprogramozási interfész), mely például a karakterláncokat (String változókat) mint paramétereket közvetíti a folyamatok között, kétféle függvényt (Routine) kellett definiálni:

- széles: 16-bites UNICODE karakteres
- keskeny: 8-bites ANSI karakteres

A 8-bites ANSI függvények megvalósítása két egyszerű lépésben történik. Első lépés a String paramétereket átkódolása 16-bites UNICODE-ba, a második lépés pedig a függvény UNICODE-os megfelelőjének meghívása.

3. A Microsoft célkitűzései

A Microsoft az Windows NT alapú operációs rendszerek mindenkori minőségének, korszerűségének és továbbfejleszthetőségének biztosítása érdekében jelölték meg a következő célokat:

- A Windows NT kódja legyen könnyen továbbfejleszthető, vagyis kiterjeszthető. (Ez a kitétel nyilván csak a kód egyes részeire vonatkozik, azaz a kód csak részben nyílt, maga a teljes programkód nem publikus.)
- A kód legyen alkalmas az új hardver platformokon történő futtatásra is.
- A rendszer legyen több szempontból is megbízható, robusztus.
 - két program futása ne befolyásolja egymást
 - a rendszer összeomlást el kell kerülni
 - a belső komponensek együttműködése zökkenőmentes legyen
- Megfelelő kompatibilitás a meglévő rendszerekkel a felhasználói interfész, és alkalmazásprogramozási interfész (API) szintjén is. A kompatibilitás két alapvető szintje:
 - a Microsoft korábbi operációs rendszerei: MS-DOS, Windows 3.1
 - a nem Microsoft által készített, azonban széles körben elterjedt rendszerek: OS/2, UNIX, NetWare, stb.

Mára ez a szempont gyakorlatilag is okafogyottá vált.

- A rendszer a hatékonysága legyen független a hardverkörnyezettől, vagyis bármelyik hardver platformon optimálisan tudja kihasználni a rendelkezésre álló erőforrásokat.

4. A Windows NT alapú rendszerek felépítésének fő jellemzői

A Windows NT alapú operációs rendszerek felépítése szigorúan réteges szerkezetű, az alrendszerének működése pedig kliens-szerver kiszolgálás alapú. Az egyes rétegek interfészekon igénybevételel kommunikálnak. Az eredeti elképzelés szerint a Windows NT fejlesztői a mikrokernél használata mellett döntöttek, de ahhoz, hogy a legtöbb szolgáltató folyamat felhasználói (User) módban futhasson, a hibrid kernél megfellelőbbnek bizonyult. A szolgáltatások egy részét nem célszerű és nem is hatékony (főleg a gyakori és nagyszámú környezetváltás miatt) felhasználói módba kényszeríteni. Azokat a szolgáltatásokat, melyek intenzíven használják a hardvert és futásuk gyorsasága az egész rendszer teljesítménye szempontjából kritikus, célszerű mindenképpen kernél módban futtatni. (Ilyen szolgáltatás például a cache menedzsment, a címtartomány menedzsment, a szálütemezés, a folyamatok közti kommunikáció, a virtuális memóriakezelés, az objektumkezelő, a biztonsági alrendszer, illetve a WIN32-es alrendszer grafikus támogatása.)

A Windows NT alapú rendszerek fejlesztésekor alapvető volt az objektumorientált szemlélet. A Microsoft az egész operációs rendszert funkciójukban pontosan definiált objektumokból építette fel. Az objektumok közti kommunikáció szintén előre meghatározott interfészek igénybevételel történik.

A Windows NT korai verzióiban, minden szándék és elhatározás ellenére, – a hardverközeli programozás miatt – csak az alábbi három objektumorientált tulajdonságot sikerült maradéktalanul megvalósítani:

- Adatrejtés
Az operációs rendszer objektumai csak saját adataikat érhetik el.
- Interfész-használat
Az objektumok előre definiált interfészek segítségével kommunikálnak.
- Hierarchikus objektum szerkezet
Kernél objektumok és executive objektumok használata.

A következő objektumorientált tulajdonságokat viszont a Windows NT alapú rendszerek nem valósítják meg:

- Polimorfizmus

Azonos néven érhetünk el különböző objektumokat, mert ez esetben az azonosító környezetétől függ, hogy melyik objektumra hivatkozunk az adott azonosítóval. Szemléletes példa a polimorfizmusra például az, amikor egy objektumon belül azonos névvel, de különböző paraméterlistával definiálunk függvényeket. Ez esetben az, hogy melyik funkciót érjük el csak az aktuálisan használt paraméterek típusától függ.

- Öröklődés

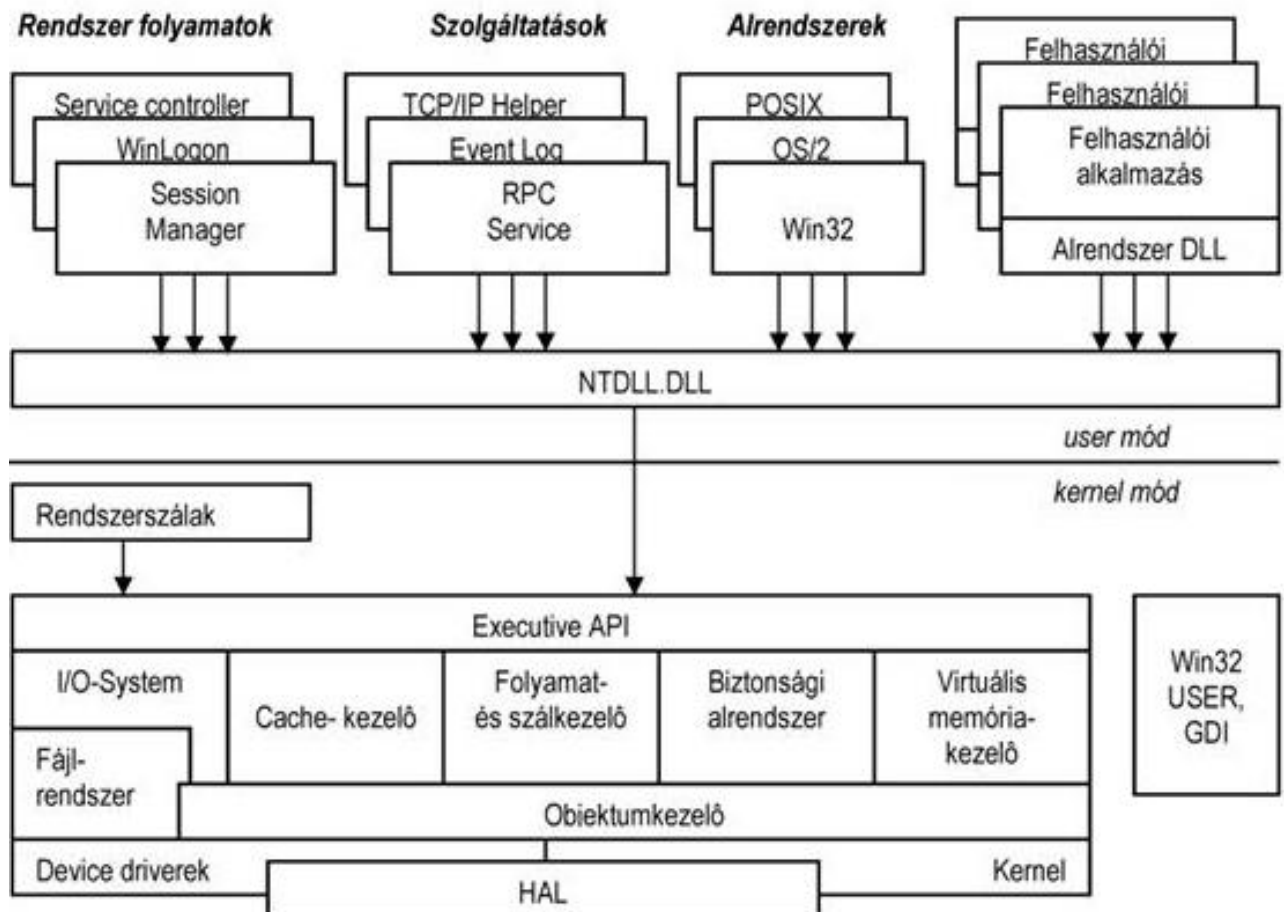
Az objektumok hierarchikusan (is) származtathatók. Egy szülő objektum megfelelően definiált adatait, függvényeit, attribútumait részben vagy egészben a származtatott objektum képes örökölni, így a későbbiekben maga is használhatja (vagy akár tovább is örökítheti) azokat.

Fontos megjegyezni, hogy az NTFS használata óta az öröklődés már részben megvalósul!

- Dinamikus adattípus kötés

Lehetőség van adattípusokkal paraméterezett objektumok definiálása is, de maga a dinamikus adattípus hozzárendelés gyakorlatilag mindig futásidőben történik, hiszen az egyes változókhoz csak ekkor rendelődik hozzá az aktuális adattípus.

5. A 32-bites Windows NT felépítésének komponensei



HAL

A HAL (Hardware Abstraction Layer) a számítógép hardverét teljesen elfedő, az operációs rendszer felépítését tekintve a legalacsonyabb szoftverréteg. A Számítógép-Architektúrák kurzuson megismertek szerint létezik ennél is hardverközelebbi szoftveres réteg – a BIOS illetve az UEFI – hiszen ezek technikailag az operációs rendszer és a hardver között helyezkedik el, viszont ezek nyilván nem részei az operációs rendszernek. Az operációs rendszer HAL fölött elhelyezkedő rétegei tehát csak a HAL-on keresztül érhetik el a hardvert. Minden Windows NT alapú operációs rendszer alatt használható CPU-hoz egyedi HAL-réteg készül, vagyis a HAL a használt CPU típusától függ. A telepítő állományok között így számos HAL-t találhatunk az \i386-os alkönyvtárban. Érdekesség, hogy a Vista előtti rendszerekben telepítéskor választódott ki, hogy milyen típusú HAL kerül felmásolásra. Többek között ezért sem volt biztos, hogy egy másik gépbe átrakva a HDD-t egyáltalán képes lesz-e a Windows elindulni. A Vista óta szerencsére ezzel a problémával már nem találkozunk.

A HAL az azonos architektúrájú CPU-knak az egyedi CPU modelltől függő funkcióit tartalmazza. A HAL feladata, hogy CPU független szolgáltatásokat nyújtson a többi réteg számára. Azonos CPU architektúra esetén – vagyis például az összes x86-os

processzoron futó Windows NT alapú rendszer között – csak a használatban lévő HAL-ban van különbség, a többi rétegben nincs (legalábbis nem szükségszerű). Nyilván ugyanez a helyzet x64 vagy RISC esetében is. Feladatát tekintve tehát a HAL egy olyan – az aktuális hardverre támaszkodó – virtuális gépként dolgozik, melynek funkcionalitása minden rendszerben azonos.

A perifériák, azaz csatlakoztatott külső hardverek esetében egy kicsit más a helyzet. Ezekhez nyilván egyedi illesztőprogramok tartoznak, melyek függhetnek az operációs rendszer konkrét típusától, illetve a CPU architektúrájától (x86 vagy x64) is. (Erről bővebben a device driverek réteg alatt lesz szó.)

(Az 1968-ban készült "2001 Űrodisszeia" című filmben szereplő HAL9000-es computer nevében a HAL természetesen mást jelentett, egy rövidítés volt, ez: Heuristically programmed ALgorithmic computer.)

Kernel

A kernel az operációs rendszer alapfunkcióit nyújtó komponense (pl. ütemezés, megszakításkezelés). A kernel a rendszernek az állandóan a memóriában (operatív tárban), azon belül is védett módban (azaz kernel módban) futó része.

A kernelben is előfordulnak hardver specifikus kódrészletek, hisz például a környezetváltás megvalósításához ismerni kell, hogy milyen regiszterei vannak a CPU-nak. Azonban amíg a HAL a CPU típusától függ, addig a kernel csak a CPU architektúrájától. Ez azt jelenti, hogy azonos architektúrájú CPU-k esetén a kernel is azonos. Például két x86-os rendszerben a kernelréteg is azonos, függetlenül attól, hogy konkrétan milyen CPU-t használunk. Nyilván ugyanez a helyzet x64 architektúra esetében is.

A kernelre (illetve a device driverek rétegre) épülő további komponensek már hardverfüggetlenek, azaz a rendszer további komponensei már gyakorlatilag hordozhatóak. A Windows NT alapú rendszerek hordozhatósága ebből következően úgy valósul meg, hogy a hardvert bármely CPU esetében hasonló interfészen keresztül elérhető szoftverrétegek fedik el. Az első két szoftverréteg tehát a HAL és a kernel. A kernelben realizálódnak a CPU architektúrától függő funkciói (például a környezetváltás és a szálkezelés).

Más szempontból is (legalább részben) hordozhatóak a Windows NT alapú rendszerek, hiszen – a UNIX rendszerekhez hasonlóan – hordozható programnyelven íródtak. A programkód legtöbb része C programnyelven készült, de a hardver közvetlen kezelését végző (például a megszakítás kezelés), illetve a rendszer teljesítményét nagymértékben befolyásoló (például a környezetváltás) kódrészek már Assembly nyelven készültek.

A rendszer további rétegeinek a hardvertől történő függetlenítésén túl a kernelnek az is a feladata, hogy az operációs rendszer többi komponense által használható, jól definiált alapmodulok – az úgynevezett primitívek – végrehajtását lehetővé tegye.

A primitívek használatával megvalósítandó funkciók:

- szálütemezés (Thread Scheduling)
- trap-kezelés, megszakítás- és kivételkezelés
- multiprocesszor ütemezés
- a kernel objektumok kezelése

A kernel objektumok felépítésüket tekintve egyszerűbbek, mint a többi réteg objektumai. A gyors kezelhetőség érdekében ugyanis a kernel az egyes objektumok elérésekor nem végez ellenőrzést, hanem abból a logikából indul ki, hogy a rendszer többi komponense megfelelően használják az objektumokat.

Készülék meghajtók (Device Driverek)

A device driverek a I/O-alrendszer és a számítógép hardvere közötti kapcsolatot biztosítják, oly módon, hogy a hardvert nem közvetlenül, hanem a HAL rétegen keresztül éri el. A device driverek akár forráskódjukban is hordozhatók a különböző hardver architektúrák között, illetve akár binárisan is hordozhatók az azonos architektúrájú, de különböző típusú CPU-k esetén.

Ezen réteg vizsgálatakor nem szabad megfeledkezni arról, hogy az igények érkezhetnek a felsőbb rétegekből kiindulva – nyilván a felhasználó is generál itt realizálandó igényeket – illetve a másik irányból, azaz hardver oldaláról is. Ez a kétirányúság is az egyik oka a device driverek réteg és az I/O réteg szoros kapcsolatának.

A device driverek négy típusa a következő:

- Hardver driverek

A hardver driverek az egyes hardver komponensek I/O csatornáit közvetlenül elérve teszik lehetővé a hardver használatát. (pl. BUS driverek: PCI, USB, FireWire, SATA)

- Fájlrendszer driverek

A fájlrendszer driverek a fájlrendszer(ek) elérésére vonatkozó igények kiszolgálását végzik, I/O műveletekké alakítva át a kéréseket.

- Szűrő típusú device driverek

Az NT alapú rendszerek által is használt réteg szerkezetű device driver struktúra lehetőséget kínál egyedi, speciális többletfunkciók megvalósítására. Ehhez arra van szükség, hogy a kívánt szoftveresen megvalósítandó funkciót megfelelő helyre, azaz a két érintett driver illetve réteg közé implementáljuk. Jellemző, hogy a szűrő típusú device driverek egy magasabb szintű rétegben lévő driver – például a fájlrendszer rétegé – és egy alacsonyabb szintű driver – a device driver rétegben található disc driver – közé ékelődnek. Ezen a módon valósulnak meg a szoftveres RAID funkciók, a kibővített illetve hibatűrő szoftveres lemezkezelés.

- Hálózati device driverek

A hálózati device driverek feladata a számítógép hálózatokkal kapcsolatos igények megvalósítása, továbbítása. Ehhez minden esetben kérések megfelelő konverziójára van szükség.

Executive (az utolsó komponens a kernel módban)

Az executive a Windows NT alapú operációs rendszerek magas szintű alrendszerének szolgáltatásait (memóriakezelés, biztonság) megvalósító réteg. Az adatokat objektumokban tárolja, melyeket a megfelelően definiált interfészekon keresztül, és csak leírókkal (Handle) lehet elérni.

Az executive réteg a következő komponenseket tartalmazza:

- Folyamat- és szálkezelő
- Virtuálistmemória-kezelő
- Biztonsági alrendszer (Monitor)
- Cache kezelő
- I/O-rendszer kezelő

Itt található Plug-And-Play kezelő is – ami az ábrán nem is szerepel, mivel csak a Windows 2000 óta része a rendszernek.

Az executive réteg kezeli le és szolgálja ki az (egyébként user módban futó) NTDLL.DLL rétegben definiált függvények igényeit az alsóbb rétegek felé, így biztosítva az operációs rendszer belső objektumai közötti kommunikációt.

Az executive réteg alapvető feladata az LPC (Local Procedure Call / Lokális eljárás hívás) szolgáltatás megvalósítása. Az LPC a Windows NT alapú operációs rendszerek IPC (Inter Process Communication / Folyamatok közötti kommunikáció) eszköze, azaz csak a Windows NT alapú rendszerek belső folyamatai közötti kommunikáció biztosítására szolgál. Az LPC igénybevételel válik egy felhasználói

objektum képessé arra, hogy egy másik felhasználói objektum adott függvényét meghívja. Azaz az LPC teszi lehetővé azt, hogy az egyes alkalmazások a hozzájuk tartozó alrendszer(ek) szolgáltatásait igénybe vehessék. Az executive réteg a rendszerfolyamatok és a szolgáltatások részére biztosít még további Run-time library függvényeket, valamint különböző támogató funkciókat megvalósító függvényeket is.

NTDLL.DLL (itt kezdődnek a user módú komponensek)

Az NTDLL.DLL egy speciális, dinamikusan kapcsolódó (kölsön)könyvtár (Dinamically Linked Library / Dinamikusan kapcsolódó [kölsön]könyvtár. A kölsönkönyvtár fogalmi magyarázata egy korábbi fejezetben már szerepelt.) A Windows NT alapú operációs rendszerekben az NTDLL.DLL használata megkerülhetetlen a user mód és a kernel mód közötti kommunikáció lebonyolításához. Az executive rétegben már tárgyalt objektumok közötti kapcsolattartás, az LPC (Local Procedure Call / Lokális eljárás hívás) az, amely a megfelelő függvényhívások segítségével teszi lehetővé ezt a kommunikációt is. A felhasználói objektumok csak az NTDLL.DLL igénybe vételével érhetik el az operációs rendszer kernel módban működő részét, illetve a hardvert.

Magát a kommunikációt az NTDLL.DLL néhány egyszerű lépésben hajtja végre.

1. lépés Az NTDLL.DLL-hez függvényhívás érkezik valamelyik felsőbb rétegből.
2. lépés Az NTDLL.DLL ellenőrzi a függvényhívás hívás paramétereit.
(Normál esetben nincs szükség hibakezelésre.)
3. lépés Végre kell hajtani egy user mód – kernel mód váltást.
4. lépés Az NTDLL.DLL átadja a rendszerhívást a system service dispatcher-nek.
5. lépés Az operációs rendszer így már képes meghívni a kért funkciót realizáló kernel módú függvényét.

NTDLL.DLL tartalmilag az executive által kiajánlott függvényeknek megfelelő függvény csonkokból áll, melyek ugyanolyan a paraméterezésűek, mint az executive-ban lévő párjuk. Ezen kívül tartalmaz számos további függvény az alrendszerek támogatására. Ezek közül a két legfontosabb:

- dinamikus memória kezelés (Heap)
- Image Loader

Az NTDLL.DLL-nek a fent leírtakon kívül létezik számos, a felhasználók számára nem dokumentált, illetve nem publikált függvénye is:

[Nem dokumentált NTDLL.DLL függvények \(Angol nyelvű\)](#)

Rendszerfolyamatok

A Windows NT alapú operációs rendszerek a független felhasználói folyamatként megvalósuló rendszerfunkciókat nevezik rendszerfolyamatnak. Nyilván ez esetben user módban futó folyamatokról van szó. Ennek ellenére a rendszerfolyamatok olyan alapvető részei az operációs rendszernek, hogy ezek nélkül a Windows NT alapú operációs rendszerek nem is lennének képesek működni.

A legfontosabb rendszerfolyamatok:

- **SMSS (Session Manager SubSystem / Munkamenet kezelő alrendszer)**
Az smss.exe program indításával jön létre az SMSS folyamat. A továbbiakban ez a folyamat felelős az alkalmazások elindításáért. Az SMSS indítja el azokat az alrendszereket is, melyeknek az indított folyamat futásánál szükség van. Feladatai közé tartozik még az, hogy kapcsolatot teremtsen az általa indított folyamatok és a Debugger (hibakereső rutin) között, valamint az, hogy biztosítsa a környezeti változók definiálását és elérését.
- **LOGON**
A winlogon.exe futtatása indítja a felhasználók beléptetéséért és kiléptetéséért felelős LOGON folyamatot, melyet a SAS (Secure Attention Sequence / Biztonsággal felügyelt programszakasz) billentyűkombinációk (alapesetben az CTRL-ALT-DEL) aktivizálnak. A beléptetés során a felhasználói azonosítót (username) és a jelszót (password) a LOGON olyan módon autentikálja, hogy először átadja azokat ellenőrzésre egy másik önálló folyamat, az LSASS (Local Security Authentication Server / Helyi biztonsági jogosultság ellenőrző) részére, melyet az lsass.exe indít. Ha az autentikáció sikeres, akkor a LOGON elindítja a userinit.exe programot. A userinit.exe a beléptetendő felhasználó adatainak ismeretében már képes beállítani a felhasználóhoz definiált környezetet. A környezet beállítása után a userinit.exe elindítja a már beléptetett felhasználóhoz rendelt Shell keretprogramot (alapesetben ez az explorer.exe).
- **SERVICECONTROLLER**
A SERVICECONTROLLER egy kernelből indított folyamat, melyet az screg.exe aktivál, és amely a szolgáltatások automatikus indításáért és leállításáért felelős.

Egyszóval a rendszerfolyamatok felelősek azért, hogy miután elindult az operációs rendszer a felhasználók be tudjanak lépni, és el tudják kezdeni az erőforrások hatékony használatát.

Szolgáltatások

A szolgáltatásként hivatkozunk a Windows NT alapú operációs rendszerekben azokra a folyamatokra, melyek a kliens programok számára többletszolgáltatásokat nyújtanak. A feladatok összetettségéből következik, hogy jellemzően a szolgáltatásokat több folyamat együttesen szolgálja ki. A többletszolgáltatásokat igénylő kliens programok – melyek lehetnek akár felhasználói vagy akár rendszer programok – így szerver-kliens viszonyba kerülnek a szolgáltatásokkal, azaz az azokat megvalósító folyamatokkal.

A Windows NT felépítésének komponensei című ábrán szerepel az RPC (Remote Procedure Call / Távoli eljáráshívás) szolgáltatás, a különböző protokollokat megvalósító hálózati kapcsolatot biztosító szolgáltatás (TCP/IP Helper), illetve az operációs rendszer eseményeit naplózó (Event Logger / Eseménynapló) szolgáltatás. A Windows NT alapú operációs rendszerek természetesen számos további szolgáltatást is tartalmaznak.

Hasonlóan, mint a rendszerfolyamatok, a szolgáltatások is user módban futnak. A hasonlóság azonban itt véget is ér, hiszen a rendszerfolyamatok nélkül a Windows NT alapú operációs rendszerek nem képesek működni, addig a szolgáltatások (azaz az a szolgáltatásokat jelentő folyamatok) nélkül a rendszer – ha korlátozottan is – de működőképes marad. Ebből következik, hogy a szolgáltatásokat, a pillanatnyi igényeknek megfelelően a rendszer futása közben leállíthatjuk, illetve elindíthatjuk a Service Manager használatával. Lehetőség van az egyes szolgáltatások automatikus-, késleltetett- vagy kézi indítására, szüneteltetésére, vagy leállítására is. Az automatikus – azaz felhasználói beavatkozás nélküli – indítás azon programok esetében elengedhetetlen, melyeknek egy bejelentkezett felhasználó hiányában is futniuk kell. Ilyen például a távoli asztal (Terminal Services) vagy a fájlmegosztás (Server), melyek a számítógép távolból történő elérését teszik lehetővé.

A szolgáltatásokat a Service Controller (SERVICES.MSC) regisztrálja és felügyeli, ennek segítségével tudja a felhasználó a kiválasztott szolgáltatást paraméterezni.

Alrendszerek

A Windows NT alapú operációs rendszerek korai változatai a különböző típusú applikációk, programok futtatását a megfelelő alrendszerek segítségével biztosították. Az idők folyamán – üzletpolitikai-, marketing- és korszerűségi okok miatt – a valamikori három alrendszerből (Win32, POSIX, és az OS/2) mára csupán a Win32 maradt meg, így közvetlenül már csak Windows-os alkalmazások futtathatók. A Windows 2000 óta kikerült az OS/2 alrendszer, a Windows XP óta pedig a POSIX alrendszer is. A Win32 alrendszer nemcsak hogy megmaradt, hanem a rendszer elválaszthatatlan része, nélküle az operációs rendszer nem is képes futni. A Win32 alrendszer egyes részei kernel módban futnak, például a grafikus megjelenésért felelős (Full Desktop Multi-User Win32 Driver / WIN32K.SYS) is, a USER32.DLL, KERNEL32.DLL, ADVAPI.DLL könyvtárak támogatásával. A fő ok a rendszer hatékonysága, ugyanis ezzel a megoldással módváltás nélkül oldható meg az executive, illetve kernel réteg által biztosított szolgáltatások függvényeinek az elérése. A WIN32 API-ban számos további grafikus funkció is definiálva van. A GDI (Graphical Device Interface / Grafikus eszközcsatoló) szabvány szerinti grafikus eszközök és nyomtatók kezelését a GDI32.DLL (WIN32 API része) biztosítja. Az itt definiált funkciók is a WIN32 alrendszer kernel módú részében (WIN32K.SYS) valósulnak meg.

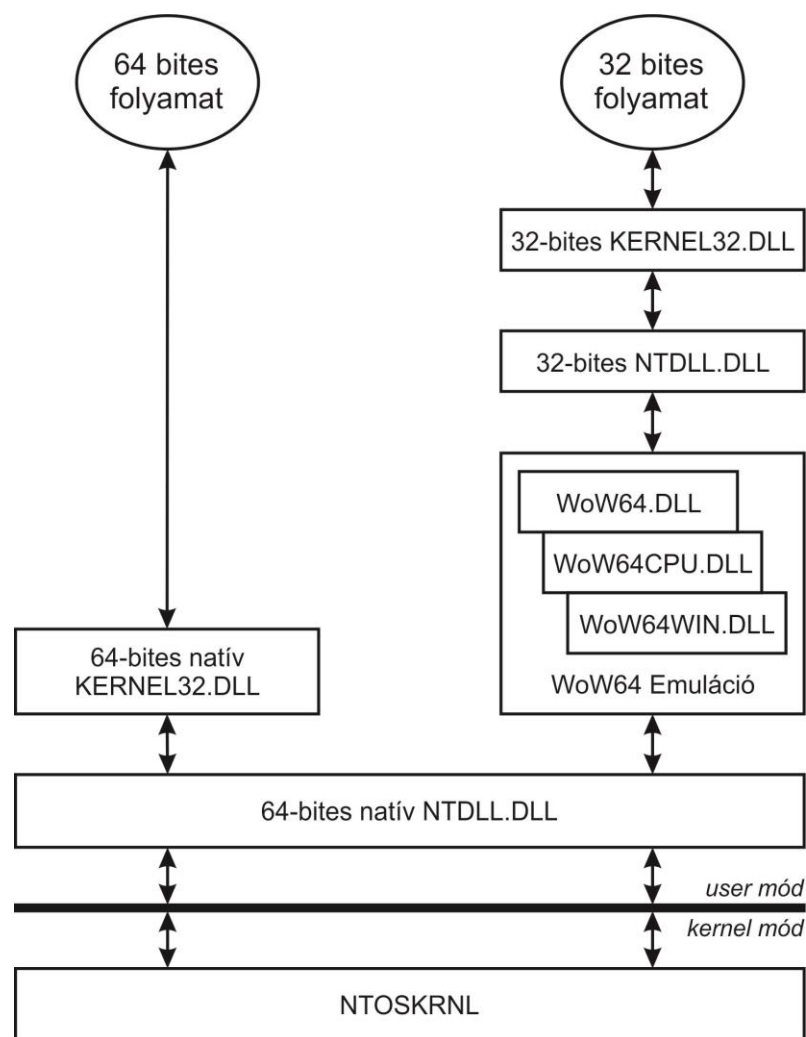
A Win32 API hívások háromfélék lehetnek a megvalósításuk helye szerint:

- Közvetlenül az alrendszer DLL-ben megvalósított egyszerű függvények hívása, melyek végrehajtásához a rendszer más részeinek elérésére nincs is szükség.
- Magában az alrendszerben megvalósított függvényhívások, melyek hívását az alrendszer DLL továbbítja az NTDLL.DLL felé, ami az executive réteg LPC szolgáltatását igénybe véve, eljut a Win32 alrendszerhez, ami a kérést teljesíti.
- Amennyiben a rendszer egy másik, kernel módban futó rétege valósítja meg a hívást, az akkor is az executive réteghez kerül, ami továbbítja a megfelelő kernel rétegnek.

6. 32-bit vs 64-bit

32-bites operációs rendszer esetén a Win32 alrendszer a 32-bites, a 16-bites és DOS alkalmazásokat is futtatja. 64-bites operációs rendszer esetében a visszafelé kompatibilitás a 16-bites, illetve a DOS alkalmazások esetében megszűnt, azaz ezek közvetlenül nem futtathatók. Megfelelő emulátor szoftver (pl. DosBox) segítségével természetesen orvosolható ez az állapot.

A 64-bites Windows NT alapú operációs rendszerek esetében a 64-bites programok futnak natív módon, azaz a 32-bites programok futtatásához van szükség egy további alrendszer, a WoW64 (Windows 32-bit on Windows 64-bit) igénybe vételére. A WoW64 alrendszer a funkcióit a következő ábra szerint, további DLL-ek igénybe vételével valósítja meg.



Előadás_#11.

1. A Windows NT alapú operációs rendszerek belső mechanizmusai

Amikor a Windows NT alapú operációs rendszerek belső mechanizmusai kerülnek terítékre, nyilván a kernel módú komponensek (Executive illetve Device Driver) által megvalósított legfontosabb, legalapvetőbb funkciókra kell gondolni:

- megszakítás- és kivételkezelés
- objektumkezelés
- szinkronizáció
- lokális eljárás hívás (Local Procedure Calls / LPCs)

Megszakítás- és kivételkezelés

A megszakítások (Interrupt) és a kivételek (Exception) olyan műveletsorozatok, melyek hatására az éppen feldolgozás alatt lévő folyamat futásának félbehagyása történik, mivel egy magasabb prioritású igény végrehajtása válik indokolttá.

A megszakítások és kivételek egyaránt lehetnek hardver vagy szoftver alapúak. Jellemző azonban, hogy a megszakítások esetében a hardver alapú megszakítások aránya a nagyobb, míg a kivételek esetében a szoftver alapú kivételek aránya a nagyobb. A kivételek kezelését egy erre a célra szolgáló kernelmodul, a kivételkezelő (Exception Dispatcher) végzi. A kivételkezelő feladata a bekövetkezett kivétel beazonosítása (pl. hibás memóriacímzés, nullával való osztás, túlcscordulás, a nyomkövető programok töréspontjai), a szükséges adminisztráció elvégzése, a probléma megoldása, majd a felfüggesztett folyamat futásának folytatása. Az előbb felsorolt példák szoftveres események, melyek nyilván függetlenek az architektúra kialakításától.

A Windows NT alapú operációs rendszerekben létezik az úgynevezett csapda (Trap) fogalma is. Ez esetben azonban nem egy folyamat, hanem egy szál futásának a félbehagyásáról van szó. A mechanizmus végrehajtásához a CPU-t user módból kernel módba át kell váltani, majd meg kell hívni a trapkezelőt (Trap Dispatcher), amely azt hivatott eldönteni, hogy szoftveres úton kezelhető-e az adott probléma. Azért, hogy a szál végrehajtása az adott probléma lekezelése után folytatható legyen, a trapkezelő elvégzi a megfelelő adminisztrációt, eltárolja az operációs rendszer számára a folytatáshoz szükséges adatokat.

Objektumkezelés

A Windows NT alapú operációs rendszerekben az objektumok kezelését az executive egyik komponense az objektumkezelő (Object Manager) végzi. Feladata az objektumok előállítás, kezelése, törlése, és védelme. Az előbb felsorolt feladatokból következik, hogy az objektumkezelő tartalmazza az erőforrás kezelő műveleteket, melyek így jól definiáltak, egy helyen találhatók meg az operációs rendszeren belül.

Az objektumkezelő felé támasztott igények:

- Biztosítson egységes mechanizmust a rendszer erőforrásainak elérésére, illetve használatára.
- Az objektumok kezelését, és védelmét biztosító kód egy helyen legyen.
- A rendszer erőforrásainak használatát optimálisan korlátozva, folyamatosan kövesse nyomon a folyamatok objektumhasználatát.
- Megfelelő névtérrel biztosítsa az objektumok és objektum csoportok egyértelmű azonosítását.
- A Windows NT alrendszerek által támasztott követelmények kielégítése.
 - A Win32 (és anno a POSIX alrendszer) esetében szempont az, hogy egy folyamat legyen képes erőforrásokat örökölni a szülő folyamattól.
 - Legyen lehetőség kis- és nagybetűt megkülönböztető fájlnevek használatára.
- Biztosítani kell az objektumok életben tartását, ami azt jelenti, hogy egy objektumnak mindaddig rendelkezésre kell hogy állnia, amíg az összes vonatkozó folyamat be nem fejezte az objektum használatát.

A Windows NT alapú operációs rendszerekben a következő két objektumtípust különböztetünk meg:

- executive objektum típus
- kernel objektum típus

Az executive objektumok az összetettebb szerkezetűek, ezeket az executive réteg különböző komponensei hozzák létre. (a folyamatkezelő, a biztonsági alrendszer, a virtuálismemória-kezelő, az I/O-alrendszer)

A kernel objektumok egyszerűbb szerkezetűek, mégis alapvető funkciókat valósítanak meg. Ilyen például a szinkronizáció, amelyet az executive objektumok is igénybe vesznek. Az executive objektumok jellemzően több kernel objektumból épülnek fel.

A folyamatok az objektumokhoz egy úgynevezett handle (szó szerinti jelentése fogantyú, nyél, – „igazi” magyar elnevezése nincs, de „leíró”-ként hivatkozunk rá)

igénybe vételével férhetnek hozzá. A handle tartalma tehát az objektumhoz való hozzáférési információk összessége. Tartalmilag egy összetett elérési címről van szó, ami gyakorlatilag egy összetett pointer, aminek igénybe vételével lényegesen gyorsabban érhető el egy objektum annál, mintha az objektum nevére keresnénk, vagy hivatkoznánk.

Szinkronizáció

A folyamatok szinkronizálásának alapvető eszköze a kölcsönös kizárás, különös tekintettel a nem osztott elérésű erőforrások elérésének szabályozására. A kölcsönös kizárás különösen fontos a Windows NT alapú operációs rendszerek esetében, mivel a több egymagos CPU, illetve a több magos CPU architektúrák használatakor is ez elengedhetetlen. A kernel, hogy elláthassa a több szál által is használt adatstruktúrák kezelését, a kölcsönös kizárást megvalósító alapmodulokat (a korábban már említett primitíveket) veszi igénybe. Ebben az esetben is elengedhetetlen a szinkronizáció, melyet az executive réteg biztosít, szintén a primitívek használatával. A kernel kódok végrehajtásakor – a végrehajtás minden fázisában – a kernelnek biztosítania kell, hogy egy kritikus szakaszt csakis egyetlen egymagos CPU, vagy többmagos CPU esetén pedig egyetlen CPU mag hajt végre.

Mivel megszakítások gyakorlatilag bármikor bekövetkezhetnek, kezelésük a szinkronizáció során nem mindig oldható meg zökkenőmentesen. Amennyiben a kernel éppen egy globális adatstruktúrát módosít, előfordulhat, hogy pont egy olyan megszakítás történik, aminek a kezelő rutinja pontosan ugyanazt a globális struktúrát módosítaná. Az ilyen helyzetekben a Windows NT alapú operációs rendszerek más megoldást alkalmaznak egy CPU mag esetén, illetve több CPU vagy több CPU mag esetén. Egy CPU mag esetén a Windows NT alapú operációs rendszerek mozgástere kicsi. Az operációs rendszer mindössze annyit tehet, hogy a globális erőforrások használata előtt ideiglenesen letiltja azokat a megszakításokat, amelyek megszakítás kezelő programja ugyanazt a globális erőforrást venné használatba. Technikailag ez úgy történik, hogy a CPU futási szintjét a szóban forgó megszakítások közül a legmagasabb szintre emeli az operációs rendszer. Több CPU mag esetében ez nyilván nem elegendő, hiszen hiába emeli meg az operációs rendszer az egyik CPU mag prioritását, egy másik CPU mag ettől még készen állhat a megszakítás elfogadására. A kölcsönös kizárás ilyen esetben az úgynevezett lezárással (Locking) garantálható, ami úgy valósítható meg, hogy a szóban forgó globális erőforráshoz tartozó Spin-Lock (szószerinti jelentése forgózár, – „igazi” magyar elnevezése nem használatos) segítségével a kernel zárolja az erőforrást.

A Spin-Lock megszerzése után az operációs rendszer már prioritizálhatja a globális erőforrás használatára jelentkező igények végrehajtási sorrendjét.

Multiprocesszoros környezetben a globális adatstruktúrákhoz való hozzáférést az executive réteg komponenseinek is szinkronizálnia kell. A kernel funkciók hívásakor is célszerű a lezárás használata, annak ellenére, hogy járulékos időveszteség nélkül elvileg sem vitelezhető ki maga a lezárás, hiszen a prioritási sorban elfoglalt helyének megfelelően minden folyamat több-kevesebb ideig mindenképpen várakozni kényszerül, sőt az összes érintett CPU mag is várakozni kényszerül. Ezért a módszer nem alkalmazható korlátozásmentesen:

- A lezárással védett erőforrásnak gyorsan elérhetőnek kell lennie, azaz nem lehetnek szerteágazó logikai kapcsolatai.
- A kritikus szakaszok nem lapozhatók ki a memóriából, nem hívhatnak külső eljárásokat (rendszer szolgáltatásokat sem), valamint nem okozhatnak se megszakítást, se kivételt.

Ezek a korlátozások – különösen annak figyelembe vételével, hogy az executive objektumok a kölcsönös kizárás mellett igényt tarthatnak az előidejűsége, illetve az egyidejűsége is, valamint a user folyamatokkal is megkövetelt szinkronizáció miatt – nem minden esetben érvényesíthetők pusztán a Spin-Lock igénybevételével. Szükség van további szinkronizációs objektum kezelőkre (Dispatcher Object) is, melyek megfelelő függvényhívások segítségével képesek dinamikusan kezelni – azaz főleg várakoztatni – a hozzáférést igénylő objektumokat (WaitForSingleObject, WaitForMultiplyObject). Az operációs rendszer a várakoztatott objektum felszabadulásakor, annak típusától függően, egy vagy több várakozó folyamatot is futásra kész állapotba tehet.

Lokális eljáráshívás (LPC)

A lokális eljáráshívás (LPC / Local Procedure Call) a folyamatok közötti kommunikációt biztosítja gyors üzenet továbbítással. Windows NT alapú operációs rendszerek esetében az LPC kizárólag a rendszer komponensek számára használható, így például a Win32 API-n keresztül nem is érhető el. Az LPC tipikus esete a Windows NT alapú operációs rendszerek szolgáltatásait realizáló server folyamatok és a kliens folyamatok közötti kapcsolattartás. LPC kapcsolat létrehozható két user módú folyamat között, vagy egy kernel módú komponens és egy user módú folyamat között. Maga az üzenetváltás realizációja az üzenet méretétől függ. Rövid, azaz 256 bájt üzenetméret alatt egy egyszerű puffer elegendő, nagyobb adatmennyiség esetén azonban közös elérési, osztott memóriaterületre van szükség.

2. A Windows NT alapú operációs rendszerek folyamatai és szálai

Alapfogalmak:

- A program maga a végrehajtható kód.
- A folyamat egy végrehajtás alatt lévő program(rész).
- A folyamat egy szála az, ami éppen fut egy CPU magon, és nem maga a folyamat.
- Minden folyamathoz tartozik legalább egy szál, ami elinduláskor elkezd futni.

A Windows NT alapú operációs rendszerekben a programok futtatásához a folyamatok egy vagy több – a program aktuális kódját végrehajtó – szálból állnak. A szálak az erőforrásaikat nem közvetlenül az operációs rendszertől kapják, hanem azokat az őket létrehozó folyamatoktól öröklik.

A Windows NT alapú operációs rendszerek folyamat modelljének tartalma:

- A végrehajtandó program kódja és adatai.
- A saját virtuális memória címtere, ami a folyamat rendelkezésére áll.
- Rendszererőforrások (szemaforok, fájlok, stb.) amelyeket az operációs rendszer foglalt le a folyamat részére. Ezek megfelelő része öröklődik igény esetén a folyamathoz tartozó szálakhoz.
- A folyamat egyedi azonosítója (Process ID / PID).
- Legalább egy végrehajtható szál.

A szálak a következő komponensekből állnak:

- A szálat végrehajtó CPU mag regiszterei, melyek a CPU mag állapotát írják le.
- Két veremtárat (Stack), egyiket a kernel módú-, a másikat a user módú futáshoz.
- Egyedi, osztatlan tárterületet a DLL-ek, Run-time könyvtárak részére.
- A szál egyedi azonosítóját (Thread ID). Mivel a Thread ID és a Process ID ugyanabból a névtérből kerül kiosztásra, így kizárt az azonos név használata.

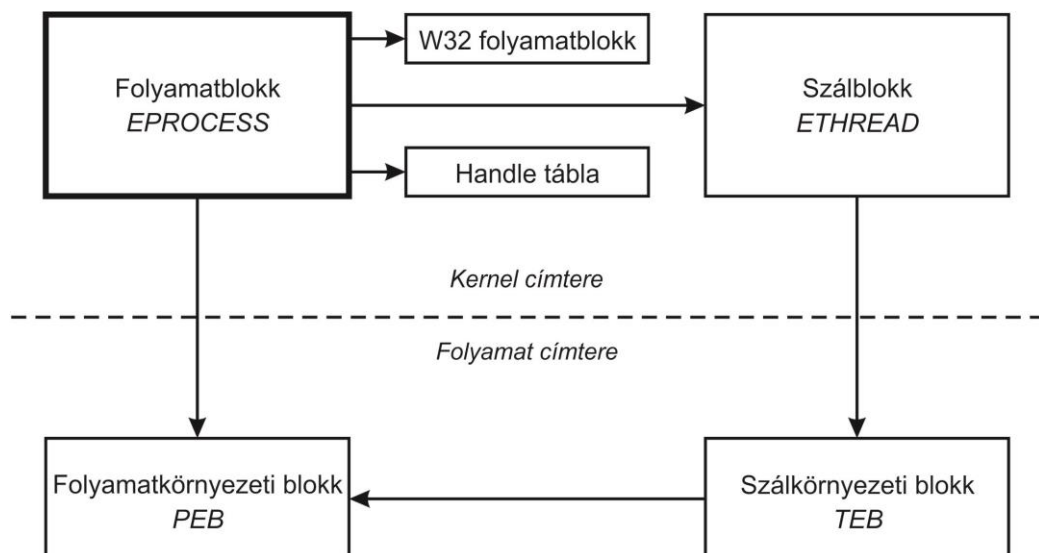
A fenti lista első három eleme együttesen alkotja a szál környezetét (Thread Context).

Az egy folyamathoz tartozó szálak közös virtuális címtartományt használnak. A különböző folyamatok természetesen külön, önálló címtartományban futnak, így például csak az osztott elérésű memória használata esetén lehet átfedés a két vagy több folyamat által használt memóriaterületek között.

A folyamatok – hosszabb távon a folyamatokat alkotó szálak – által használni kívánt memóriaterületeket természetesen az operációs rendszertől kell igényelni, és le is kell foglalni azokat. A folyamatnak továbbá a használni kívánt erőforrásokat is le kell foglalnia, amelyeket a Windows NT alapú operációs rendszer objektumokként reprezentál. A folyamat a későbbi gyorsabb elérés érdekében egy egyedi handle-t is rendel közvetlenül a megnyitás után minden ilyen objektumhoz. A rendszer erőforrásainak védelme miatt, valamint az erőforrások használatának szabályozása érdekében minden folyamathoz tartozik egy elérési token. Az elérési token az adott folyamat jogosultságainak leírása mellett a folyamat biztonsági azonosítóját is tartalmazza.

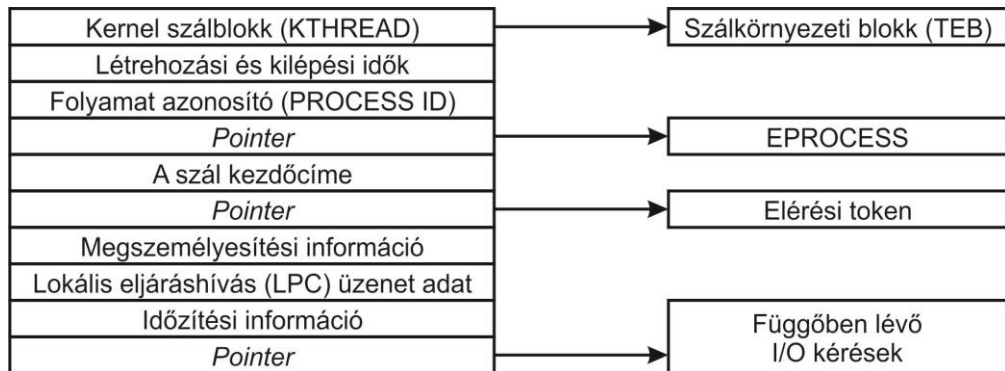
A folyamatok és a szálak adatstruktúrája

A folyamatokhoz tartozó adatokat az executive réteg egy folyamat blokkban (EPROCESS) tárolja el. Az EPROCESS és a hozzá tartozó adatstruktúrák – egy blokk kivételével – mind a kernel címterében találhatóak. Ez a kivétel a folyamatkörnyezeti blokk (Process Environment Block / PEB) ami a folyamat címterében található. Ennek oka, hogy a PEB – ami a folyamat futási környezetét leírja – az az adatstruktúra, ami olyan információkat tartalmaz, melyeket a user módban futó kódok is meg kell hogy tudjanak változtatni. A Win32 alrendszerben a CSRSS.EXE folyamat hozzárendel a Win32 kódot végrehajtó folyamatokhoz ezen felül még egy további folyamatleíró adatstruktúrát (W32PROCESS) is.



A Windows NT alapú operációs rendszerek a folyamatokhoz már azonnal, az indulásukkor hozzárendelik EPROCESS-t és annak adatstruktúráit, így kapcsolva össze a folyamat kísérő adatait, és a kapcsolódó adatstruktúrákra vonatkozó mutatókat. A folyamathoz tartozó egy vagy több szál az executive szálblokk (ETHREAD) – vagy más néven: az adminisztratív szálblokk – írja le.

Az executive szálblokk felépítése a következő ábrán látható.



Az összes olyan információt, amire a kernelnek szüksége van a szálütemezéshez és a szinkronizációhoz a kernel szálblokk, a KTHREAD tartalmazza. A többi mező a szál vonatkozó adatait, illetve a vonatkozó mutatóit (Pointer) tartalmazza.

Folyamat létrehozása

Egy Win32 folyamat úgy keletkezik, hogy egy alkalmazás meghívja a Win32 CreateProcess függvényt. A folyamat létrehozása komplex, összetett feladat, melyben az operációs rendszer következő három eleme vesz részt:

- a Win32 kliensoldali könyvtárban a KERNEL32.DLL
- a Windows NT executive
- a Win32 alrendszer folyamat (CSRSS)

Az előbb felsorolt három elem a következő hét lépésben hajtja végre a Win32 CreateProcess függvény hívását:

1. A vonatkozó paraméterek ellenőrzése után, az alrendszer jelzőit (Flag) és opcióit – valamint az attribútum listát is – elemzés és érvényesítés után át kell állítani a natív megfelelőikre.
2. Azon végrehajtandó fájl (.EXE) megnyitása, aminek az elindítandó folyamaton belül végre kell hajtódnia.
3. A Windows NT executive folyamat- és szálkezelője létrehozza az új folyamat objektumot.
4. Az indításhoz szükséges, úgynevezett kezdeti szál (Initial Thread) létrehozása.

5. Az alrendszer specifikus inicializáció, azaz Win32 alrendszer értesül az új folyamatról és a kezdeti szálról.
6. A kezdeti szál elindítása (kivéve azt az esetet, ha a CREATE_SUSPEND flag érvényes rá).
7. Az új folyamat és szál környezetben a címtér inicializálása (a vonatkozó DLL-ek betöltése) után elkezdődik a program végrehajtása.

Általános megjegyzések a fenti lépésekkel kapcsolatban:

- A CreateProcess függvényben az új folyamat prioritási osztálya független bitként van megadva a CreationFlags paraméterben. Így egyetlen CreateProcess híváshoz akár több prioritási osztály is specifikálható.
- Amennyiben nincs egyedi prioritási osztály hozzárendelve az új folyamathoz, a prioritási osztály a normál (Normal) besorolással indul, kivéve, ha a létrehozó folyamat prioritási osztálya tétlen (Idle) vagy a normál (Normal) osztály alatti. Ez utóbbi esetekben az új folyamat prioritása ugyanolyan osztályú lesz, mint amilyen a létrehozó folyamaté volt.
- Grafikus megjelenítés esetén minden ablakot egy adott grafikus környezethez (Desktop) rendel a rendszer. Egy user akár több desktopot is használhat. Ha a CreateProcess külön nem definiálja, hogy melyik desktopot használja a folyamat, akkor alapértelmezetten a hívó aktuális desktopjához rendelődik hozzá a folyamat.
- Amennyiben az új folyamathoz valós idejű prioritásosztály (Real Time) van megadva, de a folyamat hívója nem rendelkezik az ütemezési prioritás növelésének jogosultságával, akkor a magas prioritású (High) osztályt kell használni.

Szálak létrehozása és kezelése

A kezdeti szál és környezetének létrehozása a következő lépésekkel indul.

1. Ezen a ponton a Windows NT executive folyamat objektuma teljesen készen áll. Ennek ellenére még mindig nincs szála, így még nem tehet semmit. Az új szál létrehozásáért a PspCreateThread függvény felelős, amit az NtCreateThread hív meg. Mivel azonban a kezdeti szálat a kernel hozza létre – akár felhasználói módú bemenet nélkül is – a PspCreateThread két segítő függvényt kell hogy igénybe vegyen, a PspAllocateThread és a PspInsertThread függvényeket.

2. A PspAllocateThread kezeli az executive szál objektum tényleges létrehozását és inicializálását. A PspInsertThread végzi a handle és a biztonsági attribútumok létrehozását, valamint a KeStartThread hívását, ami az executive objektumot ütemezhető szállá alakítja. A szál azonban még nem tesz semmit, mivel felfüggesztett állapotban jön létre.

Az ezután következő lépéssort a PspAllocateThread a hajtja végre:

1. Létrejön és inicializálódik egy executive szálblokk (ETHREAD).
2. A szál végrehajtása előtt kell beállítani a szál környezetet (Thread Context).
3. A szálkörnyezeti blokk (TEB / Thread Environment Block) hozzárendelése az új szálhoz.
4. A user módú szál kezdési címe az ETHREAD-ben tárolódik. Ez a rendszer által biztosított szálindítási funkció az Ntdll.dll fájlban (RtlUserThreadStart). A felhasználó által megadott kezdőcím is az ETHREAD blokkban található, de egy másik helyen, így a hibakeresési eszközök, például a Process Explorer lekérdezhetik ezeket az információkat.
5. A KelnitThread meghívásra kerül a KTHREAD blokk beállításához. A szál kezdeti és jelenlegi alap prioritásai a folyamat alap prioritására vannak beállítva, processzor affinitása és kvantuma szintén a folyamaté. Ekkor lesz definiálva a kezdeti szálhoz az ideális processzor is. A KelnitThread ezután kernel veremtárat foglal le a szálhoz, és inicializálja a szál gépfüggő környezetet (Thread Context), a csapdát (Trap) és a kivételeket (Exception). A szál környezete úgy van beállítva, hogy a szál kernel módban indul a KiThreadStartup segítségével. Végül a KelnitThread inicializált állapotba állítja a szál állapotát, és visszatér a PspAllocateThread függvényhez.

Az előzőleg leírt lépések után, az NtCreateUserProcess meghívja a PspInsertThread függvényt a következő lépések végrehajtásához:

1. Létrejön egy szál azonosító (Thread ID) az új szálhoz.
2. A folyamathoz tartozó szálak száma eggyel növekszik, és a szál hozzáadódik a folyamat szálainak listájához.
3. A szál felfüggesztett állapotba kerül.
4. Az ObOpenObjectByName segítségével létrejön a vonatkozó handle.
5. A szál a KeStartThread meghívásával készen áll a végrehajtásra.

3. A Windows NT alapú operációs rendszerek ütemezése:

Alapfogalmak ismételése, összefoglalása:

- Windows NT alapú operációs rendszerek estén az ütemezés szál alapú, prioritásvezérelt, és preemptív.
- Folyamat (Process): Egy program futásának számára biztosított, fenntartott rendszererőforrások egysége, mely nagy vonalakban a következőket tartalmazza. A program kódja, és adatai, saját virtuális címterület, operációs rendszer által hozzárendelt rendszererőforrások (szemaforok, kommunikációs portok, fájlok, stb.), egyedi azonosító, legalább egy szál.
- Szál (Thread): Egy folyamathoz tartozó entitás, amin a program futása történik. Egy szálhoz a következő dolgok tartoznak: A processzor állapotát jelző regiszterek tartalma, két veremtár (kernel illetve user módbeli futáshoz), egyedi azonosító, saját tárolóterület, ahol a szálhoz kapcsolódó információk tárolhatók. A két veremtárat, a saját tárterületet és a tárolt regisztereket nevezzük a szál környezetének (Context). Ez az egyetlen olyan publikusan elérhető adatstruktúra a Windows NT alapú operációs rendszerekben, ami "számítógépfüggő".
- A Windows NT alapú operációs rendszerekben két processzor hozzáférési mód létezik. Az egyik a user mód, ahol a felhasználói programok futnak. Az itt futó két alkalmazás nem tudja sem egymás, sem az operációs rendszer fontos adatait módosítani, elérni. A másik a kernel mód, ahol az operációs rendszer fut. Egy kernel módban futó kód eléri a teljes memóriát, és minden processzorutasítást használhat.
- Megszakítási szintek (IRQL): A különböző típusú processzoroknak eltérő a megszakítási rendszere, ezért definiáltak a kernelben egységes, hordozható megszakítási szinteket. Egy bizonyos szintű megszakítás után már nem fogadható el az aktuálisnál alacsonyabb szintű megszakítás, az aktuálisnál magasabb szintű megszakítás viszont félbeszakíthatja az aktuális megszakítás feldolgozásának menetét. Az ütemezés során az első három szinttel találkozhatunk (IRQL 0-2), melyek "szoftver megszakítások" (azaz nem egy hardver eszköz váltja ki őket).
IRQL 0: a CPU egy normál kernel vagy user módú folyamatot futtat
IRQL 1: a CPU egy aszinkron programhívást (APC) vagy laphibát futtat
IRQL 2: a CPU egy halasztott eljárashívást (DPC) és szálütemezést végez

Mivel az ütemezés szál alapú, ezért ha egy folyamat több szálon fut, mint egy másik folyamat, és a szálak prioritása azonos, akkor az első folyamatnak több idő fog jutni (ha – első megközelítésben – úgy tekintjük, hogy minden szál azonos ideig fut).

Az ütemezés prioritásvezérelt, vagyis az, hogy melyik szál fog futni, alapvetően a szálak prioritásától fog függeni. Alapvetően mindig a legnagyobb prioritású, futásra kész szál fut, hacsak a szál processzor affinitása ezt nem korlátozza. A processzor affinitás egy szálhoz azokat a processzorokat (magokat) rendeli, melyeken a futása engedélyezett.

Az ütemezés preemptív, vagyis egy szál futását az ütemező bármikor megszakíthatja, ha például egy magasabb prioritású szál lesz futásra kész. A Windows NT alapú operációs rendszerek ütemezője a kernel kódjában “szétszórva” található, azaz nincs egy kitüntetett ütemező modul.

Az ütemezés jellemzői:

- Preemptivitás
- Időszlet alapú ütemezés
- Többprocesszoros ütemezés

Többprocesszoros ütemezés (illetve több CPU mag) esetén a cél a szálak (azaz a terhelés) egyenletes elosztása, a következő három paraméter segítségével:

- processzor affinitás
A szál azon paramétere, hogy melyik processzoron, illetve magon fusson.
- ideális processzor
Azaz az előnyben részesített processzor, illetve mag. Ha a szál külön nem kéri, akkor véletlenszerűen választódik ki a tétlen processzorok közül az, amelyiken a szál futni fog. Ha nincs tétlen processzor, akkor a prioritás dönt a processzor használatról.
- következő processzor
Azaz a másodlagosan preferált processzor, illetve mag, jellemzően az, amin a szál legutóbb futott.

Az ütemezéssel kapcsolatos rutinok (összefoglaló néven: diszpécser) IRQL2 szinten futnak, és a következő események válthatják ki a működésüket:

- egy szál végrehajtásra készvé válik
- egy szál abbahagyja a futást
- megváltozik egy szálnak a prioritása
- megváltozik a végrehajtás alatt álló szál processzor affinitása

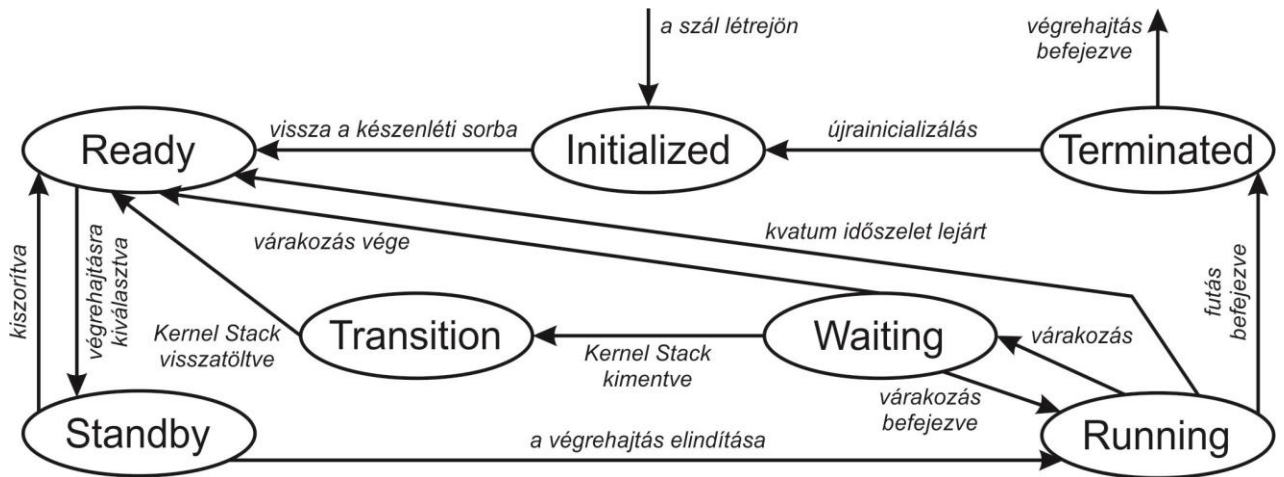
A kvantum időszlet

Minden szálnak tartozik egy egyedi kvantum időszlet (Quantum), melynek lejártá után a szálnak mindenképpen abba kell hagynia a futást. Ezután újbóli kiválasztás és döntés születik, hogy melyik szál legyen a következő, amely futni fog. Egy kvantum időszlet tehát az a maximális időtartam, amennyit egy szál futhat, mielőtt a Windows NT alapú operációs rendszer biztosan megszakítja a szál futását. A futás megszakításának a célja az, hogy kiderülhessen, nem vár-e futásra egy másik, ugyanakkora prioritású szál, illetve, hogy szükség esetén csökkenteni lehessen a megszakított szál prioritását. Előfordulhat az is, hogy egy szál el sem jut a kvantum időszlete végéig, a preemptív ütemezés következtében. Ugyanis, ha egy másik, nagyobb prioritású szál futásra kész állapotba kerül, akkor a futásban levő szál futása még a saját időszlete lejártá előtt meg lesz szakítva, vagyis a szál ki lesz szorítva. Így az is előfordulhat, hogy egy szál hiába választódik ki futásra, még azelőtt ki is lesz szorítva, hogy elkezdené a kvantum időszletét.

Az egyes szálak futására rendelkezésre álló kvantum időszletek száma nyilván egy viszonylag kis érték kell hogy legyen, hiszen szálak komplexitása töredéke a folyamatok komplexitásának. Ez a kvantum érték.

Órajelmegszakításonként a Windows NT alapú operációs rendszerek a szálak kvantum értékét 3-al csökkentik. A kvantum érték a Windows NT Workstation esetében 6-ról, a Windows NT Server esetében pedig 36-ról indul. Ez azt jelenti, hogy Windows NT Workstation esetén 2, Windows NT Server esetén 12 alkalom alatt van lehetősége lefutni egy szálnak. A Windows NT Server esetében azért nagyobb ez az érték, mert statisztikailag ennyi idő alatt jellemzően befejezhető egy kliens kiszolgálása is. Egy előtérben futó alkalmazás szálainak kvantum értéke indokolt esetben a fent leírtakhoz képest akár nagyobb is lehet.

A szálak állapotai



Egy szálnak a fenti ábra szerint a következő állapotai lehetnek:

- **Inicializált (Initialized)**
Az induló lépés egy újonnan létrejött szál objektum inicializálása. Egy már befejezett szál objektum az újrainicializálása után is ebbe az állapotba kerül.
- **Végrehajtásra kész (Ready)**
Azok a szálak, melyek futásra készek, itt arra várnak, hogy a diszpécser által kiválasztásra kerüljenek. A Terminated állapot kivételével bármelyik másik állapotból ide kerülhet a szál.
- **Készenléti (Standby)**
Ez a végrehajtásra kiválasztott, processzoronként (CPU magonként) egyetlen szál helye. Minden processzorhoz (azaz CPU maghoz) csak egy olyan szál tartozhat, amely már ki van választva futásra, de még nem fut, mert pl. maga az ütemező is ezen a processzoron fut. Közvetlenül a Ready állapotból kerül ide egy szál, és innen vagy a Running állapot következik; vagy ha egy magasabb prioritású szál kiszorítja, akkor visszakerül a Ready állapotba.
- **Futó (Running)**
A végrehajtás alatt álló szál állapota. A szál ide vagy a Standby állapotból, vagy egy várakozás befejeződése után kerülhet. Ha a szál futása befejeződik, akkor Terminated állapotba kerül. Ha a szál várakozni kényszerül, akkor Wait állapotba kerül, illetve ha a kvantum időszelete lejár, de kvantum értéke még nem nulla, akkor a Ready állapotba kerül vissza.
- **Várakozó (Waiting)**
A szál akkor kerül Wait állapotba, ha valamilyen szinkronizációs objektumra, vagy I/O műveletre vár, illetve ha egy környezeti alrendszer utasítja erre a szálat. A szál a várakozási idejének letelte után, a prioritásának megfelelően, kerülhet újra Running állapotba, Ready állapotba vagy Transition állapotba.

- **Átmeneti (Transition)**
Ebbe az állapotba csak akkor kerülhet egy szál, ha a szál kernel veremtére kilapozódott a memóriából a várakozás közben. A veremtár visszalapozása után a szál újra Ready állapotba kerül.
- **Befejezett (Terminated)**
A szál befejezte a futását, végrehajtása véget ért. Ezután vagy törölhető a szál objektum, vagy újrainicializálható.

Prioritási szintek

Amikor prioritási szintekről beszélünk, két dolgot kell megkülönböztetnünk. Az egyik az a prioritás, mely a Windows NT alapú operációs rendszer alatt a programozó által elérhető, és a Windows API-n keresztül állítható be. A másik, mely a kernel belső ábrázolásában használatos, így ehhez csak a kernel folyamatai férnek közvetlenül hozzá. Az API-ból beállított érték is végül egy ilyen értéként képeződik le.

Egy folyamatnak adott prioritási osztállyal a folyamatunk fontosságát fejezhetjük ki. Ezen kívül a folyamathoz tartozó minden szálnak egyenként adhatunk egy prioritást, mely a folyamaton belül, a szálak közötti fontossági sorrendet jelenti. Ebben az anyagban korábban már szerepelt, hogy a Windows NT alapú operációs rendszerek ütemezése szál alapú, prioritásvezérelt, és preemptív, de fontosnak tartom erre itt újra felhívni a figyelmet, mivel mindez még hangsúlyosabbá teszi a prioritási szintek és osztályok szerepét a CPU-hoz jutás során.

Az API-ból beállított prioritásokból végül is minden szálnak képződik egy 0 és 31 közötti érték, mely a folyamat és a szál megadott prioritásának kombinációjaként jön létre a következő leképezés alapján:

		Folyamat prioritási osztályok			
		Real-time	High	Normal	Idle
Szál prioritási szintek	Time critical	31	15	15	15
	Highest	26	15	10	6
	Above normal	25	14	9	5
	Normal	24	13	8	4
	Below normal	23	12	7	3
	Lowest	22	11	6	2
	Idle	16	1	1	1

A prioritási osztályok szintjei három csoportra oszthatók.

- 16-31 valós idejű szintek
- 1-15 változó szintek
- 0 zero szint (a táblázatban nem szerepel)

Az első, a real-time folyamat prioritási csoportban használatos értékek, vagyis a 16-tól 31-ig terjedő prioritásértékek. Ha egy szálnak ilyen a prioritása, akkor azt az operációs rendszer nem módosítja. A real-time, azaz a valós idő a gyakorlatban leginkább csak elvileg létezik, közelebb áll a valósághoz a just-in-time, azaz a megfelelő időben megközelítés, ami a lehető legkisebb késedelemre utal.

A másik csoport az 1-től 15-ig lévő tartomány, ahol is az operációs rendszer ezen a tartományon belül módosíthatja is a szál aktuális prioritásának értéket egy időre, a megadott érték csak egy alapérték. A módosítás viszont csak ebben a tartományban történhet, vagyis egy szál sohasem kerülhet fel ilyen módosítás során a 16-31 lévő csoportba. A módosítás lehet emelés (éheztetés vagy I/O folyamat vége után) illetve csökkentés (a szál futásának megszakítása után).

A harmadik csoport valójában nem is csoport, csak egy különálló érték, a 0 prioritás. Ez igazából egy szálnak sem tartozhat, ezért nem is szerepel a fenti táblázatban. A Windows NT alapú operációs rendszerek minden processzorhoz (CPU maghoz) egy darab "látszólagos" szálakat rendelnek hozzá, melyet a Task Manager-ben "System Idle Process" néven találhatunk meg.

Alapvetően minden szál normál prioritással kezdi a futását.

Előadás_#12.

1. Az NT alapú operációs rendszerek memóriakezelése

A Windows feladatkezelő (a Teljesítmény fülön) a memória állapotáról is ad információt a felhasználó számára, a következők szerint. A Memória elnevezésű területen ezt láthatjuk:

- Windows 10 esetén a legfelső sorban található az összes érték, azaz a számítógépre telepített RAM mennyisége gigabájtban. A következő négy érték grafikusán is ábrázolásra kerül a „Memóriamegoszlás” részen. A kurzort a megfelelő terület fölé mozdítva jelenik meg a részletes információ.
- A „Használatban” érték a folyamatok, az illesztőprogramok illetve az operációs rendszer által használt memória mennyiségét jelzi. A CPU-k teljesítményének növekedése tette lehetővé azt, hogy ezen memória mennyiség egy részét a rendszer tömörített formában is képes legyen kezelni, amivel így fizikai memória helyet tud megtakarítani.
- A „Módosított” érték annak a memória területnek a méretét jelzi, amit háttértárra kell áthelyezni.
- A „Készenléti” az a memóriamennyiség, mely aktívan nem használt, gyorsítótárazott adatokat és programkódokat tartalmaz. A Windows 10 sajnos hajlamos az összes rendelkezésre álló memóriát ide besorolni, ami a legjobb szándék ellenére is lassuláshoz vezet.
- A „Szabad” érték a jelenleg használaton kívüli, hasznos információt nem tartalmazó memóriát jelöli, melyet a rendszer elsőként oszt ki újra, ha a folyamatoknak, illesztőprogramoknak és az operációs rendszernek további memóriára van szüksége.
- A „Gyorsítótárazott” érték a jelenleg elfoglalt, de kihasználatlan memórialapok méretét jelöli.
- A „Rendelkezésre álló” érték az, amely elérhetővé tehető a folyamatok, az illesztőprogramok és az operációs rendszer általi használatra.

Ezen kívül érdemes megnézni a virtuális memória állapotát is. Gyakorlatilag a szabadnak jelzett memória területnél nagyobb memóriaigényű program is elindítható, hiszen egyrészt az osztott memóriahasználat miatt lehet közös memóriaterülete más programokkal, másrészt a foglalt, de használatban nem lévő memória egy része akár át is helyezhető háttértárra. Ha a program, amit el akarok indítani, sok megosztott memóriát használ, akkor lehet, hogy sokkal kevesebbet foglal el más programokkal együtt futtatva, mint akkor, ha csak egyedül fut.

A memória maximális méretének fizikai okokból (azaz az architektúrából következő) illetve üzleti megfontolásokból fakadó különböző korlátai vannak. Ezen korlátok részleteivel az Operációs Rendszerek tárgy első gyakorlata foglalkozott.

A 32-bites (x86) illetve a 64-bites (x64) rendszereket megvizsgálva szembeötlő különbség a fizikai memória mérete. Természetesen a 32-bites rendszerek egy részénél a Microsoft bevetett memória méret növelő eszközöket. Ilyen eszköz például a Physical Address Extension (PAE). A PAE egyes 32-bites processzorokban meglévő támogatás, aminek a lényege az, hogy ez esetben 32 címbit helyett 36 címbit van használatban. Amelyik 32-bites operációs rendszer ezt ki tudja használni, az képes akár 64GB fizikai memóriát is kezelni. Figyelni kell azonban arra, hogy a folyamatok címtére továbbra is 32-bites marad, így csak az operációs rendszer tud több fizikai memórialapot kiosztani a folyamatoknak.

Jellemzően a 32-bites rendszereken (ahol nincs PAE), 4 GB fizikai memória esetén is néha kevesebbet lát az operációs rendszer, mert a memóriaterület felső részére I/O eszközöket szokás berakni (pl. ha van egy 512 MB memóriagigánt "alaplap" videokártyánk, akkor nyilván csak 3,5 GB-ot lát szabadon az operációs rendszer).

Kliens operációs rendszerek esetében nem használatos a PAE támogatás, mert a kliensekben lévő eszközök meghajtói jellemzően nem, vagy nem megfelelően kezelik le a 4GB-nál több fizikai memóriát.

A 64-bites NT alapú operációs rendszerek kompatibilisek továbbá az IA-64-el (Intel Architecture 64), egy az Intel és a Hewlett-Packard által közösen kifejlesztett utasításkészlet-architektúrával is (Instruction Set Architecture, ISA; az utasításkészlet-architektúra az utasításokon kívül a felépítés több más elemét is meghatározza, pl. az adatformátumokat, regisztereket, címezési módokat, a memória felépítését, a megszakítás- és csapdakezelést, valamint a kapcsolatot a külvilág felé – I/O). Az IA64 architektúra jelenleg csak az Itanium és az Itanium 2 processzorokban került megvalósításra.

A Windows NT alapú operációs rendszerek memóriakezelőjének egyik alapvető feladata, hogy a folyamatok virtuális címtérének címeit hozzárendelje a fizikai címekhez, így valósítva meg a logikai- és fizikai címek transzformációját. A másik alapvető feladata a virtuális memóriakezelés megvalósítása. A gyakorlatban ez azt jelenti, hogy a memória telítődésekor bizonyos (pl. a régen nem használt) memóriaterületeket a háttértárra menti. Természetesen a folyamat kétirányú, azaz ha olyan memórialapra történik hivatkozás, amely a háttértáron található, akkor memóriakezelő a keresett lapot visszamozgatja az operatív memóriába. A mozgatás alapegysége minden esetben a memória lap (Page).

A fent említett alapszolgáltatáson felül a Windows NT alapú operációs rendszerek memóriakezelője a rendszer hatékonyságát megnövelő többlatszolgáltatásokat is nyújt a felhasználóknak. A legfontosabb ilyen szolgáltatás a fájlok memóriaként történő elérése (Memory Mapped Files), ami lehetővé teszi a fájlok osztott elérését, illetve a Copy-on-Write mechanizmus használatát.

A Windows NT 32-bites memóriakezelője x86 címtartományban alapesetben két egyenlő részre osztja a memóriát, azaz a 4GB-ot egy alsó és egy felső 2 – 2GB-ra. Az alsó 2GB az operációs rendszeré, a felső 2GB pedig az alkalmazásoké.

Alapesetben egy 32-bites rendszerben egy felhasználói folyamat maximum 2GB-os címtartományt használhat fel a saját kódjának és adatának tárolására. A boot.ini-ben a /3GB kapcsoló aktiválásával az arány 3GB – 1GB-ra változik. Ez eleve egy kényszer megoldás volt, azon cél érdekében, hogy egy folyamat – például egy nagy adatbázis-kezelő – a 64-bites rendszer nélkül is képes legyen 2GB feletti memóriahasználatra. Vista óta ezt 4GT-nek hívják (4-Gigabyte Tuning).

64-bites memóriakezelés esetén a címtér (ami egyébként 48-bites) maximuma jelentősen megnő, hiszen 6.657GB válik címezhetővé a rendszer folyamatainak számára, és 8.192GB a felhasználói folyamatok számára. Nyilván ehhez a megfelelő mennyiségű memóriára is szükség lenne, azaz az x64 egy lehetőség, amit megfelelő HW környezetben ki tudunk használni, hiszen jelenleg a maximális fizikai memóriaméret 2.048GB, azaz a rendszerben van még bőven tartalék.

A memóriakezelés alapelvei:

- Virtuális tárkezelés

- Lapszervezés (4kB / 2MB méretű lapok, 2/3/4 szintű).
- Lapozófájl használata.

Kezdetben az igény szerinti lapozás volt jellemző, de ez nem bizonyult kellően hatékonynak.

Clustering: Az igény szerinti lapozás kiegészítése azzal, hogy a kért lap környékén lévő pár lapot is behozta (lokalitás elv).

Prefetch: XP óta van egy Prefetcher, ami rögzíti, hogy a programok induláskor miket szoktak igényelni, és az azokhoz tartozó lapokat előre behozza. A gondot az okozza, hogy a rendszer Prefetch nélkül először pár lapot az egyik fájlból, majd pár lapot a másik dll-ből tölt be (ahogy éppen az utasítások jönnek a program kódban), és így a merevlemezen a fejnek össze-vissza kell mozognia. Ezért egy adott program esetében a Prefetcher az első 10 másodperc hozzáféréseit megjegyzi és következő programindításkor ezek szerint jár el, hiszen adott program

esetében mindig ugyanazokat a lapokat kell betölteni. A lapok ismeretében a betöltés lehet aszinkron módú is.

Vista óta további Prefetch jellegű szolgáltatások is vannak:

Superfetch: 8 szintű prioritást rendel a memórialapokhoz, követi a betöltött lapok "életciklusát", így képezve egy stand-by listát, amire felkerülnek azok a lapok, amelyek nincsenek a memóriában, de előreláthatóan hamarosan kelleni fognak.

ReadyBoost: Frappánsan hívhatjuk úgy, hogy "a szegény ember SSD-je", hiszen funkcióját USB Flash alapú eszköz használatával éri el. Célja a merevlemez hosszú elérési idejének csökkentése, ReadyBoost tehát nem a memória bővítésére szolgál, hanem a lassú lemezműveletek felgyorsítására. Lényege hogy egy program számára a HDD-ről használandó adatok előre áttöltődnek a Flash meghajtóra, ahonnan a rendszer gyorsabban éri el azokat.

- Hatékonyság
 - Igény szerinti lapozás + Clustering + Prefetch.
 - Memória megosztás, Copy-on-Write.
 - Fájl cache-elés memóriában (Memory Mapped File).
- Biztonság
 - Minden folyamatnak külön címtartomány.
 - Elérés leírókon keresztül (hozzáférési token).

A memóriakezelő felhasználói interfész

A memóriakezelő szolgáltatásai a Win32 API-n keresztül érhetők el, szolgáltatásai a következők:

- Virtuális memória allokáció, illetve felszabadítás.
- Osztott elérésű (Shared) memória létrehozása.
- A file-ok osztott elérésű memóriához hasonló elérése.
- Virtuális memória kezelés
(pl.: információkérés, tárolás memóriába, kiírás háttértárra).
- Memória védelmi funkciók.
- Kernel szintű funkciók (elsősorban a Device Driver-ek) támogatása
(pl.: fix fizikai memóriaterület használata).

A folyamatok memória foglalása

A memória foglalás mindig két lépésben történik. Előnye, hogy mindig csak annyi memória kerül lefoglalásra, amennyire a folyamatnak ténylegesen szüksége van.

- Reserve

A virtuális címtartomány lefoglalása, a szándék kifejezése, nem jelent fizikai memória foglalást. Jelzi az operációs rendszer számára, hogy mennyi memória lesz, vagy lehet szüksége. Alapértelmezett foglalási méret x86 és x64 esetében is 1MB, 4kB-os lapok formájában.

- Commit

A virtuális memória tényleges lefoglalása. Alapértelmezett foglalási méret x86 és x64 esetében is 4kB, azaz egy lapnyi. A Commit művelet a Reserve művelettel lefoglalt területen belül hajtódik végre, igény szerinti lappal. Az a folyamat, amelyik a memória foglalását kezdeményezte csak a Commit művelet után veheti a memória lapokat használatba.

A két folyamat hívása történhet egy függvénnyel is, de fizikailag ebben az esetben is két lépésben fog végrehajtódni.

A két lépcsőben történő foglalás előnye a hatékonyabb működés:

- Reserve csak címtartomány foglalást végez.
(egy bejegyzés az operációs rendszer foglalási táblázatában)
- Tényleges erőforrás használat csak a Commit után indul.
- A folyamatok futtatásához használt fizikai memóriaigény csökkenthető.
- Lehetséges előre lefoglalni egybefüggő címtartományokat a rendszer terhelése nélkül.

A szálak user módú veremtárának (User Stack) memória foglalása

Alapkövetelmény, hogy a user módú veremtár egybefüggően kerüljön kijelölésre, azaz címtartománya folyamatos kiosztású legyen.

Egy szál indulásakor a Reserve jelzi az 1MB igényt, de a Commit fizikailag csak 2 lapot, azaz 2x4kB-ot fog lefoglalni. Az első lapban lesz a user módú veremtár indulási címe, míg a másik lap azt a célt szolgálja, hogy induláskor, pontosabban az első lap teljes kitöltése után további lapot, lapokat lehessen újabb Commit művelettel lefoglalni.

Amennyiben nem a Commit/Reserve rendszerben, az nem két- hanem egy lépésben történne a foglalás, akkor minden egyes szál indulásakor le kellene foglalni egy 1MB-os területet, melynek jelentős részét a szál valószínűleg nem is használná.

Osztott elérésű memória (Shared Memory)

Az osztott elérésű memória optimális memóriagazdálkodást tesz lehetővé, hiszen amennyiben két vagy több folyamat azonos adatokat használ, akkor ezen adatoknak elég fizikailag egyszer megjelenni a memóriában. Azaz a folyamatok adott virtuális címtartományához azonos fizikai memórialap kerül kiosztásra.

Például ha a rendszerben két folyamat is éppen C programot fordít. Az osztott memória használata lehetővé teszi, hogy a C fordító csak egy példányban foglaljon helyet a memóriában.

A kezelés alapeleme egy speciális objektum, az úgynevezett szekció objektum (Section Object). A szekció objektum jellemzői:

- neve
- helye, azaz hogy melyik virtuális memóriatartományban található
- az objektum indulásakor hozzá társított fájl vagy fájl részlet
- ACL (Access Control List) -ben meghatározott elérhetőségi paraméterei

A memória védelme

A memória védelme alatt azt értjük, hogy – az osztott elérésű memórián kívül – a se a folyamatok, se az operációs rendszer ne legyen képes módosítani egy másik folyamat címtartományába tartozó memóriaterületeket. Ez egy roppant fontos mechanizmus, ami a Windows NT alapú operációs rendszerek megbízhatóságát biztosítja.

A memóriavédelem megvalósítási szintjei:

- Kernel módban futó modulok adatstruktúrái a felhasználói módú folyamatok számára elérhetetlenek.
- Minden folyamat önálló logikai memória területet, címtartományt használ, amit a memóriakezelőn keresztül ér el, ami biztosítja a hardware-rel támogatott logikai-fizikai transzformációt, és a kizárólagos hozzáférést a memóriaterülethez.
- Processzorfüggő memória védelmi attribútumok
r, w, x, (írás, olvasás, végrehajtás)
A kódszegmensek memóriaterületei jellemzően csak olvashatók.
- ACL (Access Control List) az elérést szabályozó védelmi lista, amely arról rendelkezik, hogy az adott objektumot melyik folyamatok érhetik el.

A memória foglalása

A Windows NT alapú operációs rendszerek a memóriát mindig memórialaponként értelmezik és kezelik. A memóriakezelő feladata az, hogy kevés szabad fizikai memória esetén az arra kiválasztott lapokat a lapozás (Paging) művelettel a háttértárra mentse.

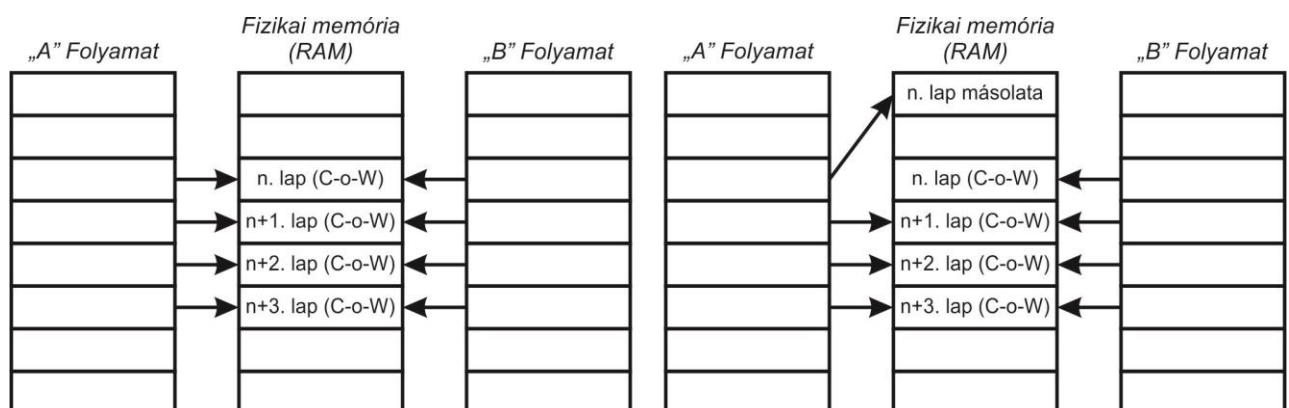
- Kernel komponensek esetén nem mindig megengedett a lapozás
 - kernel szinten megvalósított kölcsönös kizárás esetén
 - driver-ek által használt memóriaterületek fixen a memóriában maradnak, még akkor is, ha csak ritkán használjuk őket
- A kernel komponensek memória foglalása kétféle képpen történhet
 - lapozott memória tárból (Paged Memory Pool)
 - nem lapozott memória tárból (NonPaged Memory Pool)

A nem lapozott memória tár a memória azon védett részét jelenti, ami semmiképpen sem lapozódhat ki az operatív tárból, azaz a fizikai memóriából.

Copy-on-Write mechanizmus (másolás írás esetén)

A Copy-on-Write mechanizmuson alapuló memóriahasználat lényege a következő. Amennyiben egy folyamat egy olyan – írható és olvasható, más folyamatok által is potenciálisan használt – szekció objektumot kezd használni, akkor a rendszer alapértelmezésben csak átírja a memórialapokhoz tartozó Copy-on-Write (C-o-W) flag-et, de még nem készít a folyamat számára lokális másolatot.

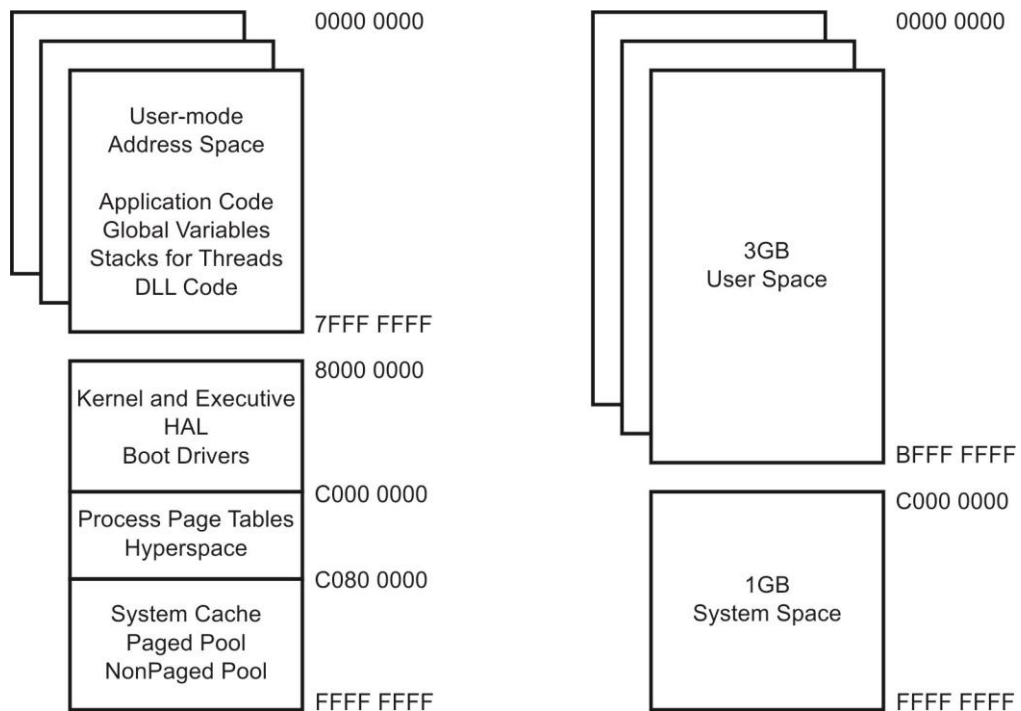
Tényleges másolás a rendszerben tehát csak azután történik, amikor az érintett szekció objektum bármelyik használója kezdeményez egy memória írási műveletet. Ekkor a rendszer mielőtt megváltoztatná a memóriatartalmat, a kérdéses memórialapról egy privát másolatot készít az írást kezdeményező folyamat számára. A másolat tehát azért és akkor készül az egész szóban forgó memória lapról, amikor az a cél, hogy a memória megváltoztatása csakis a kérdéses (az írást kezdeményező) folyamat számára legyen látható.



A Copy-on-Write mechanizmus a lusta kiértékelés (Lazy Evaluation) elvét követi. A lusta kiértékelés – mint elv – azt a végrehajtási módot jelenti, amikor a rendszerben a sok erőforrást igénylő (azaz költséges) műveletek egészen addig elhalasztásra kerülnek, amíg azok végrehajtása már ténylegesen elkerülhetetlené nem válik.

A logikai címtér felépítése

A 32-bites Windows NT alapú operációs rendszerek alapértelmezett 2GB+2GB címtérét a bal oldali ábra, a /3GB kapcsoló bekapcsolása utáni 1GB+3GB állapotot a jobb oldali ábra mutatja.



- Az alkalmazások címtartománya:
 - legfelső címtartomány a rendszer dinamikusan foglalt elemei
- Operációs rendszer címtartománya:
 - Kernel, HAL, Executive, Boot Drivers
 - A folyamatok memórialapjainak táblái
 - System Cache, Memória címtranszformációs táblázatok

Előadás_#13.

1. Az NT alapú operációs rendszerek fájlrendszere az NTFS

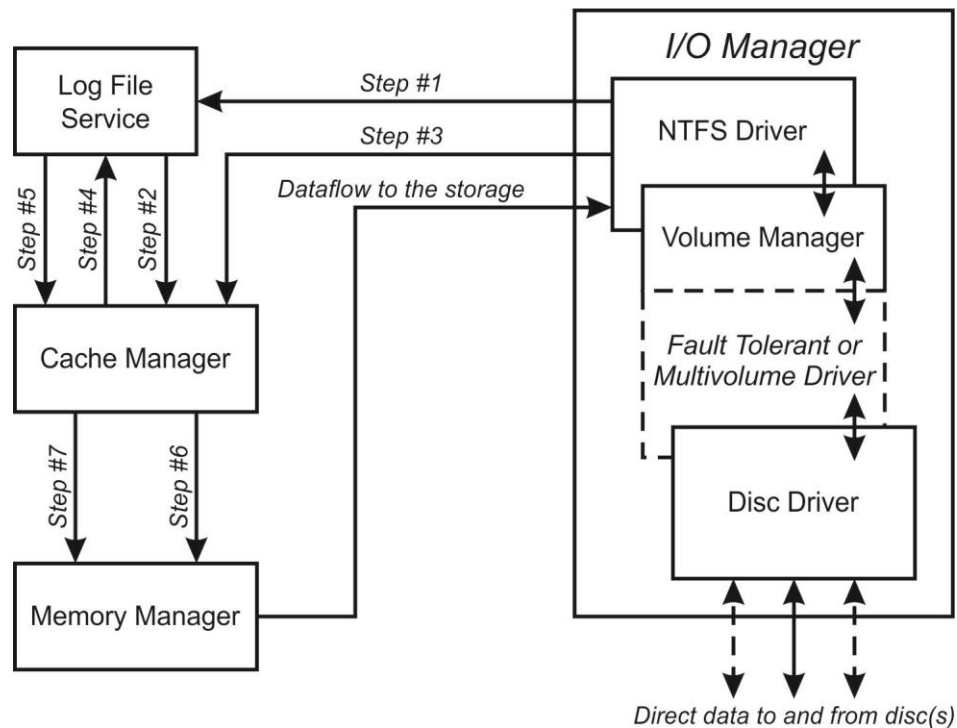
Korlátok a korábbi fájlrendszerek és az NTFS összehasonlításában:

- FAT12 alatt a fájl név 8.3 szerkezetű, egy fájl maximális mérete nem lehet több mint 32MB. A maximális partíció mérete is 32MB. A fájlok maximális száma egy partíción 4077db.
- FAT16 alatt a fájl név 8.3 szerkezetű. A maximális partíció mérete alapvetően 2GB (de pl. XP alatt 4GB). A fájlok maximális száma egy partíción 65.517db.
- A FAT32 támogatja a hosszú (255 karakter LFN / Long Filename) fájlneveket. Egy fájl maximális mérete nem lehet több mint 4GB mínusz 1Bájt. A maximális partíció mérete 2TB. A fájlok maximális száma egy partíción 268.435.427db.
- Az NTFS támogatja a hosszú (255 karakter) fájlneveket. Egy fájl maximális mérete nem lehet több mint 16TB. A maximális partíció mérete 256TB. A fájlok maximális száma egy partíción $2^{32}-1$ db. A korábbi (Microsoft által is támogatott) fájlrendszerek a „Convert” parancs segítségével NTFS-re konvertálhatók.

[pl.: Az „F:” meghajtót ez a parancs konvertálja FAT32-ről NTFS-re egy CMD ablakban: `convert f: /FS:NTFS`]

Az NTFS fájlrendszer működéséhez a három alapvető komponens az NTFS Driver, a Log File Service és a Cache Manager. A következő ábra nem csupán ennek a három komponensnek az együttműködését mutatja be, hanem bővebb áttekintést is enged, mivel a réteges felépítésű struktúra további opcionális rétegek (az ábrán szaggatott téglalapban) beépülését is lehetővé teszi. Az opcionális rétegek jellemzően többlet funkciókat valósítanak meg. További komponens még az adatok épségéért felelős Volume Manager, és a fizikai tárolókkal a közvetlen kapcsolatot biztosító Disc Driver.

Valós életben létező opció a szoftveres RAID támogatás, amely több lemez használatával képes a tárterületet és/vagy az adatbiztonságot illetve a sebességet is megnövelni. Például RAID 0 használata esetén több kisebb fizikai tároló összekapcsolásával egy nagy tárolót lehet kialakítani, RAID 1 használata esetén pedig az adatok duplikált tárolását teszi a rendszer lehetővé. A magasabb RAID szintek (1, 5, 6, 10, 50, 60) az adatbiztonság emelése mellett, az olvasás párhuzamosításával még sebesség növelést is biztosítanak.

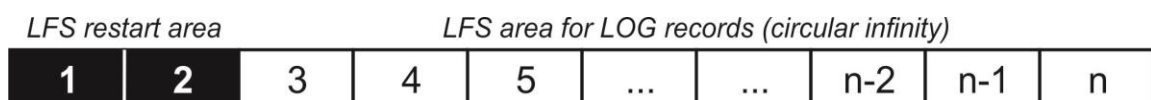


A fájl írási művelet kiszolgálásának lépései (Step #1-7):

1. Az NTFS Driver jelzi az LFS-nek, hogy fájl írás művelet következik, és egyben utasítja is, hogy készítsen a tranzakcióról egy úgynevezett „log rekord”-ot.
2. Az LFS hozzáír egy rekordot a cache-ben lévő log fájlhoz.
3. Az NTFS Driver a fájlt beírja a CM-be. (Nagyméretű fájlok nem írhatók ki egy lépésben, hanem csak a tároló blokkméretének megfelelő műveletenként.)
4. A CM jelzi, hogy az írás befejeződött, minden adat továbbításra vár.
5. Az LFS meghatározza, hogy melyik adatokat kell a cache-ből továbbítani: a log fájlt és a megváltoztatott teljes fájlt (vagy annak csak az aktuális blokkját).
6. A CM (a többi komponens segítségével) a lemezre írja a log fájlt.
7. A CM (a többi komponens segítségével) a lemezre írja a megváltoztatott fájlt (vagy annak csak az aktuális blokkját).

A log fájl szerkezete

A log file egy körkörös szervezésű tömb, mely egyrészt a fájlrendszer újra indításához szükséges információkat tartalmazza két kitüntetett rekordban – azaz két példányban – illetve további rekordjaiban az elvégzett tranzakciókról tartalmaz információkat. Ezen információk segítségével nyílik lehetőség egyes tranzakciók újbóli elvégzésére, illetve visszavonására. A megszakadt, illetve le nem zárt tranzakciók sikeres befejezését is közel száz százalékos valószínűséggel oldja meg az NTFS fájlrendszer.



Az NTFS fájlrendszer további előnyei

- Az alternatív, vagy többszörös adatfolyamok (ADS / Alternative Data Stream) teszik lehetővé azt, hogy egy fájl több adatfolyamhoz is kapcsolódhasson. Például a "001.txt" nevű fájl "001.txt:secret.txt" néven egy alternatív adatfolyamot is magába foglalhat. Az ADS név ismeretének hiányában ez az extra tartalom nem érhető el. Az alternatív adatfolyamok jelenlétére a fájl mérete közvetlenül nem is utal, így a szabad tárhelyet észrevétlenül csökkenti. Az alternatív adatfolyam elveszik, amennyiben az azt tartalmazó fájlt egy nem ADS képes fájlrendszer (pl. FAT32) használó tárolóra másoljuk.
- Unicode fájl nevek használata.
Maximum 255 karakter hosszúak lehetnek a fájlnevek – akár a pont és a szóköz karakterek is használhatók – a kis- és nagybetűket a fájlrendszer megkülönbözteti.
- Az indexelési lehetőség segítségével az adott tulajdonságú fájlok közvetlenül érhetők el.
- Dinamikus hibás szektor kezelés, menet közbeni tiltással. Amennyiben egy szektor hibássá válik, a rendszer azt a későbbiekben nem fogja adattárolásra felhasználni. A meghibásodott szektor tartalmát – amennyiben az még olvasható – a rendszer egy másik, ép szektorba másolja át.
- A kvóták segítségével nyílik lehetősége a rendszer adminisztrátorának arra, hogy az egyes felhasználók számára egyedi méretű lemezterületeket jelöljön ki. A felhasználók így természetesen már nem a fizikai tároló méretét látják a szabad- illetve a foglalt helyek ellenőrzésekor, hanem csupán a számukra kiosztott részt.
- Az NTFS az LZ77 algoritmus (amely a ZIP fájl formátumban is használatos) használatával a fájlok tömörítésére is képes. A tömörítés, azaz a be- illetve kicsomagolás egyaránt erőforrás igényes művelet, azaz csak akkor célszerű élni ezzel a lehetőséggel, ha megfelelően erős és viszonylag „kihasználatlan” CPU áll ehhez rendelkezésre. Szerverek esetében – ahol a CPU-k terhelése a kliensek igényeinek megfelelően tág határok között változhat – nem célszerű a tömörítéssel további erőforrásokat elvonni a rendszertől.
- Fájlok illetve könyvtárak – a felhasználó szempontjából átlátszó – titkosítását teszi lehetővé az EFS (Encrypting File System) használata. Ez a képesség csak az 5.0 (azaz a Windows 2000) és az azt követő verziókban érhető el, a korábbi verziókban nem áll rendelkezésre. Természetesen ez esetben sem szabad megfeledkezni arról, hogy a titkosítás is jelentős CPU erőforrásokat képes lekötöni.

Adattípusok és adatszerkezetek

- A kötet (Volume) egy elsődleges háttértár (HDD vagy SSD) egy logikai partíciója. A kötet formázása tulajdonképpen a köteten használatos logikai fájlrendszer kialakítása.
- Az adattárolás alapegysége, a helyfoglalási egység, a szektorcsoport (Cluster) [ZBR esetén nyilván zónacsoport]. A csoportképzés formázáskor kerül beállításra, a kettő hatványai szerint.
- Logical Cluster Number (LCN)
Az LCN az egy köteten nyilvántartott Clusterek sorszámait jelenti. A Cluster-eket 0-tól [az LCN 0 a kötet első Cluster-e, azaz a Boot Sector] n-ig folyamatosan számozzuk, az érték gyakorlati maximuma $2^{64}-1$. A rendszer így Cluster-ek sorozataként látja a meghajtót. Az LNC számot a Cluster méretével megszorozva kapjuk meg az adott LNC számhoz tartozó adat fizikai címét a meghajtón.
- Virtual Cluster Number (VCN)
A VCN az egy adott adatfolyamhoz (Stream) tartozó Cluster-ek számait jelenti, melyek így az adott fájlt reprezentálják. A VCN 0 az adott fájl első Cluster-ét jelenti. A VCN számok a potenciális töredezettség miatt nem feltétlenül mutatnak a fizikailag is egymást követő Cluster-ekre. Egy adott fájl fizikai címét a meghajtón a VCN és a LCN segítségével találja meg az NTFS fájlrendszer. Ehhez pontosan az LCN-ek VCN-ekhez történő hozzárendelését és nyilvántartását kell felhasználni, vagyis azt, hogy a fájl egy adott LCN-e, melyik VCN-el érhető el. A hozzárendelést természetesen fájlonként kell nyilvántartani.
- NTFS Metaadatok (Metadata)
Az NTFS Metadata a Master File Table rendszer szintű bejegyzéseinek az összefoglaló neve.

Master File Table (MFT)

A Master File Table azoknak a fájl rekordoknak sorozata, amelyek a fájlrendszer kezeléséhez, a benne tárolt fájlok eléréshez szükségesek. Az NTFS értelmezésében mindent, ami a lemezen található, azt fájlként kell definiálni és kezelni. A következő – az Master File Table-t bemutató – táblázatnak is értelem szerűen minden sora egy-egy fájlt jelent. A bejegyzésben szereplő fájlokat két kategóriába soroljuk: Metadata és Normal. A felhasználó közvetlenül csak a Normal kategóriához fér hozzá.

Egy fájl kezelésekor az alapadatokon, azaz a nevéen, kiterjesztésén és méretén túl a fájl egyéb jellegzetességei, tulajdonságai, azaz attribútumai (Attribute) is fontos szerepet töltenek be. Az NTFS fájlrendszerben egy fájl az attribútumai szerint lehet írásvédett fájl, rejtett fájl, archív fájl, rendszer fájl, tömörített fájl, index fájl, illetve az attribútumokban tárolódik a létrehozási és a módosítási idő is.

A hatékony, nem pazarló helykihasználás miatt a Master File Table nem az egész fájl rekordot, hanem annak csak az első 1kB-os darabját tartalmazza, így a Master File Table gyakorlatilag egy 1kB-os bejegyzéseket tartalmazó táblázat. A táblázat minden bejegyzése egy-egy fájl azonosít. A fájl egyedi azonosítója ezek után az a sorszám lesz, ami megmondja, hogy az Master File Table hányadik bejegyzése tartozik hozzá. A bejegyzések minden, a fájlra vonatkozó információt tartalmaznak, azonban szerkezetük nem kötött.

Az Master File Table első 16db Metadata bejegyzése a rendszerfájlok, melyek neve „\$” jellel kezdődik. Ezek a felhasználó elől rejtve vannak, ezeket az operációs rendszer használja. Ezt követi 8db jövőbeli fejlesztésre fenntartott, azaz előzetesen lefoglalt (Reserved) bejegyzés, majd ezek után következnek a felhasználói fájlok és könyvtárak, azaz a Normal fájlok listája.

Az ábrán csak a \$Root tartalma szerepel kibontva, de mindegyik Metadata bejegyzés saját adatszerkezettel és további egyedi attribútumokkal rendelkezik.

Master File Table		
	Metadata	Function
0	\$MFT	MFT, an index on every file
1	\$MFT mirror	Part image of MFT (usually first 4 records)
2	\$LogFile	Transactional logging file
3	\$Volume	Serial number, creation time, dirty flag
4	\$AttrDef	Attribute definitions
5	\$Root	Root directory called '.' is an ordinary directory
6	\$Bitmap	Volume's cluster map (free vs. in-use)
7	\$Boot	Volume boot record
8	\$BadClus	Bad cluster list on the volume
9	\$Secure	Security descriptions used by the volume
10	\$UpCase	For short and long filename comparison
11	\$Extend	Directory for more metafiles
12	\$Extend\\$\\$Reparse	Reparse Point file
13	\$Extend\UsnJrnl	Log changing file
14	\$Extend\Quota	Quote management file
15	\$Extend\ObjId	Object ID file
16-23	Reserved	Reserved
24-	User Files, Directories	User Files, Directories
n-1	User Files, Directories	User Files, Directories
n	User Files, Directories	User Files, Directories

File Record (1kB)		
Standard information		
Filename		
Data stream	Extents	
Attribute #1	Resident	
Attribute #2	Extents	
...	...	

Az attribútumok tárolási módjai

- Rezidens tárolás

Ebben az esetben közvetlenül az attribútum fejléce (Header) után tárolódik az attribútum értékét tartalmazó információ. Jellemzően a kisebb méretű attribútumok tárolódnak ezen a módon, melyek képesek a Master File Table rekordjába beférni.

Rezidens módon tárolt attribútum például normál esetben a fájlnev, az időbélyeg, illetve a biztonsági és a jogosultsági információk.

- Nem rezidens tárolás

Ebben az esetben az attribútum fejléce (Header) után csak az attribútum értékét tartalmazó adatok helye, pontosabban címe van tárolva. Jellemzően azok az attribútumok tárolódnak ezen a módon, melyek a nagy méretük vagy potenciális méretnövekedésük miatt eleve nem férnek be a Master File Table egy rekordjába, így tárolásukhoz további Cluster-eket kell felhasználni. A vonatkozó Cluster-ek nyilvántartásához egy indextáblára van szükség, mely a szükséges VCN-LCN hozzárendelést tartalmazza. A nem rezidens attribútum tárolási mód velejárója a tördelődés.

Nem rezidens módon tárolt attribútum használatos például a titkosított, kódolt, illetve tömörített állományok esetén.

Állományok illetve könyvtárak azonosítása egy 64-bites referenciaszámmal történik, mely két részből áll. Az első 48-bit mutatja meg azt, hogy hányadik Master File Table rekordról van szó, míg a következő 16-bit azt mutatja meg, hogy az adott állományt hányszor használták.

2. Hasznos linkek

[Mi az NTFS \(Microsoft, angol nyelvű\)](#)

[Hogyan működik az NTFS \(Microsoft, angol nyelvű\)](#)

[Az NTFS és a FAT típusú fájlrendszerek összehasonlítása \(Microsoft\)](#)